[BACKEND (API)] GroupWorkAPI\.dockerignore

```
**/.classpath
**/.dockerignore
**/.env
**/.git
**/.gitignore
**/.project
**/.settings
**/.toolstarget
**/.vs
**/.vscode
**/*.*proj.user
**/*.dbmdl
**/*.jfm
**/azds.yaml
**/bin
**/charts
**/docker-compose*
**/Dockerfile*
**/node_modules
**/npm-debug.log
**/obj
**/secrets.dev.yaml
**/values.dev.yaml
LICENSE
README.md
!**/.gitignore
!.git/HEAD
!.git/config
!.git/packed-refs
!.git/refs/heads/**
```

[BACKEND (API)] GroupWorkAPI\.gitignore

```
# Сборка .NET
bin/
obj/

# Персональные настройки Visual Studio и пользователей
.vs/
*.user
*.suo
*.userosv

# Настройки Rider и VS Code
.idea/
.vscode/

# Логи
*.log
```

[BACKEND (API)] GroupWorkAPI\docker-compose.yml

```
services:
  db:
    image: postgres:16-alpine
    restart: always
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: rafael
      POSTGRES_DB: postgres
    ports:
      - "5434:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./init-scripts/postgres_localhost-2026_09_08_22_10_44-dump.sql:/docker-entrypoint-initdb.d/init.sql

  api:
    build:
      context: ./GroupWorkAPI
      dockerfile: Dockerfile
```

```
    restart: always
    ports:
      - "5000:8080"
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - OpenRouter__ApiKey=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
      - OpenRouter__Model=nvidia/nemotron-3-super-120b-a12b:free
      - ConnectionStrings__DefaultConnection=Host=db;Port=5432;Database=postgres;Username=postgres;Password=rafael
    depends_on:
      - db

volumes:
  pgdata:
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI.slnx

```
<Solution>
  <Project Path="GroupWorkAPI/GroupWorkAPI.csproj" />
</Solution>
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\appsettings.Development.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5434;Database=postgres;Username=postgres;Password=rafael"
  },
  "OpenRouter": {
    "ApiKey": "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",
    "Model": "stealth/space-bunny-alpha"
  }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Dockerfile

```
# См. статью по ссылке https://aka.ms/customizecontainer, чтобы узнать как настроить контейнер отладки и как Visual Studio использует этот Dockerfile для создания образов для ускорения отладки.

# Этот этап используется при запуске из VS в быстром режиме (по умолчанию для конфигурации отладки)
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS base
USER $APP_UID
WORKDIR /app
EXPOSE 8080
EXPOSE 8081


# Этот этап используется для сборки проекта службы
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src
COPY ["GroupWorkAPI.csproj", "."]
RUN dotnet restore "./GroupWorkAPI.csproj"
COPY . .
```

```
WORKDIR "/src/."
RUN dotnet build "./GroupWorkAPI.csproj" -c $BUILD_CONFIGURATION -o /app/build

# Этот этап используется для публикации проекта службы, который будет скопирован на последний этап
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "./GroupWorkAPI.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

# Этот этап используется в рабочей среде или при запуске из VS в обычном режиме (по умолчанию, когда конфигурация отладки не используется)
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "GroupWorkAPI.dll"]
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\GroupWorkAPI.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UserSecretsId>7c73a6a9-fd75-4efd-954d-c7cb60aade97</UserSecretsId>
    <DockerDefaultTargetOS>Linux</DockerDefaultTargetOS>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="10.0.9" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Proxies" Version="10.0.11" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="10.0.11">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.VisualStudio.Azure.Containers.Tools.Targets" Version="1.23.0" />
    <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.3" />
    <PackageReference Include="Swashbuckle.AspNetCore" Version="10.2.3" />
    <PackageReference Include="Swashbuckle.AspNetCore.SwaggerGen" Version="10.2.3" />
    <PackageReference Include="Swashbuckle.AspNetCore.SwaggerUI" Version="10.2.3" />
  </ItemGroup>

</Project>
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\GroupWorkAPI.http

```
@GroupWorkAPI_HostAddress = http://localhost:5003

GET {{GroupWorkAPI_HostAddress}}/weatherforecast/
Accept: application/json

###
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Program.cs

```
using GroupWorkAPI.Model;
using GroupWorkAPI.Services;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
var connectionString = builder.Configuration.GetConnectionString("DefaultConnection");
builder.Services.AddDbContext<PostgresContext>(options =>
    options.UseLazyLoadingProxies().UseNpgsql(connectionString));

// Регистрация сервисов
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddScoped<AiService>();
```

```
var myAllowSpecificOrigins = "_myAllowSpecificOrigins";
builder.Services.AddCors(options =>
{
    options.AddPolicy(myAllowSpecificOrigins, policy =>
    {
        policy.WithOrigins("http://localhost:3000", "http://localhost:5000", "http://localhost:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

// Pipeline
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI(options =>
    {
        options.SwaggerEndpoint("/swagger/v1/swagger.json", "v1");
        options.RoutePrefix = string.Empty;
    });
}

app.UseCors(myAllowSpecificOrigins);
app.UseStaticFiles();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\AiController.cs

```
using System.Text.Json;
using System.Text.RegularExpressions;
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;
using GroupWorkAPI.Services;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AiController : ControllerBase
    {
        private readonly PostgresContext _context;
        private readonly AiService _aiService;

        public AiController(PostgresContext context, AiService aiService)
        {
            _context = context;
            _aiService = aiService;
        }

        [HttpPost("Coach")]
        public async Task<IActionResult> Coach([FromBody] AiCoachRequestDTO request)
        {
            if (string.IsNullOrWhiteSpace(request?.Message))
            {
                return Ok(new AiCoachResponseDTO { Reply = "Пожалуйста, задайте вопрос тренеру." });
            }

            try
            {
                // Загружаем список товаров для передачи в ИИ
                var availableProducts = await _context.Products
                    .Select(p => new { p.Id, p.Name, p.BasePrice })
                    .ToListAsync();

                    string productsList = string.Join("\n", availableProducts.Select(p => $"ID {p.Id}: {p.Name} (Цена: {p.BasePrice}
руб)"));

                  string systemPrompt = @"Ты — профессиональный спортивный тренер и эксперт по подбору экипировки ASICS ('AI Running
```

```
Coach').
Твоя задача — дать профессиональную консультацию клиенту на русском языке.
ОБЯЗАТЕЛЬНО выбери из списка доступных товаров 1-3 подходящие модели и укажи их точные ID.

Формат ответа СТРОГО следующий валидный JSON (без markdown-тегов и без ```json):
{
  ""reply"": ""Твой развернутый ответ, почему именно эти модели подходят под параметры человека"",
  ""productIds"": [id1, id2]
}";

                string userPrompt = $"Список доступных товаров в магазине:\n{productsList}\n\nВопрос покупателя:\n{request.Message}";

                string? aiResponse = await _aiService.AskAiAsync(userPrompt, systemPrompt);

                if (string.IsNullOrWhiteSpace(aiResponse))
                {
                    return Ok(new AiCoachResponseDTO
                    {
                        Reply = "К сожалению, сервис ИИ не ответил на запрос. Пожалуйста, попробуйте еще раз."
                    });
                }

                // Очистка ответа от возможных markdown-оберток
                string cleanJson = Regex.Replace(aiResponse, @"```json\s*", "");
                cleanJson = Regex.Replace(cleanJson, @"```\s*", "").Trim();

                using var doc = JsonDocument.Parse(cleanJson);
                var root = doc.RootElement;

                string reply = root.TryGetProperty("reply", out var r) ? r.GetString() ?? "" : aiResponse;
                List<int> ids = new();

                if (root.TryGetProperty("productIds", out var idsElement))
                {
                    foreach (var item in idsElement.EnumerateArray())
                    {
                        if (item.TryGetInt32(out int id)) ids.Add(id);
                    }
                }

                // Прикрепляем карточки тех товаров, которые выбрал ИИ
                var selectedByAi = await _context.Products
                    .Where(p => ids.Contains(p.Id))
                    .Select(p => new AiProductRecommendationDTO
                    {
                        ProductId = p.Id,
                        ProductName = p.Name,
                        Price = p.BasePrice,
                        Reason = "Выбор тренера ASICS"
                    })
                    .ToListAsync();

                return Ok(new AiCoachResponseDTO
                {
                    Reply = reply,
                    Recommendations = selectedByAi
                });
            }
            catch (Exception ex)
            {
                return Ok(new AiCoachResponseDTO
                {
                    Reply = $"Ошибка обработки ответа ИИ: {ex.Message}",
                    Recommendations = new()
                });
            }
        }

        [HttpPost("GenerateDescription")]
        public async Task<IActionResult> GenerateDescription([FromBody] AiGenerateDescriptionDTO dto)
        {
            if (string.IsNullOrWhiteSpace(dto.Name))
            {
                return BadRequest("Укажите модель.");
            }
```

```
            string systemPrompt = "Ты — спортивный маркетолог ASICS. Напиши профессиональное продающее описание товара на русском языке (3 содержательных предложения) без кавычек и вступительных фраз.";
             string userPrompt = $"Модель: {dto.Name}, Категория: {dto.Category}. Технологии: {dto.Technologies ?? "PureGEL, FF BLAST"}";

            string? description = await _aiService.AskAiAsync(userPrompt, systemPrompt);

            return Ok(new { description = (description ?? "").Trim('"', ' ') });
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\AnalyticsController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AnalyticsController : ControllerBase
    {
        private readonly PostgresContext _context;

        public AnalyticsController(PostgresContext context)
        {
            _context = context;
        }

        [HttpGet("GetMostLikedProducts")]
        public async Task<IActionResult> GetMostLikedProducts()
        {
            var data = await _context.ProductFavorites
                .GroupBy(f => f.ProductId)
                .Select(g => new
                {
                    ProductId = g.Key,
                    ProductName = _context.Products
                        .Where(p => p.Id == g.Key)
                        .Select(p => p.Name)
                        .FirstOrDefault() ?? "Неизвестный товар",
                    LikesCount = g.Count()
                })
                .OrderByDescending(x => x.LikesCount)
                .Take(10)
                .ToListAsync();

            return Ok(data);
        }

        [HttpGet("GetMostLikedVariants")]
        public async Task<IActionResult> GetMostLikedVariants()
        {
            var data = await _context.VariantFavorites
                .GroupBy(f => f.ProductVariantId)
                .Select(g => new
                {
                    ProductVariantId = g.Key,
                    VariantName = _context.ProductVariants
                        .Where(v => v.Id == g.Key)
                        .Select(v => v.Product.Name + " (Цвет: " + v.Color.Name + ", Размер: " + v.Size + ")")
                        .FirstOrDefault() ?? "Неизвестная вариация",
                    LikesCount = g.Count()
                })
                .OrderByDescending(x => x.LikesCount)
                .Take(10)
                .ToListAsync();

            return Ok(data);
        }
        [HttpPost("ToggleProductFavorite")]
```

```
public async Task<IActionResult> ToggleProductFavorite([FromBody] ProductFavoriteDTO dto)
{
    var existing = await _context.ProductFavorites
        .FirstOrDefaultAsync(f => f.UserId == dto.UserId && f.ProductId == dto.ProductId);

    if (existing != null)
    {
        _context.ProductFavorites.Remove(existing);
        await _context.SaveChangesAsync();
        return Ok(new { isLiked = false });
    }
    else
    {
        int id = await _context.ProductFavorites.AnyAsync()
            ? await _context.ProductFavorites.MaxAsync(x => x.Id) + 1
            : 1;

        ProductFavorite newFavorite = new()
        {
            Id = id,
            UserId = dto.UserId,
            ProductId = dto.ProductId
        };

        _context.ProductFavorites.Add(newFavorite);
        await _context.SaveChangesAsync();
        return Ok(new { isLiked = true });
    }
}

[HttpPost("ToggleVariantFavorite")]
public async Task<IActionResult> ToggleVariantFavorite([FromBody] VariantFavoriteDTO dto)
{
    var existing = await _context.VariantFavorites
        .FirstOrDefaultAsync(f => f.UserId == dto.UserId && f.ProductVariantId == dto.ProductVariantId);

    if (existing != null)
    {
        _context.VariantFavorites.Remove(existing);
        await _context.SaveChangesAsync();
        return Ok(new { isLiked = false });
    }
    else
    {
        int id = await _context.VariantFavorites.AnyAsync()
            ? await _context.VariantFavorites.MaxAsync(x => x.Id) + 1
            : 1;

        VariantFavorite newFavorite = new()
        {
            Id = id,
            UserId = dto.UserId,
            ProductVariantId = dto.ProductVariantId
        };

        _context.VariantFavorites.Add(newFavorite);
        await _context.SaveChangesAsync();
        return Ok(new { isLiked = true });
    }
}

[HttpGet("GetUserProductFavorites/{userId}")]
public async Task<IActionResult> GetUserProductFavorites(int userId)
{
    var productIds = await _context.ProductFavorites
        .Where(f => f.UserId == userId)
        .Select(f => f.ProductId)
        .ToListAsync();

    return Ok(productIds);
}

[HttpGet("GetUserVariantFavorites/{userId}")]
public async Task<IActionResult> GetUserVariantFavorites(int userId)
{
```

```
            var variantIds = await _context.VariantFavorites
                .Where(f => f.UserId == userId)
                .Select(f => f.ProductVariantId)
                .ToListAsync();

            return Ok(variantIds);
        }
        [HttpGet("GetActualUserProductFavorites/{userId}")]
        public async Task<IActionResult> GetActualUserProductFavorites(int userId)
        {
            var data = await _context.ProductFavorites
                .Where(f => f.UserId == userId)
                .Select(f => new
                {
                    ProductId = f.ProductId,
                    ProductName = _context.Products
                        .Where(p => p.Id == f.ProductId)
                        .Select(p => p.Name)
                        .FirstOrDefault() ?? "Неизвестный товар"
                })
                .ToListAsync();

            return Ok(data);
        }

        [HttpGet("GetActualUserVariantFavorites/{userId}")]
        public async Task<IActionResult> GetActualUserVariantFavorites(int userId)
        {
            var data = await _context.VariantFavorites
                .Where(f => f.UserId == userId)
                .Select(f => new
                {
                    ProductVariantId = f.ProductVariantId,
                    ProductId = _context.ProductVariants
                        .Where(v => v.Id == f.ProductVariantId)
                        .Select(v => v.ProductId)
                        .FirstOrDefault(),
                    VariantName = _context.ProductVariants
                        .Where(v => v.Id == f.ProductVariantId)
                        .Select(v => v.Product.Name)
                        .FirstOrDefault() ?? "Неизвестная модель",
                    Color = _context.ProductVariants
                        .Where(v => v.Id == f.ProductVariantId)
                        .Select(v => v.Color.Name)
                        .FirstOrDefault() ?? "Не указан",
                    Size = _context.ProductVariants
                        .Where(v => v.Id == f.ProductVariantId)
                        .Select(v => v.Size)
                        .FirstOrDefault()
                })
                .ToListAsync();

            return Ok(data);
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\BrandController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class BrandController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;
        [HttpGet("GetAllBrands")]
        public async Task<IActionResult> GetAllBrands()
        {
```

```
    List<Brand> brands = await _context.Brands.ToListAsync();
    if(brands.Count == 0)
    {
        return NotFound("Бренды не найдены!");
    }
    List<BrandDTO> brandDTO = [];
    foreach(var  brand in brands)
    {
        brandDTO.Add(brand.ToDto());
    }
    return Ok(brandDTO);
}
[HttpGet("GetBrand/{id}")]
public async Task<IActionResult> GetBrand(int id)
{
    Brand? brand = await _context.Brands.FirstOrDefaultAsync(x=>x.Id == id);
    if(brand == null)
    {
        return NotFound("Бренд не найден!");
    }
    return Ok(brand.ToDto());
}
[HttpGet("GetProductBrand")]
public async Task<IActionResult> GetProductBrand(int id)
{
    Brand? brand = await _context.Brands.FirstOrDefaultAsync(x=>x.Id == id);
    if(brand == null)
    {
        return NotFound("Бренд не найден!");
    }
    List<Product> products = await _context.Products.Include(x=>x.ProductImages).ToListAsync();
    products = [..products.Where(x=>x.BrandId==id)];
    if(products.Count == 0)
    {
        return NotFound("Товаров с данным брендом не найдено!");
    }
    List<ProductDTO> productDTO = [];
    foreach(var product in products)
    {
        productDTO.Add(product.ToDto());
    }
    return Ok(productDTO);
}
[HttpPost("AddBrand")]
public async Task<IActionResult> AddBrand([FromBody] BrandDTO brandDTO)
{
    if(await _context.Brands.AnyAsync(x=>x.Name == brandDTO.Name))
    {
        return BadRequest("Бренд с такимм названием уже существует!");
    }
    int id = await _context.Brands.AnyAsync() ? await _context.Brands.MaxAsync(x => x.Id) + 1 : 1;
    Brand brand = new()
    {
        Id = id,
        Name = brandDTO.Name,
    };
    _context.Brands.Add(brand);
    await _context.SaveChangesAsync();
    return Ok(brand.ToDto());
}
[HttpPut("EditBrand")]
public async Task<IActionResult> EditBrand([FromBody] BrandDTO brandDTO)
{
    Brand? brand = await _context.Brands.FirstOrDefaultAsync(x=>x.Id == brandDTO.Id);
    if (brand == null)
    {
        return NotFound("Бренд не найден!");
    }
    if(await _context.Brands.AnyAsync(x => x.Name == brandDTO.Name))
    {
        return BadRequest("Бренд уже существует!");
    }
    brand.Name = brandDTO.Name;
    _context.Brands.Update(brand);
    await _context.SaveChangesAsync();
```

```
            return Ok(brand.ToDto());
        }
        [HttpDelete("DeleteBrand")]
        public async Task<IActionResult> DeleteBrand(int id)
        {
            Brand? brand = await _context.Brands.FirstOrDefaultAsync(x=>x.Id==id);
            if (brand == null)
            {
                return NotFound("Бренд не найден");
            }
            _context.Brands.Remove(brand);
            await _context.SaveChangesAsync();
            return Ok();
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\CartItemController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CartItemController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;
        [HttpGet("GetAllCartItems")]
        public async Task<IActionResult> GetAllCartItems()
        {
            List<CartItem> cartitems = await _context.CartItems
                .Include(c => c.ProductVariant)
                    .ThenInclude(v => v.Product)
                .Include(c => c.ProductVariant)
                    .ThenInclude(v => v.Color)
                .ToListAsync();
            if (cartitems.Count == 0)
            {
                return NotFound("Данных в корзине не найдено");
            }
            List<CartItemDTO> cartItemDTO = [];
            foreach (var cart in cartitems)
            {
                cartItemDTO.Add(cart.ToDto());
            }
            return Ok(cartItemDTO);
        }
        [HttpGet("GetCartItem/{id}")]
        public async Task<IActionResult> GetCartItem(int id)
        {
            var cartitems = await _context.CartItems
                .Where(x => x.UserId == id)
                .Include(c => c.ProductVariant)
                    .ThenInclude(v => v.Product)
                .Include(c => c.ProductVariant)
                    .ThenInclude(v => v.Color)
                .ToListAsync();
            List<CartItemDTO> cartItemDTOs = [];
            if (cartitems.Count == 0)
            {
                return Ok(new List<CartItemDTO>());
            }
            foreach (var cart in cartitems)
            {
                cartItemDTOs.Add(cart.ToDto());
            }
            return Ok(cartItemDTOs);
        }

        [HttpGet("GetCartForSum/{id}")]
```

```
        public async Task<IActionResult> GetCartForSum(int id)
        {
            var cartitems = await _context.CartItems
                .FirstOrDefaultAsync(x => x.UserId == id);
            if (cartitems == null)
            {
                return NotFound("Не найдено");
            }
            return Ok(cartitems.ToDto());
        }

        [HttpPost("AddCartItem")]
        public async Task<IActionResult> AddCartItem([FromBody] CartItemDTO cartItemDTO)
        {
            User? user = await _context.Users.FirstOrDefaultAsync(x => x.Id == cartItemDTO.UserId);
            if (user == null)
            {
                return NotFound("Пользователь не найден");
            }
                            ProductVariant?  productVariant  =  await  _context.ProductVariants.FirstOrDefaultAsync(x  =>  x.Id  ==
cartItemDTO.ProductVariantId);
            if (productVariant == null)
            {
                return NotFound("Вариации продукта не найдено");
            }
            if (productVariant.StockQuantity == 0)
            {
                return BadRequest("Количество товара закончилось!");
            }
             CartItem? cart = await _context.CartItems.FirstOrDefaultAsync(x=>x.UserId == cartItemDTO.UserId && x.ProductVariantId ==
cartItemDTO.ProductVariantId);
            if (cart != null)
            {
                cart.Quantity += 1;
                _context.CartItems.Update(cart);
                //return Ok(cart.ToDto());
            }
            else
            {
                int id = await _context.CartItems.AnyAsync() ? await _context.CartItems.MaxAsync(x => x.Id) + 1 : 1;
                cart = new()
                {
                    Id = id,
                    UserId = user.Id,
                    ProductVariantId = productVariant.Id,
                    Quantity = 1
                };
                _context.CartItems.Add(cart);
            }
            productVariant.StockQuantity -= 1;
            _context.ProductVariants.Update(productVariant);
            await _context.SaveChangesAsync();
            return Ok(cart.ToDto());
        }
        [HttpPut("SubtractCartItem")]
        public async Task<IActionResult> SubtractCartItem([FromBody] CartItemDTO cartItemDTO)
        {
            User? user = await _context.Users.FirstOrDefaultAsync(x => x.Id == cartItemDTO.UserId);
            if (user == null)
            {
                return NotFound("Пользователь не найден");
            }
                            ProductVariant?  productVariant  =  await  _context.ProductVariants.FirstOrDefaultAsync(x  =>  x.Id  ==
cartItemDTO.ProductVariantId);
            if (productVariant == null)
            {
                return NotFound("Вариации продукта не найдено");
            }
             CartItem? cart = await _context.CartItems.FirstOrDefaultAsync(x => x.UserId == cartItemDTO.UserId && x.ProductVariantId ==
cartItemDTO.ProductVariantId);
            if (cart == null)
            {
                return BadRequest("Данный товар не находится в корзине");
            }
            cart.Quantity -= 1;
```

```
            if(cart.Quantity > 0)
            {
                _context.CartItems.Update(cart);
            }
            else
            {
                _context.CartItems.Remove(cart);
            }
            productVariant.StockQuantity += 1;
            _context.ProductVariants.Update(productVariant);
            await _context.SaveChangesAsync();
            return Ok(cart.ToDto());
        }
        [HttpDelete("DeleteCartItem/{id}")]
        public async Task<IActionResult> DeleteCartItem(int id)
        {
            CartItem? cartitem = await _context.CartItems.FirstOrDefaultAsync(x => x.Id == id);
            if (cartitem == null)
            {
                return NotFound("Элемент в корзине не найден");
            }
            ProductVariant? productVariant = await _context.ProductVariants
                        .FirstOrDefaultAsync(x => x.Id == cartitem.ProductVariantId);

            if (productVariant != null)
            {
                productVariant.StockQuantity += cartitem.Quantity;
                _context.ProductVariants.Update(productVariant);
            }
            else
            {
                return NotFound("Вариация продукта, привязанная к корзине, не найдена");
            }
            _context.CartItems.Remove(cartitem);
            await _context.SaveChangesAsync();
            return Ok();
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\CategoryController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Runtime.Intrinsics.Arm;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CategoryController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;
        [HttpGet("GetAllCategory")]
        public async Task<IActionResult> GetAllCategory()
        {
            List<Category> categories = await _context.Categories.ToListAsync();
            if(categories.Count == 0)
            {
                return NotFound("Категорий не найдено");
            }
            List<CategoryDTO> categoriesDTO = [];
            foreach(var category in categories)
            {
                categoriesDTO.Add(category.ToDto());
            }
            return Ok(categoriesDTO);
        }
        [HttpGet("GetCategory/{id}")]
        public async Task<IActionResult> GetCategory(int id)
        {
            Category? category = await _context.Categories.FirstOrDefaultAsync(x=> x.Id == id);
```

```
            if(category == null)
            {
                return NotFound("Категория не найдена!");
            }
            return Ok(category.ToDto());
        }
        [HttpPost("AddCategory")]
        public async Task<IActionResult> AddCategory([FromBody] CategoryDTO categoryDTO)
        {
            if(await _context.Categories.AnyAsync(x=>x.Name == categoryDTO.Name))
            {
                return BadRequest("Категория с таким названием уже существует!");
            }
            int id = await _context.Categories.AnyAsync()? await _context.Categories.MaxAsync(x=>x.Id)+1:1;
            Category category = new()
            {
                Id = id,
                Name = categoryDTO.Name,
            };
            _context.Categories.Add(category);
            await _context.SaveChangesAsync();
            return Ok(category.ToDto());
        }
        [HttpPut("EditCategory")]
        public async Task<IActionResult> EditCategory([FromBody] CategoryDTO categoryDTO)
        {
            Category? category = await _context.Categories.FirstOrDefaultAsync(x=>x.Id == categoryDTO.Id);
            if(category == null)
            {
                return NotFound("Категория не найдена!");
            }
            if(await _context.Categories.AnyAsync(x=>x.Name==categoryDTO.Name))
            {
                return BadRequest("Категория уже существует!");
            }
            category.Name = categoryDTO.Name;
            _context.Categories.Update(category);
            await _context.SaveChangesAsync();
            return Ok(category.ToDto());
        }
        [HttpDelete("DeleteCategory")]
        public async Task<IActionResult> DeleteCategory(int id)
        {
            Category? category = await _context.Categories.FirstOrDefaultAsync(x=>x.Id == id);
            if(category == null)
            {
                return NotFound("Категория не найдена");
            }
            _context.Categories.Remove(category);
            await _context.SaveChangesAsync();
            return Ok();
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\ColorController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ColorController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;
        [HttpGet("GetAllColor")]
        public async Task<IActionResult> GetAllColor()
        {
            List<Color> colors = await _context.Colors.ToListAsync();
            if(colors.Count == 0)
```

```
            {
                return NotFound("Цветов не найдено!");
            }
            List<ColorDTO> colorDTO = [];
            foreach(var color in colors)
            {
                colorDTO.Add(color.ToDto());
            }
            return Ok(colorDTO);
        }
        [HttpGet("GetColor/{id}")]
        public async Task<IActionResult> GetCategory(int id)
        {
            Color? color = await _context.Colors.FirstOrDefaultAsync(x => x.Id == id);
            if (color == null)
            {
                return NotFound("Цвет не найдена!");
            }
            return Ok(color.ToDto());
        }
        [HttpPost("AddColor")]
        public async Task<IActionResult> AddColor([FromBody] ColorDTO colorDTO)
        {
            if (await _context.Colors.AnyAsync(x => x.Name == colorDTO.Name))
            {
                return BadRequest("Цвет с таким названием уже существует!");
            }
            int id = await _context.Colors.AnyAsync() ? await _context.Colors.MaxAsync(x => x.Id) + 1 : 1;
            Color color = new()
            {
                Id = id,
                Name = colorDTO.Name,
            };
            _context.Colors.Add(color);
            await _context.SaveChangesAsync();
            return Ok(color.ToDto());
        }
        [HttpPut("EditColor")]
        public async Task<IActionResult> EditColor([FromBody] ColorDTO colorDTO)
        {
            Color? color = await _context.Colors.FirstOrDefaultAsync(x => x.Id == colorDTO.Id);
            if (color == null)
            {
                return NotFound("Цвет не найдена!");
            }
            if (await _context.Colors.AnyAsync(x => x.Name == colorDTO.Name))
            {
                return BadRequest("Цвет уже существует!");
            }
            color.Name = colorDTO.Name;
            _context.Colors.Update(color);
            await _context.SaveChangesAsync();
            return Ok(color.ToDto());
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\ImageController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Internal;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Text.RegularExpressions;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ImageController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;
```

```
        [HttpGet("GetImageAll")]
        public async Task<IActionResult> GetImageAll()
        {
            List<ProductImage> productImages = await _context.ProductImages.ToListAsync();
            if (productImages.Count == 0)
            {
                return NotFound("Фотографий не найдено");
            }
            List<ProductImageDTO> productImagesDTO = [];
            foreach (var productImage in productImages)
            {
                productImagesDTO.Add(productImage.ToDto());
            }
            return Ok(productImagesDTO);
        }

        [HttpGet("GetProductImage/{id}")]
        public async Task<IActionResult> GetProductImage(int id)
        {
            ProductImage? productImage = await _context.ProductImages.FirstOrDefaultAsync(x => x.ProductId == id);
            if (productImage == null)
            {
                return NotFound("Фотографий не найдено!");
            }
            return Ok(productImage.ToDto());
        }

        [HttpGet("GetVariantImage/{id}")]
        public async Task<IActionResult> GetVariantImage(int id)
        {
            var variantImage = await _context.VariantImages.FirstOrDefaultAsync(x => x.ProductVariantId == id);
            if (variantImage == null) return NotFound("Фото вариации не найдено");
            return Ok(variantImage.ToDto());
        }

        [HttpGet("GetProductImageByVariant/{variantId}")]
        public async Task<IActionResult> GetProductImageByVariant(int variantId)
        {
            var variant = await _context.ProductVariants.FirstOrDefaultAsync(v => v.Id == variantId);
            if (variant == null)
            {
                return NotFound("Вариация товара не найдена!");
            }
            ProductImage? productImage = await _context.ProductImages
                .FirstOrDefaultAsync(x => x.ProductId == variant.ProductId);
            if (productImage == null)
            {
                return NotFound("Фотография для этого товара не найдена!");
            }
            return Ok(productImage.ToDto());
        }

        [HttpPost("UpLoadProductImage")]
        [RequestSizeLimit(15_000_000)]
          public async Task<IActionResult> UpLoadProductImage(IFormFile? file, [FromForm] int productId, [FromForm] string? category,
[FromForm] string? name)
        {
            // Поддерживаем как прямую передачу, так и вложенный объект File
            var formFile = file ?? Request.Form.Files["File"] ?? Request.Form.Files["file"];
            if (formFile == null || formFile.Length == 0)
            {
                return BadRequest("Файл не передан или пустой");
            }

            int pId = productId;
            if (pId == 0 && int.TryParse(Request.Form["productDto.Id"], out int parsedId)) pId = parsedId;
            if (pId == 0 && int.TryParse(Request.Form["productDto[id]"], out int parsedId2)) pId = parsedId2;

                                    string   prodCategory   =   category   ??   Request.Form["productDto.Category"].ToString()   ??
Request.Form["productDto[category]"].ToString() ?? "Product";
               string prodName = name ?? Request.Form["productDto.Name"].ToString() ?? Request.Form["productDto[name]"].ToString() ??
"Item";

            Product? product = await _context.Products.FirstOrDefaultAsync(x => x.Id == pId);
            if (product == null)
```

```
            {
                return NotFound($"Товар с ID {pId} не найден");
            }

            // ВАЖНО: Гарантируем, что папка существует на диске (защита от 500 ошибки)
            string dirPath = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot", "images");
            if (!Directory.Exists(dirPath))
            {
                Directory.CreateDirectory(dirPath);
            }

            string ext = Path.GetExtension(formFile.FileName);
            string safeName = Regex.Replace(prodName, @"[^\w\.-]", "_");
            string fileName = $"{pId}-{prodCategory}-{safeName}{ext}";
            string fullPath = Path.Combine(dirPath, fileName);

            using (FileStream stream = new(fullPath, FileMode.Create))
            {
                await formFile.CopyToAsync(stream);
            }

            int imgId = await _context.ProductImages.AnyAsync() ? await _context.ProductImages.MaxAsync(x => x.Id) + 1 : 1;
            ProductImage productImage = new()
            {
                Id = imgId,
                ProductId = product.Id,
                ImageUrl = fileName,
                IsMain = true,
            };

            _context.ProductImages.Add(productImage);
            await _context.SaveChangesAsync();
            return Ok(productImage.ToDto());
        }

        [HttpPost("UpLoadProductVariantImage")]
        [RequestSizeLimit(15_000_000)]
           public async Task<IActionResult> UpLoadProductVariantImage(IFormFile? file, [FromForm] int variantId, [FromForm] string?
color, [FromForm] string? size)
        {
            var formFile = file ?? Request.Form.Files["File"] ?? Request.Form.Files["file"];
            if (formFile == null || formFile.Length == 0)
            {
                return BadRequest("Файл не передан или пустой");
            }

            int vId = variantId;
            if (vId == 0 && int.TryParse(Request.Form["variant.Id"], out int pId)) vId = pId;
            if (vId == 0 && int.TryParse(Request.Form["variant[id]"], out int pId2)) vId = pId2;

            ProductVariant? productVariant = await _context.ProductVariants.FirstOrDefaultAsync(x => x.Id == vId);
            if (productVariant == null)
            {
                return NotFound("Вариация продукта не найдена");
            }

            // ВАЖНО: Создаем папку imagesVariant (защита от 500 ошибки)
            string dirPath = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot", "imagesVariant");
            if (!Directory.Exists(dirPath))
            {
                Directory.CreateDirectory(dirPath);
            }

            string vColor = color ?? Request.Form["variant.Color"].ToString() ?? "Color";
            string vSize = size ?? Request.Form["variant.Size"].ToString() ?? "Size";

            string ext = Path.GetExtension(formFile.FileName);
            string fileName = $"{vId}-{vColor}-{vSize}{ext}";
            string fullPath = Path.Combine(dirPath, fileName);

            using (FileStream stream = new(fullPath, FileMode.Create))
            {
                await formFile.CopyToAsync(stream);
            }
```

```
            int id = await _context.VariantImages.AnyAsync() ? await _context.VariantImages.MaxAsync(x => x.Id) + 1 : 1;
            VariantImage variantImage = new()
            {
                Id = id,
                ProductVariantId = productVariant.Id,
                ImagePath = fileName,
            };

            _context.VariantImages.Add(variantImage);
            await _context.SaveChangesAsync();
            return Ok(variantImage.ToDto());
        }

        [HttpDelete("DeleteProductImage/{id}")]
        public async Task<IActionResult> DeleteProductImage(int id)
        {
            ProductImage? productImage = await _context.ProductImages.FirstOrDefaultAsync(x => x.Id == id);
            if (productImage == null)
            {
                return NotFound("Фотографий не найдено!");
            }
            _context.ProductImages.Remove(productImage);
            await _context.SaveChangesAsync();
            return Ok();
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\OrderController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Internal;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Microsoft.OpenApi;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class OrderController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;

        [HttpGet("GetAllOrders")]
        public async Task<IActionResult> GetAllOrders()
        {
            List<Order> Order = await _context.Orders.ToListAsync();
            if (Order.Count == 0)
            {
                return NotFound("Заказов не найдено");
            }
            List<OrderDTO> OrderDTO = [];
            foreach (var order in Order)
            {
                OrderDTO.Add(order.ToDto());
            }
            return Ok(OrderDTO);
        }

        [HttpGet("GetOrder/{id}")]
        public async Task<IActionResult> GetOrder(int id)
        {
            Order? Orders = await _context.Orders.FirstOrDefaultAsync(x => x.Id == id);
            if (Orders == null)
            {
                return NotFound("Заказ не найден!");
            }
            return Ok(Orders.ToDto());
        }

        [HttpPut("EditStatusOrder")]
        public async Task<IActionResult> EditStatusOrder([FromBody] OrderDTO orderDTO)
```

```
        {
            Order? order = await _context.Orders.FirstOrDefaultAsync(x => x.Id == orderDTO.Id);
            if (order == null)
            {
                return NotFound("Заказ не найден!");
            }
            OrderStatus? status = await _context.OrderStatuses.FirstOrDefaultAsync(x => x.Name == orderDTO.Status);
            if (status == null)
            {
                return NotFound("Статус не найден");
            }
            order.StatusId = status.Id;
            _context.Orders.Update(order);
            await _context.SaveChangesAsync();
            return Ok(order.ToDto());
        }

        [HttpPost("AddOrder")]
        public async Task<IActionResult> AddOrder([FromBody] OrderDTO orderDTO)
        {
            User? user = await _context.Users.FirstOrDefaultAsync(x => x.Email == orderDTO.User);
            if (user == null)
            {
                return NotFound("Пользователь не найден");
            }
            OrderStatus? status = await _context.OrderStatuses.FirstOrDefaultAsync(x => x.Name == orderDTO.Status);
            if (status == null)
            {
                return NotFound("Статус не найден");
            }
            int id = await _context.Orders.AnyAsync() ? await _context.Orders.MaxAsync(x => x.Id) + 1 : 1;
            Order order = new()
            {
                Id = id,
                UserId = user.Id,
                OrderDate = DateTime.Now,
                StatusId = status.Id,
                TotalAmount = orderDTO.TotalAmount,
                City = orderDTO.City,
                Street = orderDTO.Street,
                House = orderDTO.House,
                PostalCode = orderDTO.PostalCode,
            };
            _context.Orders.Add(order);
            await _context.SaveChangesAsync();
            return Ok(order.ToDto());
        }
        [HttpPost("addOrderItem")]
        public async Task<IActionResult> AddOrderItem([FromForm]int userId,[FromForm]int orderId)
        {
            User? user = await _context.Users.FirstOrDefaultAsync(x => x.Id == userId);
            if (user == null)
            {
                return NotFound("Пользователь не найден");
            }
            List<CartItem> cart = await _context.CartItems.Where(x=>x.UserId==user.Id).ToListAsync();
            foreach(var  cartItem in cart)
            {
                  ProductVariant? product = await _context.ProductVariants.Include(pv => pv.Product).FirstOrDefaultAsync(x => x.Id ==
cartItem.ProductVariantId);
                if (product == null)
                {
                    return NotFound("Вариация продукта не найден");
                }
                int idOrderItems = await _context.OrderItems.AnyAsync() ? await _context.OrderItems.MaxAsync(x => x.Id) + 1 : 1;
                OrderItem orderItem = new()
                {
                    Id = idOrderItems,
                    OrderId = orderId,
                    ProductVariantId = cartItem.ProductVariantId,
                    Quantity = cartItem.Quantity,
                    PriceAtPurchase = product.Product.BasePrice
                };
                _context.OrderItems.Add(orderItem);
                await _context.SaveChangesAsync();
```

```
            }
            _context.CartItems.RemoveRange(cart);
            await _context.SaveChangesAsync();
            Order? order = await _context.Orders.FirstOrDefaultAsync(o => o.Id == orderId);
            if (order != null)
            {
                await SendEmail.SendReceipt(_context, order);
            }
            return Ok();
        }

        [HttpPut("EditAddresOrder")]
        public async Task<IActionResult> EditAddresOrder([FromBody] OrderDTO orderDTO)
        {
            Order? order = await _context.Orders.FirstOrDefaultAsync(x => x.Id == orderDTO.Id);
            if (order == null)
            {
                return NotFound("Заказ не найден!");
            }
            order.City = orderDTO.City;
            order.Street = orderDTO.Street;
            order.House = orderDTO.House;
            order.PostalCode = orderDTO.PostalCode;
            _context.Orders.Update(order);
            await _context.SaveChangesAsync();
            return Ok(order.ToDto());
        }


        [HttpGet("GetUserOrderHistory/{userId}")]
        public async Task<IActionResult> GetUserOrderHistory(int userId)
        {
            var user = await _context.Users.FirstOrDefaultAsync(x => x.Id == userId);
            if (user == null)
            {
                return NotFound("Пользователь не найден");
            }

            var orders = await _context.Orders
                .AsNoTracking()
                .Where(o => o.UserId == userId && o.Status.Name == "Доставлен")
                .Include(o => o.Status)
                .Include(o => o.OrderItems)
                    .ThenInclude(oi => oi.ProductVariant)
                        .ThenInclude(pv => pv.Product)
                .Include(o => o.OrderItems)
                    .ThenInclude(oi => oi.ProductVariant)
                        .ThenInclude(pv => pv.Color)
                .OrderByDescending(o => o.OrderDate)
                .ToListAsync();

            var result = orders.Select(o => new UserOrderHistoryDTO
            {
                OrderId = o.Id,
                OrderDate = o.OrderDate.ToShortDateString(),
                Status = o.Status.Name,
                TotalAmount = o.TotalAmount,
                Items = o.OrderItems.Select(oi => new UserOrderItemDTO
                {
                    ProductVariantId = oi.ProductVariantId,
                    ProductName = oi.ProductVariant.Product.Name,
                    Color = oi.ProductVariant.Color.Name,
                    Size = oi.ProductVariant.Size,
                    Quantity = oi.Quantity,
                    Price = oi.PriceAtPurchase
                }).ToList()
            }).ToList();

            return Ok(result);
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\ProductController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Internal;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;
        [HttpGet("GetProductAll")]
        public async Task<IActionResult> GetProductAll()
        {
            List<Product> products = await _context.Products.ToListAsync();
            if(products.Count == 0)
            {
                return NotFound("Продуктов не найдено");
            }
            List<ProductDTO> productDTO = [];
            foreach (var product in products)
            {
                productDTO.Add(product.ToDto());
            }
            return Ok(productDTO);
        }

        [HttpGet("GetProduct/{id}")]
        public async Task<IActionResult> GetProduct(int id)
        {
            Product? product = await _context.Products
                .Include(x=>x.ProductVariants)
                    .ThenInclude(v=>v.Color)
                .FirstOrDefaultAsync(x => x.Id == id);
            if (product == null)
            {
                return NotFound("Продукт не найдена!");
            }
            return Ok(product.ToDto());
        }

        [HttpGet("GetProductCategory/{id}")]
        public async Task<IActionResult> GetProductCategory(int id)
        {
            Category? category = await _context.Categories.FirstOrDefaultAsync(x => x.Id == id);
            if (category == null)
            {
                return NotFound("Категория не найдена!");
            }
            List<Product> products = await _context.Products.Include(x => x.ProductImages).ToListAsync();
            products = [.. products.Where(x => x.CategoryId == id)];
            if (products.Count == 0)
            {
                return NotFound("Товаров с данной категорией не найдено!");
            }
            List<ProductDTO> productDTO = [];
            foreach (var product in products)
            {
                productDTO.Add(product.ToDto());
            }
            return Ok(productDTO);
        }

        [HttpPost("AddProduct")]
        public async Task<IActionResult> AddProduct([FromBody] ProductDTO productDTO)
        {
            if (await _context.Products.AnyAsync(x => x.Name == productDTO.Name))
            {
                return BadRequest("Товар с таким названием уже существует!");
            }
            Brand? brand = await _context.Brands.FirstOrDefaultAsync(x => x.Name == productDTO.Brand);
```

```
    if(brand == null)
    {
        return NotFound("Бренд не найден");
    }
    Category? category = await _context.Categories.FirstOrDefaultAsync(x=>x.Name == productDTO.Category);
    if (category == null)
    {
        return NotFound("Категория не найден");
    }
    int id = await _context.Products.AnyAsync() ? await _context.Products.MaxAsync(x => x.Id) + 1 : 1;
    Product prouduct = new()
    {
        Id = id,
        BrandId = brand.Id,
        CategoryId = category.Id,
        Name = productDTO.Name,
        Description = productDTO.Description,
        BasePrice = productDTO.BasePrice,
    };
    _context.Products.Add(prouduct);
    await _context.SaveChangesAsync();
    return Ok(prouduct.ToDto());
}

[HttpPut("EditProduct")]
public async Task<IActionResult> EditProduct([FromBody] ProductDTO productDTO)
{
    Product? product = await _context.Products.FirstOrDefaultAsync(x => x.Id == productDTO.Id);
    if (product == null)
    {
        return NotFound("Товар не найден!");
    }
    Brand? brand = await _context.Brands.FirstOrDefaultAsync(x => x.Name == productDTO.Brand);
    if (brand == null)
    {
        return NotFound("Бренд не найден");
    }
    Category? category = await _context.Categories.FirstOrDefaultAsync(x => x.Name == productDTO.Category);
    if (category == null)
    {
        return NotFound("Категория не найден");
    }
    product.Name = productDTO.Name;
    product.BrandId = brand.Id;
    product.CategoryId = category.Id;
    product.Description = productDTO.Description;
    product.BasePrice = productDTO.BasePrice;
    _context.Products.Update(product);
    await _context.SaveChangesAsync();
    return Ok(product.ToDto());
}
[HttpPut("EditProductVariant")]
public async Task<IActionResult> EditProductVariant([FromBody] ProductVariantDTO variantDTO)
{
    var variant = await _context.ProductVariants
        .FirstOrDefaultAsync(v => v.Id == variantDTO.Id);

    if (variant == null)
    {
        return NotFound("Вариант товара не найден");
    }

    var color = await _context.Colors.FirstOrDefaultAsync(c => c.Name == variantDTO.Color);
    if (color == null)
    {
        return NotFound("Указанный цвет не найден в базе");
    }

    variant.ColorId = color.Id;
    variant.Size = variantDTO.Size;
    variant.StockQuantity = variantDTO.StockQuantity;

    _context.ProductVariants.Update(variant);
    await _context.SaveChangesAsync();
```

```
            return Ok(variant.ToDto());
        }
        [HttpDelete("DeleteProduct/{id}")]
        public async Task<IActionResult> DeleteProduct(int id)
        {
            Product? product = await _context.Products.FirstOrDefaultAsync(x => x.Id == id);
            if (product == null)
            {
                return NotFound("Продукт не найден");
            }
            _context.Products.Remove(product);
            await _context.SaveChangesAsync();
            return Ok();
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\ProductvariantController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductvariantController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;
        [HttpGet("GetProductVariantsAll")]
        public async Task<IActionResult> GetProductVariantsAll()
        {
            List<ProductVariant> productVariants = await _context.ProductVariants.ToListAsync();
            if (productVariants.Count == 0)
            {
                return NotFound("Вариаций продуктов не найдено");
            }
            List<ProductVariantDTO> productVariantDTO = [];
            foreach (var productVariant in productVariants)
            {
                productVariantDTO.Add(productVariant.ToDto());
            }
            return Ok(productVariantDTO);
        }
        [HttpGet("GetProductVariant/{id}")]
        public async Task<IActionResult> GetProductVariant(int id)
        {
            ProductVariant? productVariant = await _context.ProductVariants.FirstOrDefaultAsync(x => x.Id == id);
            if (productVariant == null)
            {
                return NotFound("Вариация продукта не найдена!");
            }
            return Ok(productVariant.ToDto());
        }
        [HttpPost("AddProductVariant")]
        public async Task<IActionResult> AddProduct([FromBody] ProductVariantDTO productVariantDTO)
        {
            Product? product = await _context.Products.FirstOrDefaultAsync(x=>x.Id == productVariantDTO.ProductId);
            if (product == null)
            {
                return NotFound("Товар не найден");
            }
            Color? color = await _context.Colors.FirstOrDefaultAsync(x=>x.Name == productVariantDTO.Color);
            if (color == null)
            {
                return NotFound("Цвет не найден");
            }
            int id = await _context.ProductVariants.AnyAsync() ? await _context.ProductVariants.MaxAsync(x => x.Id) + 1 : 1;
            ProductVariant prouductVariant = new()
            {
                Id = id,
                ProductId = product.Id,
```

```
                ColorId = color.Id,
                Size = productVariantDTO.Size,
                StockQuantity = productVariantDTO.StockQuantity,
            };
            _context.ProductVariants.Add(prouductVariant);
            await _context.SaveChangesAsync();
            return Ok(prouductVariant.ToDto());
        }
        [HttpPut("EditProductVariant")]
        public async Task<IActionResult> EditProductVariant([FromBody] ProductVariantDTO productVariantDTO)
        {
            ProductVariant? productVariant = await _context.ProductVariants.FirstOrDefaultAsync(x => x.Id == productVariantDTO.Id);
            if (productVariant == null)
            {
                return NotFound("Товар не найден!");
            }
            Product? product = await _context.Products.FirstOrDefaultAsync(x => x.Id == productVariantDTO.ProductId);
            if (product == null)
            {
                return NotFound("Товар не найден");
            }
            Color? color = await _context.Colors.FirstOrDefaultAsync(x => x.Name == productVariantDTO.Color);
            if (color == null)
            {
                return NotFound("Цвет не найден");
            }
            if (productVariant.StockQuantity < 0) return BadRequest("Количество равно нулю!");
            productVariant.ProductId = product.Id;
            productVariant.ColorId = color.Id;
            productVariant.Size = productVariantDTO.Size;
            productVariant.StockQuantity = productVariantDTO.StockQuantity;
            _context.ProductVariants.Update(productVariant);
            await _context.SaveChangesAsync();
            return Ok(productVariant.ToDto());
        }
        [HttpDelete("DeleteProductVariant/{id}")]
        public async Task<IActionResult> DeleteProductVariant(int id)
        {
            ProductVariant? productVariant = await _context.ProductVariants.FirstOrDefaultAsync(x => x.Id == id);
            if (productVariant == null)
            {
                return NotFound("Вариация продукта не найдено");
            }
            _context.ProductVariants.Remove(productVariant);
            await _context.SaveChangesAsync();
            return Ok();
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Controllers\UserController.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class UserController(PostgresContext context) : ControllerBase
    {
        readonly PostgresContext _context = context;

        [HttpGet("GetAllUsers")]
        public async Task<IActionResult> GetAllUsers()
        {
            List<User> users = await _context.Users.ToListAsync();
            if(users.Count == 0)
            {
                return NotFound("Пользователей не найдено");
            }
```

```
    List<UserDTO> usersDTO = [];
    foreach(var user in users)
    {
        usersDTO.Add(user.ToDto());
    }
    return Ok(usersDTO);
}

[HttpGet("GetUser/{id}")]
public async Task<IActionResult> GetUser(int id)
{
    User? user = await _context.Users.FirstOrDefaultAsync(x => x.Id == id);
    if (user == null)
    {
        return NotFound("Пользователь не найден!");
    }
    return Ok(user.ToDto());
}

[HttpPost("Authorization")]
public async Task<IActionResult> Authorization( [FromForm]string login,[FromForm] string password)
{
    User? user = await _context.Users.FirstOrDefaultAsync(x=>x.Email == login);
    if(user == null)
    {
        return BadRequest("Неверный логин или пароль");
    }
    PasswordHasher<UserDTO> passwordHasher = new();
    if (passwordHasher.VerifyHashedPassword(user.ToDto(), user.Password, password) != PasswordVerificationResult.Success)
    {
        return BadRequest("Неверный логин или пароль");
    }
    return Ok(user.ToDto());
}

[HttpPost("Registration")]
public async Task<IActionResult> Registration([FromBody] UserDTO userDTO)
{
    if(await _context.Users.AnyAsync(x => x.Email == userDTO.Email))
    {
        return BadRequest("Пользователь с таким Email занят!");
    }
    if(await _context.Users.AnyAsync(x => x.Phone == userDTO.Phone))
    {
        return BadRequest("Пользователь с таким телефоном занят!");
    }
    int id = await _context.Users.AnyAsync()? await _context.Users.MaxAsync(x=>x.Id)+1 : 1;
    Role? role = await _context.Roles.FirstOrDefaultAsync(x => x.Name == "Клиент");
    if(role == null)
    {
        return BadRequest("Роль не найдена!");
    }
    PasswordHasher<UserDTO> passwordHasher = new();
    string password = passwordHasher.HashPassword(userDTO, userDTO.Password);
    User user = new()
    {
        Id = id,
        Name = userDTO.Name,
        Surname = userDTO.Surname,
        Patronymic = userDTO.Patronymic,
        Email = userDTO.Email,
        Password = password,
        Phone = userDTO.Phone,
        RoleId = role.Id,

    };
    _context.Users.Add(user);
    await _context.SaveChangesAsync();
    return Ok(user.ToDto());
}

[HttpPut("EditUser")]
public async Task<IActionResult> EditUser([FromBody] UserDTO userDTO)
{
    User? user = await _context.Users.FirstOrDefaultAsync(x=>x.Id == userDTO.Id);
```

```
            if(user == null)
            {
                return NotFound("Пользователь не найден!");
            }
            userDTO.Email = userDTO.Email.Trim();
            userDTO.Phone = userDTO.Phone.Trim();
            if (user.Email != userDTO.Email)
            {
                if (await _context.Users.AnyAsync(x => x.Email == userDTO.Email && x.Id != userDTO.Id))
                {
                    return BadRequest("Пользователь с таким Email занят!");
                }
            }
            if(user.Phone != userDTO.Phone)
            {
                if (await _context.Users.AnyAsync(x => x.Phone == userDTO.Phone && x.Id != userDTO.Id))
                {
                    return BadRequest("Пользователь с таким телефоном занят!");
                }
            }
            user.Name = userDTO.Name;
            user.Surname = userDTO.Surname;
            user.Patronymic = userDTO.Patronymic;
            user.Email = userDTO.Email;
            user.Phone = userDTO.Phone;
            _context.Users.Update(user);
            await _context.SaveChangesAsync();
            return Ok(user.ToDto());
        }

        [HttpDelete("DeleteUser")]
        public async Task<IActionResult> DeleteUser(int id)
        {
            User? user = await _context.Users.FirstOrDefaultAsync(x => x.Id == id);
            if(user == null)
            {
                return NotFound("Пользователь не найден");
            }
            _context.Users.Remove(user);
            await _context.SaveChangesAsync();
            return Ok();
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\AiDTOs.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class AiCoachRequestDTO
    {
        public string Message { get; set; } = string.Empty;
    }

    public class AiProductRecommendationDTO
    {
        public int ProductId { get; set; }
        public string ProductName { get; set; } = string.Empty;
        public decimal Price { get; set; }
        public string Reason { get; set; } = string.Empty;
    }

    public class AiCoachResponseDTO
    {
        public string Reply { get; set; } = string.Empty;
        public List<AiProductRecommendationDTO> Recommendations { get; set; } = new();
    }

    public class AiGenerateDescriptionDTO
    {
        public string Name { get; set; } = string.Empty;
        public string Category { get; set; } = string.Empty;
        public string Brand { get; set; } = string.Empty;
```

```
        public string? Technologies { get; set; }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\BrandDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class BrandDTO
    {
        public int Id { get; set; }

        public string Name { get; set; } = null!;
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\CartItemDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class CartItemDTO
    {
        public int Id { get; set; }

        public int UserId { get; set; }

        public int ProductVariantId { get; set; }

        public int Quantity { get; set; }

        //поля для фронтента
        public string ProductName { get; set; } = null!;
        public string Color { get; set; } = null!;
        public decimal Size { get; set; }
        public decimal Price { get; set; }
        //
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\CategoryDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class CategoryDTO
    {
        public int Id { get; set; }
        public string Name { get; set; } = null!;
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\ColorDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class ColorDTO
    {
        public int Id { get; set; }
        public string Name { get; set; } = null!;
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\OrderDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class OrderDTO
    {
        public int Id { get; set; }
```

```
        public string User { get; set; } = null!;

        public string OrderDate { get; set; } = null!;

        public string Status { get; set; } = null!;

        public decimal TotalAmount { get; set; }

        public string City { get; set; } = null!;

        public string Street { get; set; } = null!;

        public string House { get; set; } = null!;

        public string PostalCode { get; set; } = null!;
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\OrderItemDTO.cs

```
using GroupWorkAPI.Model;

namespace GroupWorkAPI.DataClasses
{
    public class OrderItemDTO
    {
        public int Id { get; set; }

        public Order Order { get; set; } = null!;

        public ProductVariant ProductVariant { get; set; } = null!;

        public int Quantity { get; set; }

        public decimal PriceAtPurchase { get; set; }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\OrderStatusDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class OrderStatusDTO
    {
        public int Id { get; set; }
        public string Name { get; set; } = null!;
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\ProductDTO.cs

```
    namespace GroupWorkAPI.DataClasses
{
    public class ProductDTO
    {
        public int Id { get; set; }

        public string Brand { get; set; } = null!;

        public string Category { get; set; } = null!;

        public string Name { get; set; } = null!;

        public string Description { get; set; } = null!;

        public decimal BasePrice { get; set; }

        public List<ProductVariantDTO> Variants { get; set; } = new();
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\ProductFavoriteDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class ProductFavoriteDTO
    {
        public int Id { get; set; }

        public int UserId { get; set; }

        public int ProductId { get; set; }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\ProductImageDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class ProductImageDTO
    {
        public int Id { get; set; }

        public int ProductId { get; set; }

        public string ImageUrl { get; set; } = null!;

        public bool IsMain { get; set; }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\ProductVariantDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class ProductVariantDTO
    {
        public int Id { get; set; }

        public int ProductId { get; set; }

        public string Color { get; set; } = null!;

        public decimal Size { get; set; }

        public int StockQuantity { get; set; }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\RoleDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class RoleDTO
    {
        public int Id { get; set; }

        public string Name { get; set; } = null!;

    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\UserDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class UserDTO
    {
        public int Id { get; set; }
```

```
        public string Name { get; set; } = null!;

        public string Surname { get; set; } = null!;

        public string? Patronymic { get; set; }

        public string Email { get; set; } = null!;

        public string Password { get; set; } = null!;

        public string Phone { get; set; } = null!;

        public string Role { get; set; } = null!;
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\UserOrderHistoryDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class UserOrderHistoryDTO
    {
        public int OrderId { get; set; }
        public string OrderDate { get; set; } = null!;
        public string Status { get; set; } = null!;
        public decimal TotalAmount { get; set; }
        public List<UserOrderItemDTO> Items { get; set; } = new();
    }
    public class UserOrderItemDTO
    {
        public int ProductVariantId { get; set; }
        public string ProductName { get; set; } = null!;
        public string Color { get; set; } = null!;
        public decimal Size { get; set; }
        public int Quantity { get; set; }
        public decimal Price { get; set; }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\VariantFavoriteDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class VariantFavoriteDTO
    {
        public int Id { get; set; }

        public int UserId { get; set; }

        public int ProductVariantId { get; set; }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\DataClasses\VariantImageDTO.cs

```
namespace GroupWorkAPI.DataClasses
{
    public class VariantImageDTO
    {
        public int Id { get; set; }

        public int ProductVariantId { get; set; }

        public string ImagePath { get; set; } = null!;
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Internal\SendEmail.cs

```
using System.Text;
using System.Net;
using System.Net.Mail;
using Microsoft.EntityFrameworkCore;
using GroupWorkAPI.Model;

namespace GroupWorkAPI.Internal
{
    internal static class SendEmail
    {
        public static async Task SendReceipt(PostgresContext context, Order order)
        {
            var fullOrder = await context.Orders
                .AsNoTracking()
                .Include(o => o.User)
                .Include(o => o.OrderItems)
                    .ThenInclude(oi => oi.ProductVariant)
                        .ThenInclude(pv => pv.Product)
                .Include(o => o.OrderItems)
                    .ThenInclude(oi => oi.ProductVariant)
                        .ThenInclude(pv => pv.Color)
                .FirstOrDefaultAsync(o => o.Id == order.Id);

            if (fullOrder == null || fullOrder.User == null)
            {
                throw new Exception("Заказ или пользователь не найдены для отправки чека.");
            }

            var itemsHtml = new StringBuilder();
            decimal totalProductsPrice = 0;

            foreach (var item in fullOrder.OrderItems)
            {
                decimal itemPrice = item.PriceAtPurchase;
                decimal itemSum = itemPrice * item.Quantity;
                totalProductsPrice += itemSum;

                itemsHtml.Append($@"
                    <tr style=""border-bottom: 1px dashed #cbd5e1;"">
                        <td style=""padding: 8px 0; text-align: left; vertical-align: top;"">
                             <div style=""font-weight: 900; font-size: 14px; text-transform: uppercase; font-family: sans-serif; color:
#000000; line-height: 1.25;"">
                                {item.ProductVariant.Product.Name}
                            </div>
                              <div style=""font-size: 11px; color: #64748b; font-weight: bold; text-transform: uppercase; font-family:
sans-serif; margin-top: 2px;"">
                                Цвет: {item.ProductVariant.Color.Name} / Размер: {item.ProductVariant.Size}
                            </div>
                                 <div style=""font-size: 11px; font-weight: 500; color: #94a3b8; font-family: sans-serif; margin-top:
4px;"">
                                {itemPrice} ₽ × {item.Quantity} шт.
                            </div>
                        </td>
                              <td style=""padding: 8px 0; text-align: right; vertical-align: top; font-weight: 900; font-size: 14px;
font-family: sans-serif; color: #000000; white-space: nowrap; width: 100px;"">
                            {itemSum} ₽
                        </td>
                    </tr>");
            }

            string htmlBody = $@"
<div style=""max-w: 448px; margin: 0 auto; background-color: #ffffff; padding: 24px; border: 4px solid #000000; font-family:
sans-serif; color: #000000;"">

  <div style=""text-align: center; border-bottom: 4px solid #000000; padding-bottom: 16px; margin-bottom: 24px;"">
     <h1 style=""font-size: 36px; font-weight: 900; letter-spacing: -0.05em; text-transform: uppercase; font-style: italic; color:
rgb(0,30,98); margin: 0; font-family: sans-serif;"">
      asics
    </h1>
     <p style=""font-size: 12px; text-transform: uppercase; font-weight: bold; letter-spacing: 0.1em; color: #64748b; margin: 4px 0 0
0; font-family: sans-serif;"">
      Anima Sana In Corpore Sano
    </p>
      <div style=""margin-top: 16px; background-color: #000000; color: #ffffff; font-size: 12px; font-weight: 900; text-transform:
uppercase; padding: 4px 12px; display: inline-block; letter-spacing: 0.05em; font-family: sans-serif;"">
```

```
      Товарный чек / Спецификация
    </div>
  </div>

   <table  style=""width: 100%; border-collapse: collapse; margin-bottom: 16px; border-bottom: 2px solid #000000; padding-bottom: 16px;"">
    <tr>
      <td style=""padding-bottom: 8px; font-size: 12px; font-weight: bold; text-transform: uppercase; font-family: sans-serif;"">
        <span style=""color: #64748b; display: block; font-size: 11px;"">Номер заказа:</span>
        <span style=""font-size: 14px; font-weight: 900;"">#ASICS-{fullOrder.Id}</span>
      </td>
       <td style=""padding-bottom: 8px; text-align: right; font-size: 12px; font-weight: bold; text-transform: uppercase; font-family: sans-serif;"">
        <span style=""color: #64748b; display: block; font-size: 11px;"">Дата и время:</span>
        <span style=""font-size: 14px; font-weight: 900;"">{fullOrder.OrderDate:dd.05.yyyy HH:mm}</span>
      </td>
    </tr>
    <tr>
       <td style=""padding-top: 8px; padding-bottom: 16px; font-size: 12px; font-weight: bold; text-transform: uppercase; font-family: sans-serif;"">
        <span style=""color: #64748b; display: block; font-size: 11px;"">Покупатель:</span>
        <span style=""font-weight: 900;"">{fullOrder.User.Name}</span>
      </td>
        <td style=""padding-top: 8px; padding-bottom: 16px; text-align: right; font-size: 12px; font-weight: bold; text-transform: uppercase; font-family: sans-serif;"">
        <span style=""color: #64748b; display: block; font-size: 11px;"">Тип доставки:</span>
        <span style=""font-weight: 900; color: #16a34a;"">Примерка (0 ₽)</span>
      </td>
    </tr>
  </table>

  <div style=""margin-bottom: 24px;"">
    <h3 style=""font-weight: 900; text-transform: uppercase; font-size: 14px; font-style: italic; margin-bottom: 12px; color: #334155; font-family: sans-serif;"">
       Состав заказа:
    </h3>

    <table style=""width: 100%; border-collapse: collapse;"">
      {itemsHtml}
    </table>
  </div>

  <div style=""border-top: 2px solid #000000; background-color: #f8fafc; padding: 12px; border: 1px solid #e2e8f0;"">
     <table style=""width: 100%; border-collapse: collapse; font-size: 12px; text-transform: uppercase; font-weight: bold; color: #475569; font-family: sans-serif;"">
      <tr>
        <td style=""padding: 2px 0; text-align: left;"">Стоимость товаров:</td>
        <td style=""padding: 2px 0; text-align: right; color: #000000;"">{totalProductsPrice} ₽</td>
      </tr>
      <tr>
        <td style=""padding: 2px 0; text-align: left;"">Доставка и примерка:</td>
        <td style=""padding: 2px 0; text-align: right; color: #16a34a;"">бесплатно</td>
      </tr>
      <tr>
        <td style=""padding: 2px 0; text-align: left; color: #dc2626;"">Скидка по промокоду:</td>
        <td style=""padding: 2px 0; text-align: right; color: #dc2626;"">-0 ₽</td>
      </tr>
    </table>

      <table style=""width: 100%; border-collapse: collapse; border-top: 4px solid #000000; margin-top: 12px; padding-top: 12px; font-family: sans-serif;"">
      <tr>
            <td style=""padding-top: 12px; font-size: 20px; font-weight: 900; text-transform: uppercase; letter-spacing: -0.05em; text-align: left; color: #000000;"">
            Итого к оплате:
        </td>
         <td style=""padding-top: 12px; font-size: 24px; font-weight: 900; font-style: italic; color: rgb(0,30,98); text-align: right; white-space: nowrap;"">
            {fullOrder.TotalAmount} ₽
        </td>
      </tr>
    </table>
  </div>

  <div style=""text-align: center; margin-top: 32px; border-top: 1px solid #e2e8f0; padding-top: 16px;"">
```

```
     <p style=""font-size: 10px; text-transform: uppercase; font-weight: 900; letter-spacing: -0.05em; color: #475569; margin: 0;
font-family: sans-serif;"">
      Спасибо за покупку в ASICS Store!
    </p>
    <p style=""font-size: 9px; color: #94a3b8; font-weight: 500; margin-top: 4px; margin-bottom: 0; font-family: sans-serif;"">
      Оригинальный товар сертифицирован. Возврат в течение 14 дней.
    </p>
  </div>
</div>";

            var smtpClient = new SmtpClient("smtp.gmail.com")
            {
                Port = 587,
                Credentials = new NetworkCredential("rafaelguccigang@gmail.com", "ewes mtoi dswt xrvh"),
                EnableSsl = true,
                UseDefaultCredentials = false
            };

            MailAddress from = new("rafaelguccigang@gmail.com", "ASICS Store");
            MailAddress to = new(fullOrder.User.Email);

            var mailMessage = new MailMessage(from, to)
            {
                Subject = $"Кассовый чек по заказу #ASICS-{fullOrder.Id}",
                Body = htmlBody,
                IsBodyHtml = true,
            };

            await smtpClient.SendMailAsync(mailMessage);
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Internal\UploadedFile.cs

```
namespace GroupWorkAPI.Internal
{
    public class UploadedFile
    {
        public IFormFile? File { get; set; }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\Brand.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI.Model;

public partial class Brand
{
    public int Id { get; set; }

    public string Name { get; set; } = null!;

    public virtual ICollection<Product> Products { get; set; } = new List<Product>();
    public BrandDTO ToDto()
    {
        return new()
        {
            Id = Id,
            Name = Name,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\CartItem.cs

```
using GroupWorkAPI.DataClasses;
using System.Xml.Linq;

namespace GroupWorkAPI
```

```
{
    public class CartItem
    {
        public int Id { get; set; }

        public int UserId { get; set; }

        public int ProductVariantId { get; set; }

        public int Quantity { get; set; }

        public virtual ProductVariant ProductVariant { get; set; } = null!;

        public virtual User User { get; set; } = null!;
        public CartItemDTO ToDto()
        {
            return new()
            {
                Id = Id,
                UserId = UserId,
                ProductVariantId = ProductVariantId,
                Quantity = Quantity,

                ProductName = ProductVariant.Product.Name,
                Color = ProductVariant.Color.Name,
                Size = ProductVariant.Size,
                Price = ProductVariant.Product.BasePrice
            };
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\Category.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class Category
{
    public int Id { get; set; }

    public string Name { get; set; } = null!;

    public virtual ICollection<Product> Products { get; set; } = new List<Product>();
    public CategoryDTO ToDto()
    {
        return new()
        {
            Id = Id,
            Name = Name,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\Color.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class Color
{
    public int Id { get; set; }

    public string Name { get; set; } = null!;

    public virtual ICollection<ProductVariant> ProductVariants { get; set; } = new List<ProductVariant>();
    public ColorDTO ToDto()
    {
        return new()
        {
```

```
            Id = Id,
            Name = Name,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\Order.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class Order
{
    public int Id { get; set; }

    public int UserId { get; set; }

    public DateTime OrderDate { get; set; }

    public int StatusId { get; set; }

    public decimal TotalAmount { get; set; }

    public string City { get; set; } = null!;

    public string Street { get; set; } = null!;

    public string House { get; set; } = null!;

    public string PostalCode { get; set; } = null!;

    public virtual ICollection<OrderItem> OrderItems { get; set; } = new List<OrderItem>();

    public virtual OrderStatus Status { get; set; } = null!;

    public virtual User User { get; set; } = null!;
    public OrderDTO ToDto()
    {
        return new()
        {
            Id = Id,
            User = User.Email,
            OrderDate = OrderDate.ToLongDateString(),
            Status = Status.Name,
            TotalAmount = TotalAmount,
            City = City,
            Street = Street,
            House = House,
            PostalCode = PostalCode,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\OrderItem.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class OrderItem
{
    public int Id { get; set; }

    public int OrderId { get; set; }

    public int ProductVariantId { get; set; }

    public int Quantity { get; set; }

    public decimal PriceAtPurchase { get; set; }
```

```
    public virtual Order Order { get; set; } = null!;

    public virtual ProductVariant ProductVariant { get; set; } = null!;
    public OrderItemDTO OrderItemsdto()
    {
        return new()
        {
            Id = Id,
            Order = Order,
            ProductVariant = ProductVariant,
            Quantity = Quantity,
            PriceAtPurchase = PriceAtPurchase,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\OrderStatus.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class OrderStatus
{
    public int Id { get; set; }

    public string Name { get; set; } = null!;

    public virtual ICollection<Order> Orders { get; set; } = new List<Order>();
    public OrderStatusDTO ToDto()
    {
        return new()
        {
            Id = Id,
            Name = Name,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\PostgresContext.cs

```
using Microsoft.EntityFrameworkCore;

namespace GroupWorkAPI.Model;

public partial class PostgresContext : DbContext
{
    public PostgresContext()
    {
    }

    public PostgresContext(DbContextOptions<PostgresContext> options)
        : base(options)
    {
    }

    public virtual DbSet<Brand> Brands { get; set; }

    public virtual DbSet<Category> Categories { get; set; }

    public virtual DbSet<Color> Colors { get; set; }

    public virtual DbSet<CartItem> CartItems { get; set; }

    public virtual DbSet<Order> Orders { get; set; }

    public virtual DbSet<OrderItem> OrderItems { get; set; }

    public virtual DbSet<OrderStatus> OrderStatuses { get; set; }

    public virtual DbSet<Product> Products { get; set; }
```

```
public virtual DbSet<ProductImage> ProductImages { get; set; }

public virtual DbSet<ProductVariant> ProductVariants { get; set; }

public virtual DbSet<Role> Roles { get; set; }

public virtual DbSet<User> Users { get; set; }
public virtual DbSet<VariantImage> VariantImages { get; set; }
public virtual DbSet<ProductFavorite> ProductFavorites { get; set; }
public virtual DbSet<VariantFavorite> VariantFavorites { get; set; }



protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{ }

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Brand>(entity =>
    {
        entity.HasKey(e => e.Id).HasName("brands_pk");

        entity.ToTable("brands", "asics");

        entity.Property(e => e.Id)
            .ValueGeneratedNever()
            .HasColumnName("id");
        entity.Property(e => e.Name)
            .HasMaxLength(15)
            .HasColumnName("name");
    });

    modelBuilder.Entity<CartItem>(entity =>
    {
        entity.HasKey(e => e.Id).HasName("cart_items_pk");

        entity.ToTable("cart_items", "asics");

        entity.Property(e => e.Id)
            .ValueGeneratedNever()
            .HasColumnName("id");
        entity.Property(e => e.ProductVariantId).HasColumnName("product_variant_id");
        entity.Property(e => e.Quantity).HasColumnName("quantity");
        entity.Property(e => e.UserId).HasColumnName("user_id");

        entity.HasOne(d => d.ProductVariant).WithMany(p => p.CartItems)
            .HasForeignKey(d => d.ProductVariantId)
            .HasConstraintName("cart_items_product_variants_id_fk");

        entity.HasOne(d => d.User).WithMany(p => p.CartItems)
            .HasForeignKey(d => d.UserId)
            .HasConstraintName("cart_items_users_id_fk");
    });

    modelBuilder.Entity<Category>(entity =>
    {
        entity.HasKey(e => e.Id).HasName("categories_pk");

        entity.ToTable("categories", "asics");

        entity.Property(e => e.Id)
            .ValueGeneratedNever()
            .HasColumnName("id");
        entity.Property(e => e.Name)
            .HasMaxLength(25)
            .HasColumnName("name");
    });

    modelBuilder.Entity<Color>(entity =>
    {
        entity.HasKey(e => e.Id).HasName("colors_pk");

        entity.ToTable("colors", "asics");

        entity.Property(e => e.Id)
```

```
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.Name)
        .HasMaxLength(30)
        .HasColumnName("name");
});

modelBuilder.Entity<Order>(entity =>
{
    entity.HasKey(e => e.Id).HasName("orders_pk");

    entity.ToTable("orders", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.City)
        .HasMaxLength(25)
        .HasColumnName("city");
    entity.Property(e => e.House)
        .HasMaxLength(6)
        .HasColumnName("house");
    entity.Property(e => e.OrderDate)
        .HasColumnType("timestamp without time zone")
        .HasColumnName("order_date");
    entity.Property(e => e.PostalCode)
        .HasMaxLength(6)
        .HasColumnName("postal_code");
    entity.Property(e => e.StatusId).HasColumnName("status_id");
    entity.Property(e => e.Street)
        .HasMaxLength(35)
        .HasColumnName("street");
    entity.Property(e => e.TotalAmount)
        .HasPrecision(8, 2)
        .HasColumnName("total_amount");
    entity.Property(e => e.UserId).HasColumnName("user_id");

    entity.HasOne(d => d.Status).WithMany(p => p.Orders)
        .HasForeignKey(d => d.StatusId)
        .HasConstraintName("orders_order_statuses_id_fk");

    entity.HasOne(d => d.User).WithMany(p => p.Orders)
        .HasForeignKey(d => d.UserId)
        .HasConstraintName("orders_users_id_fk");
});

modelBuilder.Entity<OrderItem>(entity =>
{
    entity.HasKey(e => e.Id).HasName("order_items_pk");

    entity.ToTable("order_items", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.OrderId).HasColumnName("order_id");
    entity.Property(e => e.PriceAtPurchase)
        .HasPrecision(8, 2)
        .HasColumnName("price_at_purchase");
    entity.Property(e => e.ProductVariantId).HasColumnName("product_variant_id");
    entity.Property(e => e.Quantity).HasColumnName("quantity");

    entity.HasOne(d => d.Order).WithMany(p => p.OrderItems)
        .HasForeignKey(d => d.OrderId)
        .HasConstraintName("order_items_orders_id_fk");

    entity.HasOne(d => d.ProductVariant).WithMany(p => p.OrderItems)
        .HasForeignKey(d => d.ProductVariantId)
        .HasConstraintName("order_items_product_variants_id_fk");
});

modelBuilder.Entity<OrderStatus>(entity =>
{
    entity.HasKey(e => e.Id).HasName("order_statuses_pk");
```

```
    entity.ToTable("order_statuses", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.Name)
        .HasMaxLength(20)
        .HasColumnName("name");
});

modelBuilder.Entity<Product>(entity =>
{
    entity.HasKey(e => e.Id).HasName("products_pk");

    entity.ToTable("products", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.BasePrice)
        .HasPrecision(8, 2)
        .HasColumnName("base_price");
    entity.Property(e => e.BrandId).HasColumnName("brand_id");
    entity.Property(e => e.CategoryId).HasColumnName("category_id");
    entity.Property(e => e.Description).HasColumnName("description");
    entity.Property(e => e.Name)
        .HasMaxLength(40)
        .HasColumnName("name");

    entity.HasOne(d => d.Brand).WithMany(p => p.Products)
        .HasForeignKey(d => d.BrandId)
        .HasConstraintName("products_brands_id_fk");

    entity.HasOne(d => d.Category).WithMany(p => p.Products)
        .HasForeignKey(d => d.CategoryId)
        .HasConstraintName("products_categories_id_fk");
});

modelBuilder.Entity<ProductImage>(entity =>
{
    entity.HasKey(e => e.Id).HasName("product_images_pk");

    entity.ToTable("product_images", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.ImageUrl).HasColumnName("image_url");
    entity.Property(e => e.IsMain).HasColumnName("is_main");
    entity.Property(e => e.ProductId).HasColumnName("product_id");

    entity.HasOne(d => d.Product).WithMany(p => p.ProductImages)
        .HasForeignKey(d => d.ProductId)
        .HasConstraintName("product_images_products_id_fk");
});

modelBuilder.Entity<ProductVariant>(entity =>
{
    entity.HasKey(e => e.Id).HasName("product_variants_pk");

    entity.ToTable("product_variants", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.ColorId).HasColumnName("color_id");
    entity.Property(e => e.ProductId).HasColumnName("product_id");
    entity.Property(e => e.Size)
        .HasPrecision(5, 2)
        .HasColumnName("size");
    entity.Property(e => e.StockQuantity).HasColumnName("stock_quantity");

    entity.HasOne(d => d.Color).WithMany(p => p.ProductVariants)
        .HasForeignKey(d => d.ColorId)
        .HasConstraintName("product_variants_colors_id_fk");
```

```
    entity.HasOne(d => d.Product).WithMany(p => p.ProductVariants)
        .HasForeignKey(d => d.ProductId)
        .HasConstraintName("product_variants_products_id_fk");
});

modelBuilder.Entity<Role>(entity =>
{
    entity.HasKey(e => e.Id).HasName("roles_pk");

    entity.ToTable("roles", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.Name)
        .HasMaxLength(45)
        .HasColumnName("name");
});

modelBuilder.Entity<User>(entity =>
{
    entity.HasKey(e => e.Id).HasName("users_pk");

    entity.ToTable("users", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.Email)
        .HasMaxLength(50)
        .HasColumnName("email");
    entity.Property(e => e.Name)
        .HasMaxLength(30)
        .HasColumnName("name");
    entity.Property(e => e.Password)
        .HasMaxLength(128)
        .HasColumnName("password");
    entity.Property(e => e.Patronymic)
        .HasMaxLength(35)
        .HasColumnName("patronymic");
    entity.Property(e => e.Phone)
        .HasMaxLength(12)
        .HasColumnName("phone");
    entity.Property(e => e.RoleId).HasColumnName("role_id");
    entity.Property(e => e.Surname)
        .HasMaxLength(30)
        .HasColumnName("surname");

    entity.HasOne(d => d.Role).WithMany(p => p.Users)
        .HasForeignKey(d => d.RoleId)
        .HasConstraintName("users_roles_id_fk");
});

modelBuilder.Entity<VariantImage>(entity =>
{
    entity.HasKey(e => e.Id).HasName("variant_images_pk");

    entity.ToTable("variant_images", "asics");

    entity.Property(e => e.Id)
        .ValueGeneratedNever()
        .HasColumnName("id");
    entity.Property(e => e.ImagePath)
        .HasMaxLength(255)
        .HasColumnName("image_path");
    entity.Property(e => e.ProductVariantId).HasColumnName("product_variant_id");

    entity.HasOne(d => d.ProductVariant).WithMany(p => p.VariantImages)
        .HasForeignKey(d => d.ProductVariantId)
        .HasConstraintName("variant_images_product_variants_id_fk");
});

modelBuilder.Entity<ProductFavorite>(entity =>
{
```

```
            entity.HasKey(e => e.Id).HasName("product_favorites_pk");

            entity.ToTable("product_favorites", "asics");

            entity.HasIndex(e => new { e.UserId, e.ProductId }, "product_favorites_user_product_uq").IsUnique();

            entity.Property(e => e.Id)
                .ValueGeneratedNever()
                .HasColumnName("id");
            entity.Property(e => e.ProductId).HasColumnName("product_id");
            entity.Property(e => e.UserId).HasColumnName("user_id");

            entity.HasOne(d => d.Product).WithMany(p => p.ProductFavorites)
                .HasForeignKey(d => d.ProductId)
                .HasConstraintName("product_favorites_products_id_fk");

            entity.HasOne(d => d.User).WithMany(p => p.ProductFavorites)
                .HasForeignKey(d => d.UserId)
                .HasConstraintName("product_favorites_users_id_fk");
        });

        modelBuilder.Entity<VariantFavorite>(entity =>
        {
            entity.HasKey(e => e.Id).HasName("variant_favorites_pk");

            entity.ToTable("variant_favorites", "asics");

            entity.HasIndex(e => new { e.UserId, e.ProductVariantId }, "variant_favorites_user_variant_uq").IsUnique();

            entity.Property(e => e.Id)
                .ValueGeneratedNever()
                .HasColumnName("id");
            entity.Property(e => e.ProductVariantId).HasColumnName("product_variant_id");
            entity.Property(e => e.UserId).HasColumnName("user_id");

            entity.HasOne(d => d.ProductVariant).WithMany(p => p.VariantFavorites)
                .HasForeignKey(d => d.ProductVariantId)
                .HasConstraintName("variant_favorites_product_variants_id_fk");

            entity.HasOne(d => d.User).WithMany(p => p.VariantFavorites)
                .HasForeignKey(d => d.UserId)
                .HasConstraintName("variant_favorites_users_id_fk");
        });

        OnModelCreatingPartial(modelBuilder);
    }

    partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\Product.cs

```
using GroupWorkAPI.DataClasses;
using GroupWorkAPI.Model;

namespace GroupWorkAPI;

public partial class Product
{
    public int Id { get; set; }

    public int BrandId { get; set; }

    public int CategoryId { get; set; }

    public string Name { get; set; } = null!;

    public string Description { get; set; } = null!;

    public decimal BasePrice { get; set; }

    public virtual Brand Brand { get; set; } = null!;

    public virtual Category Category { get; set; } = null!;
```

```
    public virtual ICollection<ProductImage> ProductImages { get; set; } = new List<ProductImage>();

    public virtual ICollection<ProductVariant> ProductVariants { get; set; } = new List<ProductVariant>();
    public virtual ICollection<ProductFavorite> ProductFavorites { get; set; } = new List<ProductFavorite>();
    public ProductDTO ToDto()
    {
        return new()
        {
            Id = Id,
            Brand = Brand.Name,
            Category = Category.Name,
            Name = Name,
            Description = Description,
            BasePrice = BasePrice,

            Variants = [.. ProductVariants.Select(v => new ProductVariantDTO
            {
                Id = v.Id,
                ProductId = v.ProductId,
                Color = v.Color?.Name ?? "Unknown",
                Size = v.Size,
                StockQuantity = v.StockQuantity
            })]
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\ProductFavorite.cs

```
using GroupWorkAPI.DataClasses;
using Microsoft.EntityFrameworkCore;
using System.Net.NetworkInformation;

namespace GroupWorkAPI
{
    public class ProductFavorite
    {
        public int Id { get; set; }

        public int UserId { get; set; }

        public int ProductId { get; set; }

        public virtual Product Product { get; set; } = null!;

        public virtual User User { get; set; } = null!;
        public ProductFavorite ToDto()
        {
            return new()
            {
                Id = Id,
                UserId=UserId,
                ProductId=ProductId,
            };
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\ProductImage.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class ProductImage
{
    public int Id { get; set; }

    public int ProductId { get; set; }

    public string ImageUrl { get; set; } = null!;
```

```
    public bool IsMain { get; set; }

    public virtual Product Product { get; set; } = null!;
    public ProductImageDTO ToDto()
    {
        return new()
        {
            Id = Id,
            ProductId = ProductId,
            ImageUrl = ImageUrl,
            IsMain = IsMain,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\ProductVariant.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class ProductVariant
{
    public int Id { get; set; }

    public int ProductId { get; set; }

    public int ColorId { get; set; }

    public decimal Size { get; set; }

    public int StockQuantity { get; set; }

    public virtual Color Color { get; set; } = null!;

    public virtual ICollection<OrderItem> OrderItems { get; set; } = new List<OrderItem>();
    public virtual ICollection<CartItem> CartItems { get; set; } = new List<CartItem>();
    public virtual ICollection<VariantImage> VariantImages { get; set; } = new List<VariantImage>();
    public virtual ICollection<VariantFavorite> VariantFavorites { get; set; } = new List<VariantFavorite>();


    public virtual Product Product { get; set; } = null!;
    public ProductVariantDTO ToDto()
    {
        return new()
        {
            Id = Id,
            ProductId = ProductId,
            Color = Color.Name,
            Size = Size,
            StockQuantity = StockQuantity,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\Role.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class Role
{
    public int Id { get; set; }

    public string Name { get; set; } = null!;

    public virtual ICollection<User> Users { get; set; } = new List<User>();
    public RoleDTO ToDto()
    {
        return new()
```

```
        {
            Id = Id,
            Name = Name,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\User.cs

```
using GroupWorkAPI.DataClasses;

namespace GroupWorkAPI;

public partial class User
{
    public int Id { get; set; }

    public string Name { get; set; } = null!;

    public string Surname { get; set; } = null!;

    public string? Patronymic { get; set; }

    public string Email { get; set; } = null!;

    public string Password { get; set; } = null!;

    public string Phone { get; set; } = null!;

    public int RoleId { get; set; }

    public virtual ICollection<Order> Orders { get; set; } = new List<Order>();

    public virtual ICollection<CartItem> CartItems { get; set; } = new List<CartItem>();
    public virtual ICollection<ProductFavorite> ProductFavorites { get; set; } = new List<ProductFavorite>();
    public virtual ICollection<VariantFavorite> VariantFavorites { get; set; } = new List<VariantFavorite>();


    public virtual Role Role { get; set; } = null!;

    public UserDTO ToDto()
    {
        return new()
        {
            Id = Id,
            Name = Name,
            Surname = Surname,
            Patronymic = Patronymic,
            Email = Email,
            Password = Password,
            Phone = Phone,
            Role = Role.Name,
        };
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\VariantFavorite.cs

```
namespace GroupWorkAPI
{
    public class VariantFavorite
    {
        public int Id { get; set; }

        public int UserId { get; set; }

        public int ProductVariantId { get; set; }

        public virtual ProductVariant ProductVariant { get; set; } = null!;

        public virtual User User { get; set; } = null!;
```

```
        public VariantFavorite ToDto()
        {
            return new()
            {
                Id = Id,
                UserId = UserId,
                ProductVariantId = ProductVariantId,
            };
        }
    }
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Model\VariantImage.cs

```
using System;
using System.Collections.Generic;

namespace GroupWorkAPI;

public partial class VariantImage
{
    public int Id { get; set; }

    public int ProductVariantId { get; set; }

    public string ImagePath { get; set; } = null!;

    public virtual ProductVariant ProductVariant { get; set; } = null!;

    public VariantImage ToDto()
    {
        return new()
        {
            Id = Id,
            ProductVariantId = ProductVariant.Id,
            ImagePath = ImagePath
        };
    }

}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Properties\launchSettings.json

```
{
  "profiles": {
    "http": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "http://localhost:5003"
    },
    "https": {
      "commandName": "Project",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      },
      "dotnetRunMessages": true,
      "applicationUrl": "https://localhost:7081;http://localhost:5003"
    },
    "Container (Dockerfile)": {
      "commandName": "Docker",
      "launchUrl": "{Scheme}://{ServiceHost}:{ServicePort}",
      "environmentVariables": {
        "ASPNETCORE_HTTPS_PORTS": "8081",
        "ASPNETCORE_HTTP_PORTS": "8080"
      },
      "publishAllPorts": true,
      "useSSL": true
    }
```

```
  },
  "$schema": "https://json.schemastore.org/launchsettings.json"
}
```

[BACKEND (API)] GroupWorkAPI\GroupWorkAPI\Services\AiService.cs

```
using System.Net.Http.Headers;
using System.Text;
using System.Text.Json;

namespace GroupWorkAPI.Services
{
    public class AiService
    {
        private readonly HttpClient _httpClient;

        public AiService(IConfiguration configuration)
        {
            var handler = new HttpClientHandler
            {
                ServerCertificateCustomValidationCallback = (message, cert, chain, errors) => true
            };

            _httpClient = new HttpClient(handler)
            {
                Timeout = TimeSpan.FromSeconds(45)
            };

            _httpClient.DefaultRequestHeaders.UserAgent.ParseAdd("Mozilla/5.0 (Windows NT 10.0; Win64; x64)");
        }

        public async Task<string?> AskAiAsync(string prompt, string systemRole = "You are an expert ASICS running coach.")
        {
            try
            {
                // Международный открытый эндпоинт формата OpenAI (НЕ блокирует РФ и Docker)
                string endpoint = "https://text.pollinations.ai/openai/chat/completions";

                var payload = new
                {
                    model = "mistral", // Качественная быстрая модель с отличным русским языком
                    messages = new[]
                    {
                        new { role = "system", content = systemRole },
                        new { role = "user", content = prompt }
                    },
                    temperature = 0.5
                };

                using var request = new HttpRequestMessage(HttpMethod.Post, endpoint);
                request.Content = new StringContent(JsonSerializer.Serialize(payload), Encoding.UTF8, "application/json");

                var response = await _httpClient.SendAsync(request);
                var responseBody = await response.Content.ReadAsStringAsync();

                if (!response.IsSuccessStatusCode)
                {
                    Console.WriteLine($"[AI GATEWAY ERROR]: {response.StatusCode} - {responseBody}");
                    return null;
                }

                using var doc = JsonDocument.Parse(responseBody);
                var root = doc.RootElement;

                if (root.TryGetProperty("choices", out var choices) && choices.GetArrayLength() > 0)
                {
                    return choices[0]
                        .GetProperty("message")
                        .GetProperty("content")
                        .GetString()?.Trim();
                }

                return null;
            }
            catch (Exception ex)
```

```
            {
                Console.WriteLine($"[AI CONNECTION ERROR]: {ex.Message}");
                return null;
            }
        }
    }
}
```

[OTHER] Project_ASICS\.gitignore

```
# Сборка и зависимости
node_modules/
dist/
dist-ssr/
coverage/

# Логи
logs/
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

# Локальные переменные окружения (.env.local и т.д.)
*.local

# Редакторы кода и IDE
.idea/
.vscode/*
!.vscode/extensions.json
*.suo
*.sw?

# Артефакты сборщиков и тестов
*.tsbuildinfo
.eslintcache
*.timestamp-*-*.mjs
cypress/videos/
cypress/screenshots/
__screenshots__/

# Системный мусор
.DS_Store
Thumbs.db
```

[OTHER] Project_ASICS\.oxfmtrc.json

```
{
  "$schema": "./node_modules/oxfmt/configuration_schema.json",
  "semi": false,
  "singleQuote": true
}
```

[OTHER] Project_ASICS\index.html

```
<!DOCTYPE html>
<html lang="">
  <head>
    <meta charset="UTF-8">
    <link rel="icon" href="/favicon.ico">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Vite App</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.js"></script>
  </body>
</html>
```

[OTHER] Project_ASICS\jsconfig.json

```
{
  "compilerOptions": {
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "exclude": ["node_modules", "dist"]
}
```

[OTHER] Project_ASICS\package.json

```
{
  "name": "project-asics",
  "version": "0.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "format": "oxfmt src/"
  },
  "dependencies": {
    "@tailwindcss/vite": "^4.2.4",
    "axios": "^1.15.2",
    "chart.js": "^4.5.1",
    "pinia": "^3.0.4",
    "vue": "^3.5.32",
    "vue-chartjs": "^5.3.3",
    "vue-router": "^5.0.4",
    "vue-toastification": "^2.0.0-rc.5"
  },
  "devDependencies": {
    "@vitejs/plugin-vue": "^6.0.6",
    "autoprefixer": "^10.5.0",
    "oxfmt": "^0.45.0",
    "postcss": "^8.5.12",
    "tailwindcss": "^4.2.4",
    "vite": "^8.0.8",
    "vite-plugin-vue-devtools": "^8.1.1"
  },
  "engines": {
    "node": "^20.19.0 || >=22.12.0"
  }
}
```

[OTHER] Project_ASICS\README.md

```
# Project_ASICS

This template should help get you started developing with Vue 3 in Vite.

## Recommended IDE Setup

[VS Code](https://code.visualstudio.com/) + [Vue (Official)](https://marketplace.visualstudio.com/items?itemName=Vue.volar) (and disable Vetur).

## Recommended Browser Setup

- Chromium-based browsers (Chrome, Edge, Brave, etc.):
  - [Vue.js devtools](https://chromewebstore.google.com/detail/vuejs-devtools/nhdogjmejiglipccpnnnanhbledajbpd)
  - [Turn on Custom Object Formatter in Chrome DevTools](http://bit.ly/object-formatters)
- Firefox:
  - [Vue.js devtools](https://addons.mozilla.org/en-US/firefox/addon/vue-js-devtools/)
  - [Turn on Custom Object Formatter in Firefox DevTools](https://fxdx.dev/firefox-devtools-custom-object-formatters/)

## Customize configuration

See [Vite Configuration Reference](https://vite.dev/config/).
```

Project Setup

```sh
npm install
```

Compile and Hot-Reload for Development

```sh
npm run dev
```

Compile and Minify for Production

```sh
npm run build
```

[OTHER] Project_ASICS\vite.config.js

```
import { fileURLToPath, URL } from 'node:url'

import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'
import vue from '@vitejs/plugin-vue'
import vueDevTools from 'vite-plugin-vue-devtools'

// https://vite.dev/config/
export default defineConfig({
  plugins: [
    tailwindcss(),
    vue(),
    vueDevTools(),
  ],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url))
    },
  },
})
```

[OTHER] Project_ASICS\src\App.vue

```
<script setup>
import useUserStore from "@/stores/user.js";
import { onMounted } from "vue";
import AppAiCoachWidget from "@/components/AppAiCoachWidget.vue";

const userStore = useUserStore()
onMounted(async () => {
 if (localStorage.getItem('user')) {
 try {
 const localStorageUser = JSON.parse(localStorage.getItem('user') ?? '')
 await userStore.authUser(localStorageUser.login, localStorageUser.password)
 } catch (e) {
 console.error("Ошибка автоавторизации:", e)
 localStorage.removeItem('user')
 }
 }
})
</script>

<template>
 <div>
 <app-header></app-header>
 <!-- :key="$route.fullPath" заставляет Vue перезагружать карточку при переходе на другой товар -->
 <router-view :key="$route.fullPath"></router-view>
 <app-footer></app-footer>
 <app-ai-coach-widget />
 </div>
</template>
```

```
<style scoped></style>
```

[OTHER] Project_ASICS\src\main.js

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'

import router from './router/index.js'
import App from './App.vue'
import './assets/base.css'
import Toast from 'vue-toastification'
import 'vue-toastification/dist/index.css'
import AppHeaders from '@/components/AppHeaders.vue'
import AppFooters from '@/components/AppFooters.vue'
import AppProductCard from '@/components/AppProductCard.vue'
import AppCartItem from '@/components/AppCartItem.vue'
import AppCartModal from "@/components/AppCartModal.vue";
import AppAdminProducts from "@/components/AppAdminProducts.vue";
import AppEditProductModal from "@/components/AppEditProductModal.vue";
import AppEditVariantModal from "@/components/AppEditVariantModal.vue";
import AppAddVariantModal from "@/components/AppAddVariantModal.vue";
import AppAddProductModal from "@/components/AppAddProductModal.vue";
import AppOrderCard from "@/components/AppOrderCard.vue";
import AppEditStatusOrderModal from "@/components/AppEditStatusOrderModal.vue";
import AppUser from "@/components/AppUser.vue";

const toastOptions = {
  transition: 'Vue-Toastification__fade',
  maxToasts: 3,
  newestOnTop: true,
  filterBeforeCreate: (toast, toasts) => {
    if (toasts.filter((t) => t.type === toast.type).length !== 0) {
      return false
    }
    return toast
  },
}

const app = createApp(App)
app.use(Toast, toastOptions)
app.component('app-header', AppHeaders)
app.component('app-footer', AppFooters)
app.component('app-product-cart', AppProductCard)
app.component('app-cart-item', AppCartItem)
app.component('app-cart-modal',AppCartModal)
app.component('app-admin-product',AppAdminProducts)
app.component('app-edit-product',AppEditProductModal)
app.component('app-edit-variant',AppEditVariantModal)
app.component('app-add-variant',AppAddVariantModal)
app.component('app-add-product',AppAddProductModal)
app.component('app-order-card',AppOrderCard)
app.component('app-order-edit-status',AppEditStatusOrderModal)
app.component('app-admin-user',AppUser)
app.use(createPinia())
app.use(router)

app.mount('#app')
```

[OTHER] Project_ASICS\src\assets\base.css

```
@import 'tailwindcss';

@font-face {
  font-family: 'Franklin';
  src: url('./Franklin Gothic Light SSi Light.ttf');
}
* {
  box-sizing: border-box;
  font-family: 'Franklin', sans-serif;
}
```

[OTHER] Project_ASICS\src\components\AppAddProductModal.vue

```
<script setup>
import { ref, onBeforeMount } from 'vue'
import useProductStore from "@/stores/product.js"
import useCategoryStore from "@/stores/category.js"
import useBrandStore from "@/stores/brand.js"
import { useAiStore } from "@/stores/ai.js"
import { useToast } from "vue-toastification"

const props = defineProps(['isOpen'])
const emit = defineEmits(['close'])
const productStore = useProductStore()
const categoryStore = useCategoryStore()
const brandStore = useBrandStore()
const aiStore = useAiStore()
const toast = useToast()

const newProduct = ref({
 name: '',
 description: '',
 brand: '',
 category: '',
 basePrice: 0
})

const techInput = ref('')
const image = ref(null)

const handleFileUpload = (event) => {
 image.value = event.target.files[0] ?? null
}

onBeforeMount(async () => {
 await brandStore.getAllBrand()
 await categoryStore.getAllCategories()
})

const handleGenerateDescription = async () => {
 if (!newProduct.value.name) {
 toast.error("Сначала укажите название модели!")
 return
 }
 const generated = await aiStore.generateDescription({
 name: newProduct.value.name,
 category: newProduct.value.category || 'Кроссовки',
 brand: newProduct.value.brand || 'ASICS',
 technologies: techInput.value
 })
 if (generated) {
 newProduct.value.description = generated
 }
}

const isSubmitting = ref(false)

const addProduct = async () => {
  if (!newProduct.value.name || !newProduct.value.brand || !newProduct.value.category || !newProduct.value.basePrice ||
    !newProduct.value.description || !image.value) {
    toast.error("Заполните основные поля!")
    return
  }

  if (isSubmitting.value) return
  isSubmitting.value = true

  try {
    const response = await productStore.addProduct(newProduct.value)
    if (!response?.data) throw new Error("Не удалось создать товар")

    const createdProduct = response.data
    await productStore.addImageProduct(image.value, createdProduct)

    toast.success("Товар успешно добавлен!")
    await productStore.productFun()
```

```
    emit('close')
  } catch (e) {
    // Если ошибка уже показана в store, не дублируем
  } finally {
    isSubmitting.value = false
  }
}
</script>
<template>
 <div v-if="isOpen" class="fixed inset-0 bg-black/60 flex items-center justify-center z-[1000] p-4 backdrop-blur-sm text-black">
  <div class="bg-white w-full max-w-xl border-2 border-black p-8 rounded-2xl shadow-[15px_15px_0px_0px_rgba(0,0,0,1)] max-h-[95vh] overflow-y-auto">
 <div class="flex justify-between items-center mb-6 border-b-4 border-black pb-2">
 <h2 class="font-black uppercase italic text-2xl">Новый продукт</h2>
 <button @click="emit('close')" class="text-2xl font-bold hover:rotate-90 transition-transform cursor-pointer"></button>
 </div>
 <div class="flex flex-col gap-5">
 <div class="flex flex-col">
 <label class="text-[10px] font-black uppercase text-gray-400 mb-1 ml-1">Название модели</label>
 <input v-model="newProduct.name" type="text" placeholder="Например: Gel-Kayano 30" class="h-12 border-2 border-black px-4 font-bold outline-none focus:bg-yellow-50" />
 </div>
 <div class="grid grid-cols-2 gap-4">
 <div class="flex flex-col">
 <label class="text-[10px] font-black uppercase text-gray-400 mb-1 ml-1">Бренд</label>
 <select v-model="newProduct.brand" class="h-12 border-2 border-black px-3 font-bold bg-white outline-none cursor-pointer">
 <option value="">Выбрать...</option>
 <option v-for="b in brandStore.brand" :key="b.id" :value="b.name">{{ b.name }}</option>
 </select>
 </div>
 <div class="flex flex-col">
 <label class="text-[10px] font-black uppercase text-gray-400 mb-1 ml-1">Категория</label>
 <select v-model="newProduct.category" class="h-12 border-2 border-black px-3 font-bold bg-white outline-none cursor-pointer">
 <option value="">Выбрать...</option>
 <option v-for="c in categoryStore.category" :key="c.id" :value="c.name">{{ c.name }}</option>
 </select>
 </div>
 </div>
 <div class="flex flex-col">
 <label class="text-[10px] font-black uppercase text-gray-400 mb-1 ml-1">Базовая цена ($)</label>
 <input v-model="newProduct.basePrice" type="number" class="h-12 border-2 border-black px-4 font-bold outline-none" />
 </div>

 <!-- БЛОК AI COPYWRITER -->
 <div class="p-3 bg-blue-50 border-2 border-black rounded-xl flex flex-col gap-2">
 <div class="flex justify-between items-center">
 <span class="text-xs font-black uppercase text-[rgb(0,30,98)]">
  AI Copywriter
 </span>
 <button
 type="button"
 @click="handleGenerateDescription"
 :disabled="aiStore.isGenerating"
   class="bg-[rgb(0,30,98)] text-white px-3 py-1.5 rounded-lg border-2 border-black font-black uppercase text-[10px] shadow-[2px_2px_0px_0px_rgba(0,0,0,1)] hover:bg-blue-800 disabled:bg-gray-400 cursor-pointer active:translate-y-0.5"
 >
 {{ aiStore.isGenerating ? 'Генерирую...' : 'Сгенерировать с помощью ИИ' }}
 </button>
 </div>
 <input
 v-model="techInput"
 type="text"
 placeholder="Технологии: PureGEL, FlyteFoam Blast, AHAR+..."
 class="h-9 border border-black bg-white px-3 text-xs font-bold outline-none"
 />
 </div>

 <div class="flex flex-col">
 <label class="text-[10px] font-black uppercase text-gray-400 mb-1 ml-1">Описание товара</label>
  <textarea v-model="newProduct.description" rows="4" placeholder="Введите характеристики или сгенерируйте с помощью ИИ..." class="border-2 border-black p-4 font-medium outline-none resize-none focus:bg-blue-50"></textarea>
 </div>
 <div class="flex flex-col">
 <input
 class="w-full bg-white border border-b focus:border-[rgb(0,30,98)] rounded-3xl px-6 py-4 outline-none text-lg"
```

```
 type="file"
 accept="image/*"
 @change="(e) => handleFileUpload(e)"
 />
 </div>
 <div class="flex gap-4 mt-4">
  <button  @click="addProduct"  class="flex-1  bg-black  text-white  h-14  font-black  uppercase  tracking-widest  hover:bg-gray-800
transition-colors shadow-[6px_6px_0px_0px_rgba(0,0,0,0.2)] active:translate-y-1 active:shadow-none cursor-pointer">
 Создать товар
 </button>
  <button  @click="emit('close')"  class="flex-1  border-2  border-black  h-14  font-black  uppercase  tracking-widest  hover:bg-gray-50
transition-colors cursor-pointer">
 Отмена
 </button>
 </div>
 </div>
 </div>
 </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppAddVariantModal.vue

```
<script setup>
import {onBeforeMount, ref} from 'vue'
import useProductStore from "@/stores/product.js"
import { useToast } from "vue-toastification"
import useColorStore from "@/stores/color.js";

const emit = defineEmits(['close'])
const productStore = useProductStore()
const toast = useToast()
const colorStore = useColorStore()
const newVariant = ref({
  productId: null,
  color: null,
  size: '',
  stockQuantity: 0
})

const image = ref(null)
const handleFileUpload = (event) => {
  image.value = event.target.files[0] ?? null
}

onBeforeMount(async () => {
  if (colorStore.color.length === 0) {
    await colorStore.getAllColors()
  }
})
const add = async () => {
    if  (!newVariant.value.productId  ||  !newVariant.value.color  ||  !newVariant.value.size  ||  !newVariant.value.stockQuantity  ||
!image.value){
    toast.error("Заполните поля")
    return
  }
  const response = await productStore.addProductVariant(newVariant.value)
  const createdVariant = response.data
  await productStore.addImageVariant(image.value, { ...createdVariant })
  toast.success("Вариация успешно добавлена")
  await productStore.productFun()
  emit('close')
}
</script>

<template>
  <div class="fixed inset-0 bg-black/60 flex items-center justify-center z-[1000] p-4 backdrop-blur-sm text-black">
    <div class="bg-white w-full max-w-md border-2 border-black p-6 rounded-2xl ">
      <h2 class="font-black uppercase mb-4 italic text-blue-600">Новая вариация</h2>

      <div class="flex flex-col gap-3">
        <select v-model="newVariant.productId" class="h-10 border-2 border-black px-2 font-bold">
          <option :value="null">Выберите товар...</option>
          <option v-for="p in productStore.products" :key="p.id" :value="p.id">{{ p.name }}</option>
        </select>
```

```
        <select v-model="newVariant.color" class="h-10 border-2 border-black px-2 font-bold">
          <option :value="null">Выберите цвет...</option>
          <option v-for="color in colorStore.color" :key="color.id" :value="color.name">{{ color.name }}</option>
        </select>

        <input v-model="newVariant.size" placeholder="Размер (например, 42)" class="h-10 border-2 border-black px-3" />
        <input v-model="newVariant.stockQuantity" type="number" placeholder="Количество" class="h-10 border-2 border-black px-3" />

        <input
            class="w-full bg-white border border-b focus:border-[rgb(0,30,98)] rounded-3xl px-6 py-4 outline-none text-lg"
            type="file"
            accept="image/*"
            @change="(e) => handleFileUpload(e)"
        />

              <button @click="add" class="bg-blue-600 text-white py-3 font-black uppercase text-xs mt-2 border-2 border-black
active:translate-y-0.5 transition-all">
          Добавить в базу
        </button>
      </div>

      <button @click="emit('close')" class="w-full mt-4 text-[10px] font-bold uppercase underline">Отмена</button>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppAdminAnalytics.vue

```
<script setup>
import { onMounted, computed } from 'vue'
import useAnalyticsStore from '@/stores/analytics.js'
import { Bar } from 'vue-chartjs'
import {
  Chart as ChartJS,
  Title,
  Tooltip,
  Legend,
  BarElement,
  CategoryScale,
  LinearScale
} from 'chart.js'

ChartJS.register(Title, Tooltip, Legend, BarElement, CategoryScale, LinearScale)

const analyticsStore = useAnalyticsStore()

onMounted(async () => {
  await analyticsStore.fetchProductAnalytics()
  await analyticsStore.fetchVariantAnalytics()
})

const productChartData = computed(() => ({
  labels: analyticsStore.mostLikedProducts.map(p => p.productName),
  datasets: [
    {
      label: 'Количество лайков (Товары)',
      backgroundColor: '#001E62',
      data: analyticsStore.mostLikedProducts.map(p => p.likesCount)
    }
  ]
}))

// Автоматически разбиваем длинную строку на две части, чтобы Chart.js перенёс характеристики на новую строчку
const variantChartData = computed(() => ({
  labels: analyticsStore.mostLikedVariants.map(v => {
    const name = v.variantName || ''
    if (name.includes('(')) {
      const parts = name.split('(')
      return [parts[0].trim(), '(' + parts[1]] // Возвращаем массив: [Название, (Характеристики)]
    }
    return name
  }),
  datasets: [
    {
      label: 'Количество лайков (Вариации)',
```

```
      backgroundColor: '#10B981',
      data: analyticsStore.mostLikedVariants.map(v => v.likesCount)
    }
  ]
}))

const productChartOptions = {
  responsive: true,
  maintainAspectRatio: false,
  scales: {
    y: {
      beginAtZero: true,
      ticks: {
        stepSize: 1
      }
    }
  }
}

const variantChartOptions = {
  responsive: true,
  maintainAspectRatio: false,
  indexAxis: 'y',
  scales: {
    x: {
      beginAtZero: true,
      ticks: {
        stepSize: 1
      }
    },
    y: {
      ticks: {
        autoSkip: false,
        crossAlign: 'end',
        // Стилизуем шрифт подписей, чтобы они читались чётко
        font: {
          size: 11,
          weight: 'bold'
        }
      }
    }
  }
}
</script>

<template>
  <div class="p-6 bg-gray-50 min-h-screen">
    <h1 class="text-3xl font-black text-[rgb(0,30,98)] uppercase mb-8">Аналитика избранного</h1>

    <div class="grid grid-cols-1 xl:grid-cols-2 gap-8 mb-10 text-black">
      <div class="bg-white p-6 border-2 border-black rounded-2xl shadow-sm">
        <h2 class="text-xl font-bold text-[rgb(0,30,98)] mb-4">Самые популярные товары</h2>
        <div class="h-80 relative">
          <Bar
              v-if="analyticsStore.mostLikedProducts.length > 0"
              :data="productChartData"
              :options="productChartOptions"
          />
          <span v-else class="text-gray-400 italic flex items-center justify-center h-full">Нет данных для анализа</span>
        </div>
      </div>

      <div class="bg-white p-6 border-2 border-black rounded-2xl shadow-sm">
        <h2 class="text-xl font-bold text-[rgb(0,30,98)] mb-4">Самые популярные вариации</h2>
        <div class="h-[500px] relative w-full">
          <Bar
              v-if="analyticsStore.mostLikedVariants.length > 0"
              :data="variantChartData"
              :options="variantChartOptions"
          />
          <span v-else class="text-gray-400 italic flex items-center justify-center h-full">Нет данных для анализа</span>
        </div>
      </div>
    </div>
```

```
    <div class="grid grid-cols-1 xl:grid-cols-2 gap-8">

      <div class="bg-white border-2 border-black rounded-2xl p-6">
        <h3 class="font-black text-lg text-[rgb(0,30,98)] mb-4 uppercase">Таблица: Популярные товары</h3>
        <table class="w-full text-left border-collapse">
          <thead>
          <tr class="border-b-2 border-black text-xs uppercase text-gray-500">
            <th class="pb-2">Номер Товара</th>
            <th class="pb-2">Название</th>
            <th class="pb-2 text-right">Лайки</th>
          </tr>
          </thead>
          <tbody>
          <tr v-for="item in analyticsStore.mostLikedProducts" :key="item.productId" class="border-b border-gray-200 text-sm">
            <td class="py-3 font-mono">#{{ item.productId }}</td>
            <td class="py-3 font-semibold">{{ item.productName }}</td>
            <td class="py-3 text-right font-bold text-blue-600">{{ item.likesCount }} шт.</td>
          </tr>
          </tbody>
        </table>
      </div>

      <div class="bg-white border-2 border-black rounded-2xl p-6">
        <h3 class="font-black text-lg text-[rgb(0,30,98)] mb-4 uppercase">Таблица: Популярные вариации</h3>
        <table class="w-full text-left border-collapse">
          <thead>
          <tr class="border-b-2 border-black text-xs uppercase text-gray-500">
            <th class="pb-2">Номер Вариации</th>
            <th class="pb-2">Характеристики</th>
            <th class="pb-2 text-right">Лайки</th>
          </tr>
          </thead>
          <tbody>
          <tr v-for="item in analyticsStore.mostLikedVariants" :key="item.productVariantId" class="border-b border-gray-200 text-sm">
            <td class="py-3 font-mono">#{{ item.productVariantId }}</td>
            <td class="py-3 font-semibold">{{ item.variantName }}</td>
            <td class="py-3 text-right font-bold text-green-600">{{ item.likesCount }} шт.</td>
          </tr>
          </tbody>
        </table>
      </div>

    </div>
  </div>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\components\AppAdminOrdersWindow.vue

```
<script setup>
import { onMounted, ref, computed } from 'vue'
import useOrderStore from "@/stores/order.js"
import AppOrderCard from "@/components/AppOrderCard.vue";
import AppOrderEditStatus from "@/components/AppEditStatusOrderModal.vue";


const orderStore = useOrderStore()

onMounted(async () => {
  await orderStore.getAllOrders()
})

const stats = computed(() => {
  const orders = orderStore.order || []
  return {
    total: orders.length,
    new: orders.filter(o => o.status === 'Ожидает').length,
    processing: orders.filter(o => o.status === 'В обработке' || o.status === 'Отправлен').length,
    delivered: orders.filter(o => o.status === 'Доставлен').length
  }
})

</script>
```

```
<template>
  <div class="flex flex-col gap-8">
    <div class="flex flex-col gap-6">
      <h2 class="text-3xl font-black uppercase italic">Управление заказами</h2>

      <div class="grid grid-cols-4 gap-4">
         <div v-for="(val, label) in { 'Всего заказов': stats.total, 'Ожидают': stats.new, 'В работе': stats.processing, 'Завершено':
stats.delivered }"
             :key="label"
             class="bg-white border-2 border-black p-5 rounded-xl ">
          <p class="text-[10px] font-black uppercase text-gray-400 mb-1">{{ label }}</p>
          <p class="text-2xl font-black">{{ val }}</p>
        </div>
      </div>
    </div>

    <div class="bg-white border-2 border-black rounded-2xl overflow-hidden shadow-[10px_10px_0px_0px_rgba(0,0,0,1)]">
       <div class="flex items-center justify-between py-4 px-6 border-b-2 border-black bg-gray-50 font-black text-[10px] text-gray-500
uppercase tracking-widest">
        <div class="w-1/6">ID / Дата</div>
        <div class="w-1/4">Пользователь</div>
        <div class="w-1/6 text-center">Сумма</div>
        <div class="w-1/6 text-center">Статус</div>
        <div class="w-1/6 text-right">Действия</div>
      </div>

      <div class="flex flex-col">
        <app-order-card
            v-for="order in orderStore.order"
            :key="order.id"
            :order="order"
        />
      </div>
    </div>

  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppAdminProducts.vue

```
<script setup>
import useProductStore from "@/stores/product.js";
import { ref, onBeforeMount } from "vue";
import AppEditProductModal from "@/components/AppEditProductModal.vue";
import {useToast} from "vue-toastification";
import shoesAlt from "@/assets/shoes_alt.png";

const props = defineProps({
  product: { type: Object, required: true }
})
const isModalOpen = ref(false)
const productStore = useProductStore()
const toast = useToast()

const productImage = ref(null)
const imageSrc = ref('')
onBeforeMount(async () => {
  productImage.value = await productStore.getProductImage(props.product.id)
  if (productImage.value && productImage.value.imageUrl) {
    imageSrc.value = `http://localhost:5000/images/${productImage.value.imageUrl}`
  } else {
    imageSrc.value = shoesAlt
  }
})

const handleImageError = () => {
  imageSrc.value = shoesAlt
}

const confirmDelete = async () => {
  if (confirm(`Удалить товар "${props.product.name}"?`)) {
    await productStore.deleteProduct(props.product.id)
    toast.success("Товар удален")
    await productStore.productFun()
  }
```

```
}
</script>

<template>
  <div class="flex items-center justify-between py-4 border-b border-gray-200 hover:bg-gray-50 transition-colors px-6">

    <div class="flex items-center gap-4 w-2/5">
      <div class="w-16 h-16 bg-gray-100 rounded flex items-center justify-center overflow-hidden shrink-0 border border-gray-200">
        <img :src="imageSrc" alt="product" class="object-contain w-full h-full" @error="handleImageError" />
      </div>
      <span class="font-bold text-gray-900">{{ product.name }}</span>
    </div>

    <div class="w-1/4 text-gray-600 font-medium text-sm uppercase">
      {{ product.category }}
    </div>

    <div class="w-1/6 font-bold text-gray-900">
      ₽{{ product.basePrice }}
    </div>

    <div class="w-1/6 flex justify-end gap-4 text-gray-500">
      <button class="hover:text-black transition-colors hover:cursor-pointer" title="Редактировать" @click="isModalOpen = true">
        <span class="material-icons" ></span>
      </button>
      <button class="hover:text-red-500 transition-colors hover:cursor-pointer" title="Удалить" @click="confirmDelete">
        <span class="material-icons"></span>
      </button>
    </div>
    <teleport to="body">
      <app-edit-product-modal
          :is-modal-open="isModalOpen"
          :product="product"
          @close-modal="isModalOpen = false"
      />
    </teleport>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppAdminProductsWindow.vue

```
<script setup>

import AppAdminProduct from "@/components/AppAdminProducts.vue";
import AppAddProduct from "@/components/AppAddProductModal.vue";
import AppAddVariant from "@/components/AppAddVariantModal.vue";
import AppEditVariant from "@/components/AppEditVariantModal.vue";
import useProductStore from "@/stores/product.js";
import {onBeforeMount, ref} from "vue";

const productStore = useProductStore()
const isEditVariantOpen = ref(false)
const isAddVariantOpen = ref(false)
const isAddProductOpen = ref(false)
onBeforeMount(async () => {
  await productStore.productFun()
})
</script>

<template>
  <div class="flex justify-between items-center mb-10">
    <h2 class="text-3xl font-black uppercase ">Товары</h2>

    <div class="flex gap-3">
      <div class="flex border-2 border-black rounded-xl overflow-hidden ">
                <button @click="isEditVariantOpen = true" class="bg-white text-black px-4 py-3 font-bold text-[10px] uppercase
hover:bg-gray-100 border-r-2 border-black transition-colors">
          Управление остатками
        </button>
                 <button @click="isAddVariantOpen = true" class="bg-white text-black px-4 py-3 font-bold text-[10px] uppercase
hover:bg-gray-100 transition-colors">
          Добавить вариацию
        </button>
      </div>
```

```
      <button class="bg-black text-white px-6 py-3 rounded-xl font-black text-xs uppercase hover:bg-gray-800 transition-all"
              @click="isAddProductOpen = true">
        + Добавить товар
      </button>
    </div>
  </div>

  <div  class="bg-white border border-gray-200 rounded-lg shadow-sm">

     <div  class="flex items-center justify-between py-4 px-6 border-b-2 border-gray-100 font-bold text-xs text-gray-400 uppercase
tracking-wider">
      <div class="w-2/5">Фотография & Название</div>
      <div class="w-1/4">Категория</div>
      <div class="w-1/6">Цена</div>
      <div class="w-1/6 text-right">Действия</div>
    </div>

    <div class="flex flex-col">
      <app-admin-product
          v-for="product in productStore.products"
          :key="product.id"
          :product="product"
      />
    </div>
  </div>
  <teleport to="body">
    <app-edit-variant v-if="isEditVariantOpen" :is-open="isEditVariantOpen" @close="isEditVariantOpen = false" />
    <app-add-variant v-if="isAddVariantOpen" :is-open="isAddVariantOpen" @close="isAddVariantOpen = false" />
    <app-add-product v-if="isAddProductOpen" :is-open="isAddProductOpen" @close="isAddProductOpen = false"/>
  </teleport>
</template>

<style scoped>

</style>
```

[OTHER] Project_ASICS\src\components\AppAdminUsersWindow.vue

```
<script setup>
import { onMounted, computed } from 'vue'
import useUserStore from "@/stores/user.js"

const userStore = useUserStore()

onMounted(async () => {
  await userStore.getAllUsers()
})

const stats = computed(() => {
  const users = userStore.allUsers || []
  return {
    total: users.length,
    admins: users.filter(u => u.role === 'Менеджер').length,
    clients: users.filter(u => u.role === 'Клиент').length
  }
})
</script>

<template>
  <div class="flex flex-col gap-8">
    <div class="flex flex-col gap-6">
      <h2 class="text-3xl font-black uppercase italic">Управление пользователями</h2>

      <div class="grid grid-cols-3 gap-4">
        <div v-for="(val, label) in { 'Всего пользователей': stats.total, 'Менеджеры': stats.admins, 'Клиенты': stats.clients }"
             :key="label"
             class="bg-white border-2 border-black p-5 rounded-xl ">
          <p class="text-[10px] font-black uppercase text-gray-400 mb-1">{{ label }}</p>
          <p class="text-2xl font-black">{{ val }}</p>
        </div>
      </div>
    </div>

    <div class="bg-white border-2 border-black rounded-2xl overflow-hidden ">
       <div class="flex items-center justify-between py-4 px-6 border-b-2 border-black bg-gray-50 font-black text-[10px] text-gray-500
```

```
uppercase tracking-widest">
        <div class="w-1/12 text-center">ID</div>
        <div class="w-1/4">ФИО Пользователя</div>
        <div class="w-1/4 text-center">Email</div>
        <div class="w-1/6 text-center">Телефон</div>
        <div class="w-1/6 text-right">Роль</div>
      </div>

      <div class="flex flex-col">
        <app-admin-user
            v-for="user in userStore.allUsers"
            :key="user.id"
            :user="user"
        />
      </div>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppAiCoachWidget.vue

```
<script setup>
import { ref, nextTick } from 'vue'
import { useRouter } from 'vue-router'
import { useAiStore } from '@/stores/ai.js'

const router = useRouter()
const aiStore = useAiStore()

const isOpen = ref(false)
const userMessage = ref('')
const messagesContainer = ref(null)

const messages = ref([
  {
    role: 'assistant',
    text: 'Привет! Я твой персональный ИИ-ассистент от ASICS. Напишите свой вес, дистанцию, покрытие или особенности стопы. Если есть
особые пожелания к одежде или кроссовкам — например, нужен влагозащитный верх или вентилируемые ткани — обязательно укажите их!',
    recommendations: []
  }
])

const scrollToBottom = async () => {
  await nextTick()
  if (messagesContainer.value) {
    messagesContainer.value.scrollTop = messagesContainer.value.scrollHeight
  }
}

const sendMessage = async () => {
  const text = userMessage.value.trim()
  // Блокируем пустые запросы или спам во время загрузки
  if (!text || aiStore.isCoachLoading) return

  messages.value.push({ role: 'user', text })
  userMessage.value = ''
  scrollToBottom()

  const data = await aiStore.askCoach(text)
  if (data) {
    messages.value.push({
      role: 'assistant',
      text: data.reply,
      recommendations: data.recommendations || []
    })
  }
  scrollToBottom()
}

const openProduct = (productId) => {
  isOpen.value = false
  router.push({ name: 'Product', params: { id: productId } })
}
</script>
```

```
<template>
  <div class="fixed bottom-6 right-6 z-[9999] flex flex-col items-end">
    <!-- Окно чата -->
    <div
      v-if="isOpen"
        class="bg-white border-2 border-black w-88 sm:w-[420px] h-[520px] rounded-2xl shadow-[10px_10px_0px_0px_rgba(0,0,0,1)] flex
flex-col mb-4 overflow-hidden"
    >
      <!-- Шапка -->
      <div class="bg-[rgb(0,30,98)] text-white p-4 flex justify-between items-center border-b-2 border-black">
        <div class="flex items-center gap-2">
          <span class="text-2xl">🤖</span>
          <div>
            <h3 class="font-black uppercase tracking-tight text-sm">ИИ-ассистент</h3>
            <p class="text-[10px] text-gray-300 uppercase">Подбор экипировки ASICS</p>
          </div>
        </div>
        <button
          @click="isOpen = false"
          class="text-white hover:text-yellow-300 font-black text-xl px-2 transition-colors cursor-pointer"
        >

        </button>
      </div>

      <!-- Лента сообщений -->
      <div ref="messagesContainer" class="flex-1 p-4 overflow-y-auto bg-gray-50 flex flex-col gap-3">
        <div
          v-for="(msg, idx) in messages"
          :key="idx"
          :class="['flex flex-col max-w-[85%]', msg.role === 'user' ? 'self-end items-end' : 'self-start items-start']"
        >
          <div
            :class="[
              'p-3 rounded-xl border-2 border-black text-xs font-medium leading-relaxed',
              msg.role === 'user'
                ? 'bg-yellow-300 text-black shadow-[3px_3px_0px_0px_rgba(0,0,0,1)]'
                : 'bg-white text-black shadow-[3px_3px_0px_0px_rgba(0,0,0,1)]'
            ]"
          >
            {{ msg.text }}
          </div>

          <!-- Рекомендованные карточки товаров -->
          <div v-if="msg.recommendations && msg.recommendations.length > 0" class="mt-2 flex flex-col gap-2 w-full">
            <div
              v-for="prod in msg.recommendations"
              :key="prod.productId"
              @click="openProduct(prod.productId)"
                    class="bg-white border-2 border-black p-2.5 rounded-lg shadow-[3px_3px_0px_0px_rgba(0,30,98,1)] hover:bg-blue-50
cursor-pointer transition-all flex justify-between items-center"
            >
              <div>
                <p class="font-black text-xs uppercase text-[rgb(0,30,98)]">{{ prod.productName }}</p>
                <p class="text-[10px] text-gray-500 font-bold">₽{{ prod.price }}</p>
              </div>
              <span class="text-xs font-black text-blue-600 uppercase underline">Смотреть →</span>
            </div>
          </div>
        </div>

                    <div   v-if="aiStore.isCoachLoading"   class="self-start   bg-white   border-2   border-black   p-2.5   rounded-xl
shadow-[3px_3px_0px_0px_rgba(0,0,0,1)]">
          <span class="text-xs font-bold text-gray-500 animate-pulse">Тренер анализирует каталог...</span>
        </div>
      </div>

      <!-- Ввод сообщения с защитой от спама -->
      <div class="p-3 bg-white border-t-2 border-black flex gap-2">
        <input
          v-model="userMessage"
          @keyup.enter="sendMessage"
          :disabled="aiStore.isCoachLoading"
          maxlength="1000"
          placeholder="Вес 85 кг, бегаю 10 км по асфальту..."
```

```
          class="flex-1 h-11 border-2 border-black px-3 text-xs font-bold outline-none focus:bg-yellow-50 disabled:bg-gray-100"
        />
        <button
          @click="sendMessage"
          :disabled="aiStore.isCoachLoading || !userMessage.trim()"
                    class="bg-black  text-white  px-4  h-11  border-2  border-black  font-black  uppercase  text-xs  hover:bg-gray-800
disabled:bg-gray-300 disabled:cursor-not-allowed active:translate-y-0.5 cursor-pointer"
        >

        </button>
      </div>
    </div>

    <!-- Кнопка вызова виджета -->
    <button
      @click="isOpen = !isOpen"
                          class="bg-black    text-white    hover:bg-[rgb(0,30,98)]    border-2    border-black    p-3.5    rounded-2xl
shadow-[5px_5px_0px_0px_rgba(0,0,0,1)] hover:translate-x-0.5 hover:translate-y-0.5 transition-all flex items-center gap-2 font-black
uppercase text-xs tracking-wider cursor-pointer"
    >
      <span class="text-lg">></span>
      <span class="hidden sm:inline">ИИ-ассистент от ASICS</span>
    </button>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppCartItem.vue

```
<script setup>
import useCartItemStore from '@/stores/cartItems.js'
import useUserStore from '@/stores/user.js'
import useProductStore from "@/stores/product.js"
import { computed, ref, onBeforeMount } from 'vue'
import shoesAlt from "@/assets/shoes_alt.png"

const props = defineProps({
  cartItem: {
    type: Object,
    required: true,
  },
})
const cartItemStore = useCartItemStore()
const userStore = useUserStore()
const productStore = useProductStore()

const imageSrc = ref('')

const handleImageError = () => {
  imageSrc.value = shoesAlt
}
onBeforeMount(async () => {
  const variantImage = await productStore.getVariantImage(props.cartItem.productVariantId)

  if (variantImage && variantImage.imagePath) {
    imageSrc.value = `http://localhost:5000/imagesVariant/${variantImage.imagePath}`
  } else {
    imageSrc.value = shoesAlt
  }
})

const prepareCartData = () => {
  return {
    userId: Number(props.cartItem.userId),
    productVariantId: Number(props.cartItem.productVariantId),
    quantity: 1,
    productName: props.cartItem.productName || '',
    color: props.cartItem.color || '',
    size: parseFloat(props.cartItem.size) || 0,
  }
}
const addToCart = async () => {
  const cart = prepareCartData()

  if (!isNaN(cart.userId)) {
    await cartItemStore.addToCart(cart)
```

```
    await cartItemStore.getCartItems(cart.userId)
  }
}

const minusToCart = async () => {
  const cart = prepareCartData()

  if (!isNaN(cart.userId)) {
    await cartItemStore.minusToCart(cart)
    await cartItemStore.getCartItems(cart.userId)
  }
}
const deleteCartItem = async () => {
  await cartItemStore.removeCart(props.cartItem.id)
  await cartItemStore.getCartItems(props.cartItem.userId)
}
const totalPrice = computed(() => {
  const price = Number(props.cartItem.price !== undefined ? props.cartItem.price : props.cartItem.Price) || 0
  const quantity = Number(props.cartItem.quantity !== undefined ? props.cartItem.quantity : props.cartItem.Quantity) || 0

  return Math.round(price * quantity * 100) / 100
})
defineExpose({
  totalPrice,
})
</script>

<template>
  <div
    class="flex items-center justify-between border border-gray-200 rounded-xl p-4 bg-white shadow-sm mb-4 w-full"
  >
    <div class="flex items-center gap-6">
      <div
        class="w-32 h-32 bg-gray-100 rounded-lg flex items-center justify-center overflow-hidden">
        <img :src="imageSrc" alt="shoes" class="object-contain w-full h-full mix-blend-multiply" @error="handleImageError" />
      </div>

      <div class="flex flex-col gap-1">
        <h2 class="text-xl font-bold uppercase tracking-tight">{{ cartItem.productName }}</h2>
        <p class="text-gray-500 text-sm">
          Цвет: <span class="text-black font-medium">{{ cartItem.color }}</span>
        </p>
        <p class="text-gray-500 text-sm">
          Размер: <span class="text-black font-medium">{{ cartItem.size }}</span>
        </p>
      </div>
    </div>

    <div class="flex flex-col items-end justify-between h-32">
      <button
        class="text-gray-400 hover:text-red-500 transition-colors text-xl"
        @click="deleteCartItem"
      >
        <span class="material-icons"></span>
      </button>

      <div class="flex items-center gap-6 flex-col">
        <div class="flex items-center border border-black rounded-md px-2 py-1 gap-4">
          <button
            class="text-xl font-medium hover:text-gray-500"
            v-if="userStore.user"
            @click="minusToCart"
          >
            -
          </button>
          <span class="text-lg font-semibold min-w-5 text-center">{{ cartItem.quantity }}</span>
          <button
            class="text-xl font-medium hover:text-gray-500"
            v-if="userStore.user"
            @click="addToCart"
          >
            +
          </button>
        </div>
        <p class="text-xl font-black">{{ totalPrice }} ₽</p>
```

```
      </div>
    </div>
  </div>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\components\AppCartModal.vue

```
<script setup>
import {useTemplateRef} from "vue";
import useOrderStore from "@/stores/order.js";
import {useToast} from "vue-toastification";
import useUserStore from "@/stores/user.js";
import useCartItemStore from "@/stores/cartItems.js";

const props = defineProps({
  isModalOpen: {
    type: Boolean,
    default: false,
  },
  totalPrice: {
    type: Number,
    required: true,
  }
})

const emit = defineEmits(['closeModal'])
const toast = useToast()


const orderStore = useOrderStore()
const userStore = useUserStore()
const cartStore = useCartItemStore()

const city = useTemplateRef('city')
const street = useTemplateRef('street')
const numberHouse = useTemplateRef('numberHouse')
const postalIndex = useTemplateRef('postalIndex')

const addOrder = async () => {
  if (
      !city.value.value ||
      !street.value.value ||
      !numberHouse.value.value ||
      !postalIndex.value.value
  ) {
    toast.error('Заполните все поля')
    return
  }
  const order = {
    id: 0,
    user: userStore.user.email,
    orderDate: '',
    status: 'Ожидает',
    totalAmount: props.totalPrice,
    city: city.value.value,
    street: street.value.value,
    house: numberHouse.value.value,
    postalCode: postalIndex.value.value,
  }
  await orderStore.addOrder(order)
  emit('closeModal')
  if (!orderStore.order) {
    return
  }
  await orderStore.addOrderItem(userStore.user.id,orderStore.order.id)
  cartStore.clearAllCart()
  toast.success('Успешно оформлено')

}
</script>

<template>
```

```
  <div
      v-if="isModalOpen"
      class="fixed inset-0 bg-black/50 flex items-center justify-center z-999 p-4"
  >
    <div
        class="bg-white w-full max-w-lg border-2 border-black p-8 flex flex-col relative rounded-2xl"
    >
      <button
          @click="emit('closeModal')
"
          class="absolute top-4 right-4 text-2xl font-bold cursor-pointer"
      >

      </button>

      <h1 class="font-black text-3xl mb-6 uppercase text-[rgb(0,30,98)]">Заполните адрес</h1>

      <div class="flex flex-col gap-4">
        <div class="flex flex-col">
          <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Город</label>
          <input
              ref="city"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="text"
          />
        </div>

        <div class="flex flex-col">
          <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Улица</label>
          <input
              ref="street"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="text"
          />
        </div>

        <div class="flex flex-col">
          <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Номер дома</label>
          <input
              ref="numberHouse"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="number"
              maxlength="3"
              max="999"
              min="1"
          />
        </div>

        <div class="flex flex-col">
          <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Почтовый индекс</label>
          <input
              ref="postalIndex"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="number"
              maxlength="6"
              max="999999"
              min="100000"
          />
        </div>

        <div class="flex flex-row justify-between gap-3 mt-6">
          <button
              @click="emit('closeModal')
"
                 class="w-full h-14 rounded-xl border border-black bg-blue-50 text-[rgb(0,30,98)] text-xl uppercase hover:bg-gray-100
cursor-pointer"
          >
            Отмена
          </button>
          <button
              @click="addOrder"
                       class="w-full  h-14  rounded-xl  bg-[rgb(0,30,98)]  text-white  font-black  text-xl  uppercase  hover:bg-gray-800
cursor-pointer"
          >
            Оформить
```

```
          </button>
        </div>
      </div>
    </div>
  </div>
</template>

<style scoped>

</style>
```

[OTHER] Project_ASICS\src\components\AppEditProductModal.vue

```
<script setup>
import {ref, computed, watch, onMounted} from "vue";
import { useToast } from "vue-toastification";
import useProductStore from "@/stores/product.js";
import useCategoryStore from "@/stores/category.js";

const props = defineProps({
  isModalOpen: Boolean,
  product: Object
})
const emit = defineEmits(['closeModal'])
const toast = useToast()
const productStore = useProductStore()
const categoryStore = useCategoryStore()

onMounted(async () => {
  if (categoryStore.category.length === 0) {
    await categoryStore.getAllCategories()
  }
})
const save = async () => {
  try {
    await productStore.editProduct(props.product)
    toast.success("Данные успешно сохранены!")
    emit('closeModal')
    await productStore.productFun()
  } catch (e) {
    toast.error("Ошибка при сохранении")
  }
}
</script>

<template>
  <div v-if="isModalOpen" class="fixed inset-0 bg-black/60 flex items-center justify-center z-[1000] p-4 backdrop-blur-sm text-black">
            <div   class="bg-white   w-full   max-w-lg   border-2   border-black   p-8   flex   flex-col   relative   rounded-2xl
shadow-[10px_10px_0px_0px_rgba(0,0,0,1)]">

      <button @click="emit('closeModal')" class="absolute top-4 right-4 text-xl font-bold">×</button>

      <h1 class="font-black text-xl mb-6 uppercase italic border-b-2 border-black pb-2">Редактирование</h1>

      <div class="flex flex-col gap-4">
        <div class="flex flex-col">
          <label class="text-[10px] font-black uppercase text-gray-400">Название</label>
          <input v-model="product.name" class="h-10 border-2 border-black px-3 font-bold outline-none focus:bg-yellow-50" />
        </div>

        <div class="flex flex-col">
          <label class="text-[10px] font-black uppercase text-gray-400">Категория</label>
          <select v-model="product.category" class="h-10 border-2 border-black px-3 font-bold bg-white outline-none">
            <option v-for="cat in categoryStore.category" :key="cat.id" :value="cat.name">
              {{ cat.name }}
            </option>
          </select>
        </div>

        <div class="flex flex-col">
          <label class="text-[10px] font-black uppercase text-gray-400">Цена ($)</label>
          <input v-model="product.basePrice" type="number" class="h-10 border-2 border-black px-3 font-bold outline-none" />
        </div>

        <div class="flex flex-col">
```

```
          <label class="text-[10px] font-black uppercase text-gray-400">Описание</label>
          <textarea v-model="product.description" class="h-24 border-2 border-black p-3 font-medium outline-none resize-none" />
        </div>

        <div class="flex gap-3 mt-4">
          <button @click="save" class="flex-1 h-12 bg-black text-white font-black uppercase hover:bg-gray-800 transition-all">
            Сохранить
          </button>
              <button @click="emit('closeModal')" class="flex-1 h-12 border-2 border-black font-black uppercase hover:bg-gray-100
transition-all">
            Отмена
          </button>
        </div>
      </div>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppEditStatusOrderModal.vue

```
<script setup>
import { useToast } from "vue-toastification"
import useOrderStore from "@/stores/order.js"

const props = defineProps({
  order: { type: Object, required: true },
  isEditModalOpen: { type: Boolean, default: false }
})

const emit = defineEmits(['close'])
const orderStore = useOrderStore()
const toast = useToast()

const statuses = ['Ожидает', 'В обработке', 'Отправлен', 'Доставлен', 'Отменен']

const saveStatus = async () => {
  try {
    await orderStore.editOrderStatus(props.order)
    toast.success("Статус успешно обновлен!")
    emit('close')
  } catch (e) {
    toast.error("Ошибка при обновлении")
  }
}
</script>

<template>
    <div v-if="isEditModalOpen" class="fixed inset-0 bg-black/60 flex items-center justify-center z-[1000] p-4 backdrop-blur-sm
text-black">
              <div class="bg-white w-full max-w-md border-2 border-black p-8 flex flex-col relative rounded-2xl
shadow-[15px_15px_0px_0px_rgba(0,0,0,1)]">

      <button @click="emit('close')" class="absolute top-4 right-4 text-xl font-bold hover:scale-110 transition-transform"></button>

      <h1 class="font-black text-xl mb-6 uppercase italic border-b-2 border-black pb-2">Заказ #{{ order.id }}</h1>

      <div class="flex flex-col gap-6">
        <div class="bg-gray-50 border-2 border-black border-dashed p-4 flex flex-col gap-2">
          <div>
            <label class="text-[9px] font-black uppercase text-gray-400">Клиент</label>
            <p class="text-xs font-bold">{{ order.user }}</p>
          </div>
          <div>
            <label class="text-[9px] font-black uppercase text-gray-400">Адрес доставки</label>
            <p class="text-xs font-bold leading-tight">
              г. {{ order.city }}, ул. {{ order.street }}, д. {{ order.house }}<br>
              <span class="text-gray-500 italic">Индекс: {{ order.postalCode }}</span>
            </p>
          </div>
        </div>

        <div class="flex flex-col">
          <label class="text-[10px] font-black uppercase text-gray-400 mb-1">Изменить статус</label>
            <select v-model="order.status" class="h-12 border-2 border-black px-4 font-black bg-white outline-none focus:bg-yellow-50
transition-colors cursor-pointer">
```

```
            <option v-for="status in statuses" :key="status" :value="status">
              {{ status }}
            </option>
          </select>
        </div>

        <div class="flex gap-3 mt-2">
            <button @click="saveStatus" class="flex-1 h-14 bg-black text-white font-black uppercase hover:bg-gray-800 transition-all
shadow-[4px_4px_0px_0px_rgba(0,0,0,0.3)]">
            Сохранить
          </button>
                <button @click="emit('close')" class="flex-1 h-14 border-2 border-black font-black uppercase hover:bg-gray-100
transition-all">
            Отмена
          </button>
        </div>
      </div>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppEditVariantModal.vue

```
<script setup>
import { ref, computed } from 'vue'
import useProductStore from "@/stores/product.js"
import { useToast } from "vue-toastification"

const props = defineProps(['isOpen'])
const emit = defineEmits(['close'])
const productStore = useProductStore()
const toast = useToast()

const selectedProductId = ref(null)
const selectedColor = ref('')
const selectedSize = ref('')

const selectedProduct = computed(() =>{
    if (!productStore.products || !Array.isArray(productStore.products)) return null;
    return productStore.products.find(p => p.id === selectedProductId.value)
})

const colors = computed(() =>
    selectedProduct.value ? [...new Set(selectedProduct.value.variants.map(v => v.color))] : []
)

const sizes = computed(() =>
    selectedProduct.value ? selectedProduct.value.variants.filter(v => v.color === selectedColor.value).map(v => v.size) : []
)

const currentVariant = computed(() =>
    selectedProduct.value?.variants.find(v => v.color === selectedColor.value && String(v.size) === String(selectedSize.value))
)

const save = async () => {
  if (!currentVariant.value) return
  if (currentVariant.value.stockQuantity < 0) {
    toast.error("Количество товара на складе не может быть меньше 0!")
    return
  }
  await productStore.editProductVariant(currentVariant.value)
  toast.success("Количество обновлено")
}

const remove = async () => {
  if (!currentVariant.value) return
  if (confirm("Удалить эту вариацию?")) {
    await productStore.deleteProductVariant(currentVariant.value.id)
    toast.success("Вариация удалена")
    await productStore.productFun()
    emit('close')
  }
}
</script>
```

```
<template>
  <div class="fixed inset-0 bg-black/60 flex items-center justify-center z-[1000] p-4 backdrop-blur-sm text-black">
    <div class="bg-white w-full max-w-md border-2 border-black p-6 rounded-2xl shadow-[10px_10px_0px_0px_rgba(0,0,0,1)]">
      <h2 class="font-black uppercase mb-4 italic">Управление остатками</h2>

      <div class="flex flex-col gap-4">
        <select v-model="selectedProductId" class="h-10 border-2 border-black px-2 font-bold">
          <option :value="null">Выберите товар...</option>
          <option v-for="p in productStore.products" :key="p.id" :value="p.id">{{ p.name }}</option>
        </select>

        <div v-if="selectedProductId" class="grid grid-cols-2 gap-2">
          <select v-model="selectedColor" class="h-10 border-2 border-black px-2">
            <option value="">Цвет</option>
            <option v-for="c in colors" :key="c" :value="c">{{ c }}</option>
          </select>
          <select v-model="selectedSize" class="h-10 border-2 border-black px-2">
            <option value="">Размер</option>
            <option v-for="s in sizes" :key="s" :value="s">{{ s }}</option>
          </select>
        </div>

        <div v-if="currentVariant" class="bg-gray-100 p-4 border-2 border-black border-dashed">
          <label class="text-[10px] font-bold uppercase">Количество на складе</label>
          <input v-model="currentVariant.stockQuantity" type="number" min="0" class="w-full h-10 border-2 border-black px-3 mt-1" />

          <div class="flex gap-2 mt-4">
            <button @click="save" class="flex-1 bg-black text-white py-2 font-bold uppercase text-[10px]">Сохранить</button>
            <button @click="remove" class="flex-1 bg-red-600 text-white py-2 font-bold uppercase text-[10px]">Удалить</button>
          </div>
        </div>
      </div>

      <button @click="emit('close')" class="w-full mt-4 text-[10px] font-bold uppercase underline">Закрыть</button>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppFavoritesProduct.vue

```
<script setup>
import { useRouter } from 'vue-router'
import { ref, onMounted } from 'vue'
import useProductStore from '@/stores/product.js'
import useUserStore from '@/stores/user.js'
import useAnalyticsStore from '@/stores/analytics.js'
import shoesAlt from "@/assets/shoes_alt.png"

const props = defineProps({
  item: {
    type: Object,
    required: true
  }
})

const router = useRouter()
const productStore = useProductStore()
const userStore = useUserStore()
const analyticsStore = useAnalyticsStore()
const imageSrc = ref(shoesAlt)

onMounted(async () => {
  const product = await productStore.getProduct(props.item.productId)
  const firstVariant = product?.variants?.[0]

  if (firstVariant) {
    const variantImage = await productStore.getVariantImage(firstVariant.id)
    if (variantImage && variantImage.imagePath) {
      imageSrc.value = `http://localhost:5000/imagesVariant/${variantImage.imagePath}`
      return
    }
  }
  imageSrc.value = shoesAlt
})
```

```
const goToProduct = () => {
  router.push({ name: 'Product', params: { id: props.item.productId } })
}

const removeProductLike = (e) => {
  if (userStore.user) {
    analyticsStore.toggleProductFavorite(userStore.user.id, props.item.productId)
  }
}
</script>

<template>
  <div
      @click="goToProduct"
                        class="bg-white    border-4    border-black    p-4    rounded-xl    hover:translate-x-1    hover:translate-y-1
hover:shadow-[4px_4px_0px_0px_rgba(0,0,0,1)] transition-all cursor-pointer flex flex-col justify-between h-full text-black"
  >
    <div class="h-48 w-full flex items-center justify-center overflow-hidden border-2 border-black border-dashed mb-4 p-2 bg-gray-50">
      <img :src="imageSrc" alt="Product image" class="h-full object-contain mix-blend-multiply" />
    </div>

    <div class="flex-1 flex flex-col justify-between">
      <div class="flex items-start justify-between gap-2">
        <h3 class="font-black uppercase text-xl leading-tight line-clamp-2">{{ item.productName }}</h3>

        <button
            @click.stop="removeProductLike"
                 class="p-1.5 border-2 border-black rounded-lg bg-white shadow-[2px_2px_0px_0px_rgba(0,0,0,1)] hover:translate-y-0.5
hover:shadow-[1px_1px_0px_0px_rgba(0,0,0,1)] transition-all flex items-center justify-center shrink-0"
            title="Удалить из избранного"
        >
          <svg
              xmlns="http://www.w3.org/2000/svg"
              class="h-5 w-5 text-red-500 fill-current"
              viewBox="0 0 24 24"
              stroke="currentColor"
              stroke-width="2.5"
          >
            <path
                stroke-linecap="round"
                stroke-linejoin="round"
                   d="M4.318 6.318a4.5 4.5 0 000 6.364L12 20.364l7.682-7.682a4.5 4.5 0 00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5 0
00-6.364 0z"
            />
          </svg>
        </button>
      </div>
      <p class="text-xs text-gray-500 font-bold uppercase mt-1">Товар целиком</p>
    </div>

    <div class="mt-4 flex items-center justify-between">
      <span class="font-mono text-sm text-gray-400">#{{ item.productId }}</span>
      <span class="text-blue-800 font-black uppercase text-xs underline">Посмотреть размеры →</span>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppFavoritesVariant.vue

```
<script setup>
import { useRouter } from 'vue-router'
import { ref, onMounted } from 'vue'
import useProductStore from '@/stores/product.js'
import useUserStore from '@/stores/user.js'
import useAnalyticsStore from '@/stores/analytics.js'
import shoesAlt from "@/assets/shoes_alt.png"

const props = defineProps({
  item: {
    type: Object,
    required: true
  }
})

const router = useRouter()
```

```
const productStore = useProductStore()
const userStore = useUserStore()
const analyticsStore = useAnalyticsStore()
const imageSrc = ref(shoesAlt)

onMounted(async () => {
  if (props.item.productVariantId) {
    const variantImage = await productStore.getVariantImage(props.item.productVariantId)
    if (variantImage && variantImage.imagePath) {
      imageSrc.value = `http://localhost:5000/imagesVariant/${variantImage.imagePath}`
      return
    }
  }
  imageSrc.value = shoesAlt
})

const goToVariant = () => {
  router.push({
    name: 'Product',
    params: { id: props.item.productId },
    query: {
      color: props.item.color,
      size: props.item.size
    }
  })
}

const removeVariantLike = (e) => {
  if (userStore.user) {
    analyticsStore.toggleVariantFavorite(userStore.user.id, props.item.productVariantId)
  }
}
</script>

<template>
  <div
      @click="goToVariant"
                      class="bg-white    border-4    border-black    p-4    rounded-xl       hover:translate-x-1    hover:translate-y-1
hover:shadow-[4px_4px_0px_0px_rgba(0,0,0,1)] transition-all cursor-pointer flex flex-col justify-between h-full text-black"
  >
    <div class="h-48 w-full flex items-center justify-center overflow-hidden border-2 border-black border-dashed mb-4 p-2 bg-gray-50">
      <img :src="imageSrc" alt="Variant image" class="h-full object-contain mix-blend-multiply" />
    </div>

    <div class="flex-1 flex flex-col justify-between">
      <div class="flex items-start justify-between gap-2">
        <h3 class="font-black uppercase text-xl leading-tight line-clamp-2">{{ item.variantName }}</h3>

        <button
            @click.stop="removeVariantLike"
                 class="p-1.5 border-2 border-black rounded-lg bg-white shadow-[2px_2px_0px_0px_rgba(0,0,0,1)] hover:translate-y-0.5
hover:shadow-[1px_1px_0px_0px_rgba(0,0,0,1)] transition-all flex items-center justify-center shrink-0"
            title="Удалить размер из избранного"
        >
          <svg
              xmlns="http://www.w3.org/2000/svg"
              class="h-5 w-5 text-red-500 fill-current"
              viewBox="0 0 24 24"
              stroke="currentColor"
              stroke-width="2.5"
          >
            <path
                stroke-linecap="round"
                stroke-linejoin="round"
                   d="M4.318 6.318a4.5 4.5 0 000 6.364L12 20.364l7.682-7.682a4.5 4.5 0 00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5 0
00-6.364 0z"
            />
          </svg>
        </button>
      </div>

      <div class="flex flex-wrap gap-2 mt-2">
        <span class="bg-emerald-100 border border-black text-xs font-black px-2 py-0.5 rounded uppercase">
          Цвет: {{ item.color }}
        </span>
```

```
        <span class="bg-blue-100 border border-black text-xs font-black px-2 py-0.5 rounded uppercase">
          Размер: {{ item.size }}
        </span>
      </div>
    </div>

    <div class="mt-4 flex items-center justify-between">
      <span class="font-mono text-sm text-gray-400">#{{ item.productVariantId }}</span>
      <span class="text-emerald-700 font-black uppercase text-xs underline">Купить этот размер →</span>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppFooters.vue

```
<script setup></script>

<template>
  <footer class="font-[Franklin] bg-black text-white pt-16 pb-8 mt-10">
    <div class="container mx-auto px-6 flex justify-around items-start gap-10">
      <div class="flex flex-col max-w-sm">
        <h1 class="font-bold uppercase tracking-widest mb-6">О нас</h1>
        <p class="text-gray-400 text-sm leading-relaxed">
          ASICS — это сочетание японских традиций и инновационных технологий. Мы создаем экипировку,
          которая помогает атлетам достигать новых высот, обеспечивая максимальный комфорт и
          поддержку в каждом движении.
        </p>
      </div>

      <div class="flex flex-col">
        <h1 class="font-bold uppercase tracking-widest mb-6">Контакты</h1>
        <div class="text-gray-400 text-sm space-y-2">
          <p>Почта: rezervrafael@gmail.com</p>
          <p>Телефон: 89871432578</p>
          <p>Адрес: г.Уфа, ул.Кирова, 47</p>
        </div>
      </div>

      <div class="flex flex-col">
        <h1 class="font-bold uppercase tracking-widest mb-6">Подпишитесь на нас</h1>
        <div class="text-gray-400 text-sm">
          <p>Instagram | Facebook | Twitter</p>
        </div>
      </div>
    </div>

    <div class="border-t border-gray-800 mt-16 pt-8 text-center text-gray-500 text-xs">
      © 2026 ASICS Europe. All rights reserved.
    </div>
  </footer>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\components\AppHeaders.vue

```
<script setup>
import { useRouter } from 'vue-router'
import useUserStore from '@/stores/user.js'
import user from '@/stores/user.js'

const router = useRouter()
const userStore = useUserStore()
</script>

<template>
  <header class="bg-white shadow-md">
    <nav class="container mx-auto px-6 py-4 flex items-center justify-between">
      <div class="flex items-center justify-between gap-3">
        <router-link to="/">
          <img src="../assets/asics-seeklogo.png" alt="Logo" class="h-15 w-40" />
        </router-link>
      </div>
```

```
      <div class="flex items-center space-x-6 text-gray-600">
        <router-link class="hover:text-blue-500" to="/catalog">Товары</router-link>
        <router-link class="hover:text-blue-500" to="/profile" v-if="userStore.user"
          >Мой профиль</router-link
        >
        <router-link class="hover:text-blue-500" to="/admin" v-if="userStore.user?.role === 'Менеджер'"
          >Услуги</router-link
        >
      </div>
      <div class="flex items-center space-x-6 text-gray-600">
        <button
            v-if="userStore.user"
            class="flex items-center gap-3 text-4xl  hover:scale-110 transition-transform"
            @click="router.push({ name: 'Favorite' })"
            title="Избранное"
        >

        </button>
        <router-link to="/cart">
          <button
            class="flex items-center gap-3 text-2xl"
            v-if="userStore.user"
            @click="router.push({ name: 'Cart' })"
          >

          </button>
        </router-link>
        <span v-if="userStore.user">{{ userStore.user.name }} {{ userStore.user.surname }}</span>
        <button
          class="bg-black text-white px-4 py-2 rounded-full text-1xl"
          @click="router.push({ name: 'Authorization' })"
          v-if="!userStore.user"
        >
          Войти
        </button>
      </div>
    </nav>
  </header>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\components\AppOrderCard.vue

```
<script setup>
import AppOrderEditStatus from "@/components/AppEditStatusOrderModal.vue";
import { ref } from "vue";

const props = defineProps({
  order: { type: Object, required: true }
})

const isEditModalOpen = ref(false)
</script>

<template>
  <div class="flex items-center justify-between py-5 px-6 border-b border-gray-100 hover:bg-gray-50 transition-colors font-bold">
    <div class="w-1/6">
      <p class="text-blue-600">#{{ order.id }}</p>
      <p class="text-[9px] text-gray-400 uppercase tracking-tighter">{{ order.orderDate }}</p>
    </div>

    <div class="w-1/4 truncate text-xs">
      {{ order.user }}
    </div>

    <div class="w-1/6 text-center text-sm">
      ₽{{ order.totalAmount }}
    </div>

    <div class="w-1/6 text-center">
      <span class="px-3 py-1 rounded-full text-[9px] font-black uppercase border-2 border-black bg-yellow-300 ">
        {{ order.status }}
      </span>
```

```
    </div>

    <div class="w-1/6 text-right">
           <button @click="isEditModalOpen = true" class="p-2 hover:bg-black hover:text-white border-2 border-black rounded-lg
transition-all">

      </button>
    </div>

    <teleport to="body">
      <app-order-edit-status
          :is-edit-modal-open="isEditModalOpen"
          :order="order"
          @close="isEditModalOpen = false"
      />
    </teleport>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppProductCard.vue

```
<script setup>
import {computed, onBeforeMount, ref} from "vue";
import useProductStore from "@/stores/product.js";
import shoesAlt from "@/assets/shoes_alt.png";
import useAnalyticsStore from "@/stores/analytics.js";
import useUserStore from "@/stores/user.js";

const props = defineProps({
  product: {
    type: Object,
    required: true,
  },
})

const productStore = useProductStore();
const userStore = useUserStore();
const analyticsStore = useAnalyticsStore();

const productImage = ref(null)

const imageSrc = ref('')

onBeforeMount(async () => {
  productImage.value = await productStore.getProductImage(props.product.id)
  if (productImage.value && productImage.value.imageUrl) {
    imageSrc.value = `http://localhost:5000/images/${productImage.value.imageUrl}`
  } else {
    imageSrc.value = shoesAlt
  }
  if (userStore.user && analyticsStore.userProductFavorites.length === 0) {
    await analyticsStore.fetchUserFavorites(userStore.user.id)
  }
})

const handleImageError = () => {
  imageSrc.value = shoesAlt
}

const isLiked = computed(() => {
  return analyticsStore.userProductFavorites.includes(props.product.id)
})
const handleLikeClick = async () => {
  if (!userStore.user) {
    alert("Для добавления в избранное необходимо авторизоваться!")
    return
  }
  await analyticsStore.toggleProductFavorite(userStore.user.id, props.product.id)
}
</script>

<template>
  <div
    class="flex justify-start flex-col border border-black rounded-xl overflow-hidden bg-white p-3 m-2 relative"
  >
```

```
    <img :src="imageSrc" alt="logo" class="w-50 h-50 object-contain p-2" @error="handleImageError" />
    <h2>{{ product.name }}</h2>
    <span>{{ product.category }}</span>
    <p>{{ product.basePrice }} ₽</p>
    <button
        @click.stop="handleLikeClick"
            class="absolute bottom-3 right-3 p-2 rounded-full border border-gray-200 hover:bg-gray-100 transition-colors bg-white
shadow-sm"
    >
      <svg
          xmlns="http://www.w3.org/2000/svg"
          :class="['h-6 w-6 transition-colors', isLiked ? 'text-red-500 fill-current' : 'text-gray-400']"
          fill="none"
          viewBox="0 0 24 24"
          stroke="currentColor"
      >
        <path
            stroke-linecap="round"
            stroke-linejoin="round"
            stroke-width="2"
                d="M4.318 6.318a4.5 4.5 0 000 6.364L12 20.364l7.682-7.682a4.5 4.5 0 00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5 0
00-6.364 0z"
        />
      </svg>
    </button>
  </div>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\components\AppProfileOrder.vue

```
<script setup>
import { ref } from 'vue'
import AppProfileOrderItem from './AppProfileOrderItem.vue'

const props = defineProps({
  order: {
    type: Object,
    required: true
  }
})

const isExpanded = ref(false)
</script>

<template>
  <div class="w-full border border-black p-4 bg-white flex flex-col mb-4">
    <div class="flex items-center justify-between w-full">
      <div class="flex flex-col gap-1">
        <span class="font-black text-lg text-[rgb(0,30,98)] uppercase">Заказ #ASICS-{{ order.orderId }}</span>
        <span class="text-xs text-gray-500 font-medium">Дата доставки: {{ order.orderDate }}</span>
      </div>

      <div class="flex items-center gap-6">
        <div class="text-right">
          <span class="text-xs text-gray-400 uppercase font-bold block">Сумма заказа</span>
          <span class="font-black text-xl text-black">{{ order.totalAmount }} ₽</span>
        </div>

        <button
            @click="isExpanded = !isExpanded"
            class="text-2xl font-black text-[rgb(0,30,98)] hover:text-blue-600 cursor-pointer transition-transform duration-200"
            :class="{ 'rotate-180': isExpanded }"
        >
          ▼
        </button>
      </div>
    </div>

    <div v-if="isExpanded" class="mt-4 pt-4 border-t border-dashed border-gray-300 flex flex-col gap-2">
      <h4 class="text-xs font-black uppercase text-gray-400 mb-2 tracking-wider">Купленные товары:</h4>

      <AppProfileOrderItem
```

```
          v-for="item in order.items"
          :key="item.productVariantId"
          :item="item"
      />
    </div>
  </div>
</template>

<style scoped>
.rotate-180 {
  transform: rotate(180deg);
}
</style>
```

[OTHER] Project_ASICS\src\components\AppProfileOrderItem.vue

```
<script setup>
import useProductStore from "@/stores/product.js"
import { ref, onBeforeMount } from 'vue'
import shoesAlt from "@/assets/shoes_alt.png"


const props = defineProps({
  item: {
    type: Object,
    required: true,
  },
})

const productStore = useProductStore()
const imageSrc = ref('')

const handleImageError = () => {
  imageSrc.value = shoesAlt
}

onBeforeMount(async () => {
  const variantImage = await productStore.getVariantImage(props.item.productVariantId)

  if (variantImage && variantImage.imagePath) {
    imageSrc.value = `http://localhost:5000/imagesVariant/${variantImage.imagePath}`
  } else {
    imageSrc.value = shoesAlt
  }
})
</script>

<template>
  <div class="flex items-center justify-between border border-gray-100 rounded-xl p-3 bg-gray-50 mb-2 w-full">
    <div class="flex items-center gap-4">
      <div class="w-20 h-20 bg-white rounded-lg flex items-center justify-center overflow-hidden border border-gray-200">
        <img :src="imageSrc" alt="shoes" class="object-contain w-full h-full mix-blend-multiply" @error="handleImageError" />
      </div>

      <div class="flex flex-col gap-0.5">
        <h3 class="text-base font-bold uppercase tracking-tight text-black">{{ item.productName }}</h3>
        <p class="text-gray-500 text-xs">
          Цвет: <span class="text-black font-medium">{{ item.color }}</span>
        </p>
        <p class="text-gray-500 text-xs">
          Размер: <span class="text-black font-medium">{{ item.size }}</span>
        </p>
        <p class="text-gray-400 text-xs mt-1">
          Количество: <span class="text-black font-bold">{{ item.quantity }} шт.</span>
        </p>
      </div>
    </div>

    <div class="text-right">
      <p class="text-gray-400 text-xs">{{ item.price }} ₽ / шт.</p>
      <p class="text-lg font-black text-[rgb(0,30,98)] mt-1">{{ item.price * item.quantity }} ₽</p>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\components\AppUser.vue

```
<script setup>
defineProps({
  user: { type: Object, required: true }
})
</script>

<template>
  <div class="flex items-center justify-between py-5 px-6 border-b border-gray-100 hover:bg-gray-50 transition-colors">
    <div class="w-1/12 text-center font-black text-gray-300">
      #{{ user.id }}
    </div>

    <div class="w-1/4 flex items-center gap-3">
      <div class="w-10 h-10 bg-black rounded-full flex items-center justify-center text-white font-black text-xs shrink-0">
        {{ user.name[0] }}{{ user.surname[0] }}
      </div>
      <div class="flex flex-col">
        <p class="font-black text-sm leading-tight">{{ user.surname }} {{ user.name }}</p>
        <p class="text-[10px] text-gray-400 font-bold uppercase">{{ user.patronymic || '—' }}</p>
      </div>
    </div>

    <div class="w-1/4 text-center">
      <p class="font-bold text-xs truncate text-blue-600">{{ user.email }}</p>
    </div>

    <div class="w-1/6 text-center">
      <p class="font-black text-xs">{{ user.phone }}</p>
    </div>

    <div class="w-1/6 text-right">
      <span :class="[
        'px-3 py-1 rounded-full text-[9px] font-black uppercase border-2 border-black shadow-[2px_2px_0px_0px_rgba(0,0,0,1)]',
        user.role === 'Менеджер' ? 'bg-purple-400' : 'bg-green-300'
      ]">
        {{ user.role }}
      </span>
    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\router\index.js

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '@/views/HomeView.vue'
import CatalogView from '@/views/CatalogView.vue'
import AuthorizationView from '@/views/AuthorizationView.vue'
import RegistrationView from '@/views/RegistrationView.vue'
import ProfileView from '@/views/ProfileView.vue'
import ProductView from '@/views/ProductView.vue'
import CartView from '@/views/CartView.vue'
import AdminView from "@/views/AdminView.vue";
import FavoriteView from "@/views/FavoriteView.vue";

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      component: HomeView,
      name: 'Home',
    },
    {
      path: '/catalog',
      component: CatalogView,
      name: 'Catalog',
    },
    {
      path: '/authorization',
      component: AuthorizationView,
      name: 'Authorization',
    },
    {
```

```
      path: '/registration',
      component: RegistrationView,
      name: 'Registration',
    },
    {
      path: '/profile',
      component: ProfileView,
      name: 'Profile',
    },
    {
      path: '/product/:id',
      component: ProductView,
      name: 'Product',
    },
    {
      path: '/cart',
      component: CartView,
      name: 'Cart',
    },
    {
      path: '/favorite',
      component: FavoriteView,
      name: 'Favorite',
    },
    {
      path: '/admin',
      component: () => import('@/views/AdminView.vue'),
      redirect: { name: 'admin-products' },
      children: [
        {
          path: 'products',
          name: 'admin-products',
          component: () => import('@/components/AppAdminProductsWindow.vue'),
        },
        {
          path: 'orders',
          name: 'admin-orders',
          component: () => import('@/components/AppAdminOrdersWindow.vue'),
        },
        {
          path: 'users',
          name: 'admin-users',
          component: () => import('@/components/AppAdminUsersWindow.vue'),
        },
        {
          path: 'analytics',
          name: 'admin-analytics',
          component: () => import('@/components/AppAdminAnalytics.vue')
        }
      ]
    }
  ],
  scrollBehavior(to, from, savedPosition) {
    if (savedPosition) {
      return savedPosition
    }
    return { top: 0, behavior: 'smooth' }
  }
})

export default router
```

[OTHER] Project_ASICS\src\stores\ai.js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import axios from 'axios'
import { useToast } from 'vue-toastification'
import useProductStore from '@/stores/product.js'

export const useAiStore = defineStore('ai', () => {
  const toast = useToast()
  const isGenerating = ref(false)
  const isCoachLoading = ref(false)
```

```
  const productStore = useProductStore()

  const API_KEY = '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'
  const MODEL = 'nvidia/nemotron-3-super-120b-a12b:free'

  let activeCoachAbort = null

  // Очистка от markdown, звездочек и незаконченных обрывков слов
  const cleanAndFinishText = (text) => {
    if (!text) return ''
    let cleaned = text
      .replace(/\*\*(.*?)\*\*/g, '$1')
      .replace(/\*(.*?)\*/g, '$1')
      .replace(/#{1,6}\s?/g, '')
      .replace(/\(?\bID:?\s*\d+\)?/gi, '')
      .replace(/\s{2,}/g, ' ')
      .trim()

    // Если ответ оборвался посреди предложения (нет точки/вопроса/восклицания в конце),
    // отрезаем незаконченный огрызок слова до последней точки:
    const lastPunctuation = Math.max(cleaned.lastIndexOf('.'), cleaned.lastIndexOf('!'), cleaned.lastIndexOf('?'))
    if (lastPunctuation !== -1 && lastPunctuation < cleaned.length - 1) {
      cleaned = cleaned.slice(0, lastPunctuation + 1)
    }

    return cleaned
  }

  // 1. AI Running Coach (Консультант)
  const askCoach = async (userMessage) => {
    if (activeCoachAbort) activeCoachAbort.abort()
    activeCoachAbort = new AbortController()

    isCoachLoading.value = true

    try {
      if (!productStore.products || productStore.products.length === 0) {
        await productStore.productFun()
      }

      const products = productStore.products || []
      const catalogNames = products.map(p => p.name).join(', ')

      const combinedPrompt = `ИНСТРУКЦИЯ:
Ты — спортивный консультант официального магазина ASICS ('AI Running Coach').
Твоя задача — дать емкий, профессиональный и доброжелательный ответ на русском языке (2-3 полных предложения).

ПРАВИЛА:
1. Завершай свои мысли полными предложениями. ОБЯЗАТЕЛЬНО заканчивай последнее предложение точкой.
2. Не используй markdown-разметку, звездочки (**) и решетки (#).
3. В магазине продаются ТОЛЬКО кроссовки и спортивная одежда ASICS: ${catalogNames}.
4. Если клиент спрашивает про инвентарь или активность, не связанные со спортом (шест, стриптиз, алкоголь, диван) — вежливо и с юмором поясни, что в нашем каталоге такого инвентаря нет, так как ASICS производит экипировку для бега, тренировок и активного образа жизни.

ВОПРОС КЛИЕНТА:
${userMessage.trim().slice(0, 1000)}`

      const response = await axios.post(
        'https://openrouter.ai/api/v1/chat/completions',
        {
          model: MODEL,
          messages: [
            { role: 'user', content: combinedPrompt }
          ],
          temperature: 0.3,
          max_tokens: 800 // Увеличенный лимит: предложения гарантированно дописываются до точки
        },
        {
          headers: {
            'Authorization': `Bearer ${API_KEY}`,
            'Content-Type': 'application/json'
          },
          timeout: 30000,
          signal: activeCoachAbort.signal
        }
```

```
      )

      const rawAiResponse = response.data?.choices?.[0]?.message?.content?.trim()

      if (!rawAiResponse) {
        throw new Error('ИИ прислал пустой ответ.')
      }

      const finalReply = cleanAndFinishText(rawAiResponse)

      // ПОДБОР КАРТОЧЕК:
      // Карточки прикрепляются ТОЛЬКО если ИИ РЕАЛЬНО упомянул модель в своем ответе!
      const upperReply = finalReply.toUpperCase()
      const matchedProducts = products.filter(p => {
        const pNameUpper = p.name.toUpperCase()
        if (upperReply.includes(pNameUpper)) return true

        const keywords = pNameUpper.replace('GEL-', '').split(' ').filter(k => k.length >= 3)
        return keywords.some(k => upperReply.includes(k))
      })

      return {
        reply: finalReply,
        // Если подходящих товаров нет (например, вопрос про шест) — карточки НЕ показываются
        recommendations: matchedProducts.slice(0, 3).map(p => ({
          productId: p.id,
          productName: p.name,
          price: p.basePrice
        }))
      }
    } catch (e) {
      if (axios.isCancel(e)) return null

      const errorMsg = e.response?.data?.error?.message || e.message
      return {
        reply: `Ошибка: ${errorMsg}`,
        recommendations: []
      }
    } finally {
      isCoachLoading.value = false
      activeCoachAbort = null
    }
  }

  // 2. AI Copywriter для менеджера
  const generateDescription = async (payload) => {
    isGenerating.value = true
    try {
       const prompt = `Напиши продающее описание товара для магазина ASICS на русском языке (ровно 3 полных предложения). Обязательно
закончи точкой.
Модель: ${payload.name}, Категория: ${payload.category}, Бренд: ${payload.brand}. Особенности: ${payload.technologies || 'амортизация,
легкие материалы'}.
НЕ используй кавычки, звездочки и вступительные фразы.`

      const response = await axios.post(
        'https://openrouter.ai/api/v1/chat/completions',
        {
          model: MODEL,
          messages: [
            { role: 'user', content: prompt }
          ],
          temperature: 0.3,
          max_tokens: 800
        },
        {
          headers: {
            'Authorization': `Bearer ${API_KEY}`,
            'Content-Type': 'application/json'
          },
          timeout: 25000
        }
      )

      const desc = response.data?.choices?.[0]?.message?.content?.trim()
      if (desc) {
```

```
        toast.success('Описание сгенерировано!')
        return cleanAndFinishText(desc.replace(/^["']|["']$/g, ''))
      }
      throw new Error('Пустой ответ')
    } catch (e) {
      toast.error(`Ошибка генерации: ${e.response?.data?.error?.message || e.message}`)
      return null
    } finally {
      isGenerating.value = false
    }
  }

  return {
    isGenerating,
    isCoachLoading,
    askCoach,
    generateDescription
  }
})

export default useAiStore
```

[OTHER] Project_ASICS\src\stores\analytics.js

```
import {defineStore} from "pinia";
import { ref } from 'vue'
import axios from 'axios'
import { useToast } from 'vue-toastification'

const useAnalyticsStore = defineStore('analytics', () => {
    const mostLikedProducts = ref([])
    const mostLikedVariants = ref([])
    const userProductFavorites = ref([])
    const userVariantFavorites = ref([])
    const userFullProducts = ref([])
    const userFullVariants = ref([])
    const toast = useToast()

    const fetchProductAnalytics = async () => {
        try {
            const res = await axios.get('http://localhost:5000/api/Analytics/GetMostLikedProducts')
            mostLikedProducts.value = res.data
        } catch (err) {
            toast.error('Ошибка при загрузке аналитики товаров')
        }
    }

    const fetchVariantAnalytics = async () => {
        try {
            const res = await axios.get('http://localhost:5000/api/Analytics/GetMostLikedVariants')
            mostLikedVariants.value = res.data
        } catch (err) {
            toast.error('Ошибка при загрузке аналитики вариаций')
        }
    }
    const fetchUserFavorites = async (userId) => {
        try {
            const prodRes = await axios.get(`http://localhost:5000/api/Analytics/GetUserProductFavorites/${userId}`)
            userProductFavorites.value = prodRes.data

            const varRes = await axios.get(`http://localhost:5000/api/Analytics/GetUserVariantFavorites/${userId}`)
            userVariantFavorites.value = varRes.data

            const fullProdRes = await axios.get(`http://localhost:5000/api/Analytics/GetActualUserProductFavorites/${userId}`)
            userFullProducts.value = fullProdRes.data

            const fullVarRes = await axios.get(`http://localhost:5000/api/Analytics/GetActualUserVariantFavorites/${userId}`)
            userFullVariants.value = fullVarRes.data
        } catch (err) {
            console.error('Не удалось загрузить избранное пользователя')
        }
    }

    const toggleProductFavorite = async (userId, productId) => {
        try {
```

```
            const res = await axios.post('http://localhost:5000/api/Analytics/ToggleProductFavorite', {
                id: 0,
                userId: userId,
                productId: productId
            })

            if (res.data.isLiked) {
                userProductFavorites.value.push(productId)
                toast.success('Товар добавлен в избранное')
            } else {
                userProductFavorites.value = userProductFavorites.value.filter(id => id !== productId)
                userFullProducts.value = userFullProducts.value.filter(p => p.productId !== productId)
                toast.info('Товар удален из избранного')
            }
        } catch (err) {
            toast.error('Не удалось изменить статус избранного')
        }
    }

    const toggleVariantFavorite = async (userId, variantId) => {
        try {
            const res = await axios.post('http://localhost:5000/api/Analytics/ToggleVariantFavorite', {
                id: 0,
                userId: userId,
                productVariantId: variantId
            })

            if (res.data.isLiked) {
                userVariantFavorites.value.push(variantId)
                toast.success('Размер добавлен в избранное')
            } else {
                userVariantFavorites.value = userVariantFavorites.value.filter(id => id !== variantId)
                userFullVariants.value = userFullVariants.value.filter(v => v.productVariantId !== variantId)
                toast.info('Размер удален из избранного')
            }
        } catch (err) {
            toast.error('Не удалось изменить статус избранного для вариации')
        }
    }
    const clearFavorites = () => {
        userProductFavorites.value = []
        userVariantFavorites.value = []
    }

    return {
        mostLikedProducts,
        mostLikedVariants,
        userProductFavorites,
        userVariantFavorites,
        userFullProducts,
        userFullVariants,
        fetchProductAnalytics,
        fetchVariantAnalytics,
        fetchUserFavorites,
        toggleProductFavorite,
        toggleVariantFavorite,
        clearFavorites
    }
})

export default useAnalyticsStore
```

[OTHER] Project_ASICS\src\stores\brand.js

```
import {defineStore} from "pinia";
import {ref} from "vue";
import {useToast} from "vue-toastification";
import axios from "axios";

const useBrandStore = defineStore('brand',() => {
    const brand = ref([])
    const toast = useToast()
    const getAllBrand = async () => {
        await axios
            .get('http://localhost:5000/api/brand/GetAllBrands')
```

```
            .then((res) => {brand.value = res.data})
            .catch((err) => toast.error(err.response.data))
    }
    return {
        brand,
        getAllBrand,
    }
})
export default useBrandStore;
```

[OTHER] Project_ASICS\src\stores\cartItems.js

```
import { defineStore } from 'pinia'
import {computed, ref} from 'vue'
import { useToast } from 'vue-toastification'
import axios from 'axios'

const useCartItemStore = defineStore('cartItem', () => {
  const cartItem = ref(null)
  const currentCartItem = ref([])
  const toast = useToast()
  const addToCart = async (cartData) => {
    try {
      const res = await axios.post('http://localhost:5000/api/cartItem/AddCartItem', cartData)
      cartItem.value = res.data
      return true
    } catch (err) {
      toast.error(err.response?.data || 'Ошибка при добавлении')
      return false
    }
  }
  const minusToCart = async (cartData) => {
    try {
      const res = await axios.put('http://localhost:5000/api/cartItem/SubtractCartItem', cartData)
      cartItem.value = res.data
      return true
    } catch (err) {
      toast.error(err.response?.data || 'Ошибка при уменьшении количества')
      return false
    }
  }
  const cartTotalPrice = computed(() => {
    return currentCartItem.value.reduce((sum, item) => {
      const price = Number(item.price !== undefined ? item.price : item.Price) || 0
      const quantity = Number(item.quantity !== undefined ? item.quantity : item.Quantity) || 0

      return sum + (price * quantity)
    }, 0)
  })
  const getCartItems = async (id) => {
    await axios
      .get(`http://localhost:5000/api/cartItem/GetCartItem/${id}`)
      .then((res) => {
        currentCartItem.value = res.data
      })
      .catch((err) => toast.error(err.response.data))
  }
  const getCart = async (id) => {
    let cartSum
    await axios
      .get(`http://localhost:5000/api/cartItem/GetCartForSum/${id}`)
      .then((res) => {
        cartSum = res.data
      })
      .catch((err) => toast.error(err.response.data))
    return cartSum
  }
  const removeCart = async (id) => {
    await axios
      .delete(`http://localhost:5000/api/cartItem/DeleteCartItem/${id}`)
      .catch((err) => toast.error(err.response.data))
  }
  const clearAllCart = () =>{
    cartItem.value = null
    currentCartItem.value = []
```

```
  }
  return {
    cartItem,
    addToCart,
    currentCartItem,
    getCartItems,
    minusToCart,
    getCart,
    removeCart,
    clearAllCart,
    cartTotalPrice,
  }
})
export default useCartItemStore
```

[OTHER] Project_ASICS\src\stores\category.js

```
import {defineStore} from "pinia";
import {ref} from "vue";
import {useToast} from "vue-toastification";
import axios from "axios";

const useCategoryStore =  defineStore('category',() =>{
    const category = ref([])
    const toast = useToast()
    const getAllCategories = async () => {
        if (category.value && category.value.length > 0) return;
        await  axios
        .get(`http://localhost:5000/api/category/GetAllCategory`)
        .then((res) => (category.value = res.data))
        .catch((err) => toast.error(err.response.data))
    }
    return {
        category,
        getAllCategories,
    }
})
export default useCategoryStore;
```

[OTHER] Project_ASICS\src\stores\color.js

```
import {defineStore} from "pinia";
import {ref} from "vue";
import {useToast} from "vue-toastification";
import axios from "axios";

const useColorStore = defineStore('color',() => {
    const color = ref([])
    const toast = useToast()
    const getAllColors = async () => {
        await axios
        .get('http://localhost:5000/api/color/GetAllColor')
        .then((res) => {color.value = res.data})
        .catch((err) => toast.error(err.response.data))
    }
    return {
        color,
        getAllColors,
    }
})
export default useColorStore;
```

[OTHER] Project_ASICS\src\stores\order.js

```
import {defineStore} from "pinia";
import axios from "axios";
import {ref} from "vue";
import {useToast} from "vue-toastification";

const useOrderStore = defineStore('order', () => {
    const order = ref(null)
    const toast = useToast()
    const orderHistory = ref([])
```

```
    const addOrder = async (orderData) => {
        await axios
            .post('http://localhost:5000/api/Order/AddOrder', orderData)
            .then((res) => {
                order.value = res.data
            })
            .catch((err) => toast.error(err.response?.data))
    }
    const addOrderItem = async (userId, OrderId) => {
        await axios
            .postForm('http://localhost:5000/api/order/addOrderItem', {userId, OrderId})
            .catch((err) => toast.error(err.response.data))
    }
    const  getAllOrders = async () => {
        await axios
        .get(`http://localhost:5000/api/order/GetAllOrders`)
        .then((res) => {
            order.value = res.data
        })
        .catch((err) => toast.error(err.response?.data))
    }
    const editOrderStatus = async (orderData) =>{
        await axios
        .put('http://localhost:5000/api/order/EditStatusOrder', orderData)
        .then(async () => {
            await getAllOrders()
        })
        .catch((err) => toast.error(err.response.data))
    }
    const getUserOrderHistory = async (userId) => {
        await axios
            .get(`http://localhost:5000/api/Order/GetUserOrderHistory/${userId}`)
            .then((res) => {
                orderHistory.value = res.data
            })
            .catch((err) => toast.error(err.response?.data || 'Ошибка загрузки истории заказов'))
    }
    return {
        order,
        addOrder,
        orderHistory,
        addOrderItem,
        editOrderStatus,
        getAllOrders,
        getUserOrderHistory
    }
})

export default useOrderStore
```

[OTHER] Project_ASICS\src\stores\product.js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import axios from 'axios'
import { useToast } from 'vue-toastification'

const toast = useToast()

const useProductStore = defineStore('product', () => {
  const products = ref([])
  const currentProduct = ref(null)

  async function productFun() {
    await axios
      .get('http://localhost:5000/api/product/GetProductAll')
      .then((res) => (products.value = res.data))
      .catch((err) => toast.error(err.response.data))
  }
  const getProduct = async (id) => {
    let getProductForSum
    await axios
      .get(`http://localhost:5000/api/product/GetProduct/${id}`)
      .then((res) => {
        currentProduct.value = res.data
```

```
      getProductForSum = res.data
    })
    .catch((err) => toast.error(err.response.data))
  return getProductForSum
}
const editProduct = async (productData) => {
  await  axios
  .put('http://localhost:5000/api/product/EditProduct', productData)
  .then(async () => {
    await productFun()
  })
  .catch((err) => toast.error(err.response.data))
}
const editProductVariant = async (productVariantData) => {
  await  axios
  .put('http://localhost:5000/api/product/EditProductVariant', productVariantData)
  .then(async () => {
    await productFun()
  })
  .catch((err) => toast.error(err.response.data))
}
const deleteProduct = async (id) => {
  await axios
  .delete(`http://localhost:5000/api/product/DeleteProduct/${id}`)
  .catch((err) => toast.error(err.response.data))
}
const addProductVariant = async (variantData) => {
  return await axios.post('http://localhost:5000/api/productvariant/AddProductVariant', variantData)
      .catch((err) => toast.error(err.response.data))
}
const addProduct = async (productData) => {
  return await axios.post('http://localhost:5000/api/product/AddProduct', productData)
      .catch((err) => toast.error(err.response.data))
}

const deleteProductVariant = async (id) => {
  await axios.delete(`http://localhost:5000/api/productvariant/DeleteProductVariant/${id}`)
      .catch((err) => toast.error(err.response.data))
}

const addImageProduct = async (file, productData) => {
  const formData = new FormData()
  formData.append('file', file)
  formData.append('productId', productData.id)
  formData.append('category', productData.category)
  formData.append('name', productData.name)

  return await axios.post('http://localhost:5000/api/image/UpLoadProductImage', formData, {
    headers: { 'Content-Type': 'multipart/form-data' }
  }).catch((err) => {
    const errorMsg = err.response?.data?.title || err.response?.data || 'Ошибка загрузки фото товара'
    toast.error(typeof errorMsg === 'string' ? errorMsg : 'Ошибка валидации фото')
    throw err
  })
}

const addImageVariant = async (file, variantData) => {
  const formData = new FormData()
  formData.append('file', file)
  formData.append('variantId', variantData.id)
  formData.append('color', variantData.color)
  formData.append('size', variantData.size)

  return await axios.post('http://localhost:5000/api/image/UpLoadProductVariantImage', formData, {
    headers: { 'Content-Type': 'multipart/form-data' }
  }).catch((err) => {
    const errorMsg = err.response?.data?.title || err.response?.data || 'Ошибка загрузки фото вариации'
    toast.error(typeof errorMsg === 'string' ? errorMsg : 'Ошибка валидации фото')
    throw err
  })
}

const getProductImage = async (productId) => {
  let productImage = null
  await axios
```

```
        .get(`http://localhost:5000/api/image/GetProductImage/${productId}`)
        .then((res) => {
          productImage = res.data
        })
        .catch((err) => {
          console.warn(`У товара с ID ${productId} отсутствует изображение в БД.`);
        })

    return productImage
  }

  const getVariantImage = async (variantId) => {
    let variantImage = null
    await axios
        .get(`http://localhost:5000/api/image/GetVariantImage/${variantId}`)
        .then((res) => {
          variantImage = res.data
        })
        .catch((err) => {
          console.warn(`У вариации с ID ${variantId} отсутствует изображение в БД.`);
        })

    return variantImage
  }

  const getProductImageByVariant = async (variantId) => {
    let productImage = null
    await axios
        .get(`http://localhost:5000/api/image/GetProductImageByVariant/${variantId}`)
        .then((res) => {
          productImage = res.data
        })
        .catch((err) => {
          console.warn(`Не удалось загрузить картинку товара для вариации ID ${variantId}`);
        })

    return productImage
  }
  return {
    products,
    productFun,
    currentProduct,
    getProduct,
    editProduct,
    editProductVariant,
    deleteProduct,
    addProductVariant,
    deleteProductVariant,
    addProduct,
    addImageVariant,
    addImageProduct,
    getProductImage,
    getVariantImage,
    getProductImageByVariant,
  }
})

export default useProductStore
```

[OTHER] Project_ASICS\src\stores\user.js

```
import { defineStore } from 'pinia'
import { ref } from 'vue'
import axios from 'axios'
import { useToast } from 'vue-toastification'
import useAnalyticsStore from "@/stores/analytics.js"
import router from "@/router/index.js";

const useUserStore = defineStore('user', () => {
  const user = ref(null)
  const allUsers = ref([])
  const toast = useToast()
  const analyticsStore = useAnalyticsStore()
  const authUser = async (login, password) => {
```

```
    await axios
      .postForm('http://localhost:5000/api/user/Authorization', {
        login: login,
        password: password,
      })
        .then((res) => {
          user.value = res.data

          localStorage.setItem('user', JSON.stringify({
            login: login,
            password: password
          }))
        })
      .catch((err) => toast.error(err.response.data))
  }
  const registerUser = async (userData) => {
    await axios
      .post('http://localhost:5000/api/user/Registration', userData)
        .then((res) => {
          user.value = res.data

          localStorage.setItem('user', JSON.stringify({
            login: userData.login,
            password: userData.password
          }))
        })
      .catch((err) => toast.error(err.response.data))
  }
  const editUser = async (userData) => {
    try {
      const res = await axios.put('http://localhost:5000/api/user/EditUser', userData)
      user.value = res.data
      toast.success('Успешно обновлено')
      return true
    } catch (err) {
      toast.error(err.response?.data || 'Ошибка при обновлении данных')
      return false
    }
  }
  const getAllUsers = async () => {
    await axios
        .get('http://localhost:5000/api/user/GetAllUsers')
        .then((res) => {allUsers.value = res.data})
        .catch((err) => toast.error(err.response.data))
  }
  const logout = () => {
    user.value = null
    localStorage.removeItem('user')
    analyticsStore.clearFavorites()
    toast.success('Вы успешно вышли из аккаунта')
    router.push({ name: 'Home' })
  }
  return {
    user,
    authUser,
    registerUser,
    editUser,
    getAllUsers,
    allUsers,
    logout,
  }
})
export default useUserStore
```

[OTHER] Project_ASICS\src\views\AdminView.vue

```
<script setup>
import useProductStore from "@/stores/product.js";
import {onBeforeMount, ref} from "vue";

const productStore = useProductStore()
const isEditVariantOpen = ref(false)
const isAddVariantOpen = ref(false)
const isAddProductOpen = ref(false)
```

```
onBeforeMount(async () => {
  await productStore.productFun()
})
</script>

<template>
  <div class="flex min-h-screen bg-gray-50">

    <aside class="w-64 bg-black text-white flex flex-col pt-8 px-6 ">
      <div class="mb-12">
        <h1 class="text-xl font-black tracking-widest">Панель администратора</h1>
      </div>

      <nav class="flex flex-col gap-6 font-bold text-sm tracking-wider">
        <router-link to="/" class="flex items-center gap-3 text-gray-400 hover:text-white transition-colors">
          <span>Главная</span>
        </router-link>

                <router-link  :to="{  name:  'admin-products'  }"  active-class="text-white"  class="text-gray-400  hover:text-white
transition-colors">
          <span>Товары</span>
        </router-link>

                 <router-link  :to="{  name:  'admin-orders'  }"  active-class="text-white"  class="text-gray-400  hover:text-white
transition-colors">
          <span>Заказы</span>
        </router-link>

        <router-link :to="{ name: 'admin-users' }" active-class="text-white" class="text-gray-400 hover:text-white transition-colors">
          <span>Пользователи</span>
        </router-link>

                <router-link  :to="{  name:  'admin-analytics'  }"  active-class="text-white"  class="text-gray-400  hover:text-white
transition-colors">
          <span>Аналитика</span>
        </router-link>
      </nav>
    </aside>

    <main class="flex-1 p-10 overflow-x-hidden">
      <router-view />
    </main>
  </div>
</template>

<style scoped>

</style>
```

[OTHER] Project_ASICS\src\views\AuthorizationView.vue

```
<script setup>
import useUserStore from '@/stores/user.js'
import { useTemplateRef } from 'vue'
import { useRouter } from 'vue-router'
import { useToast } from 'vue-toastification'

const login = useTemplateRef('login')
const password = useTemplateRef('password')

const toast = useToast()
const router = useRouter()
const userStore = useUserStore()
const auth = async () => {
  if (
      !login.value.value ||
      !password.value.value
  ) {
    toast.error('Заполните все поля')
    return
  }
  await userStore.authUser(login.value.value, password.value.value)
  if (!userStore.user) {
    toast.error('Неверный логин или пароль!')
    return
```

```
  }
  toast.success('Добро пожаловать!')
  if (userStore.user) {
    await router.push('/')
  }
}
</script>

<template>
  <div class="min-h-screen w-full flex items-center justify-center bg-gray-100">
    <div class="flex items-start justify-center w-200 p-5 border border-black flex-col">
      <h1 class="font-black text-3xl mt-3">Авторизация</h1>
      <span class="font-black mt-3">Почта</span>
      <input class="w-full h-10 border border-black" type="text" ref="login" />
      <span class="font-black">Пароль</span>
      <input class="w-full h-10 border border-black" type="password" ref="password" />
      <button class="w-full h-15 bg-black text-white text-2xl mt-5 w" @click="auth">Войти</button>
      <div class="flex justify-center items-center self-center flex-row mt-2">
        <h3>Нету аккаунта?</h3>
        <router-link class="text-black border-b-2 hover:text-blue-500" to="/registration"
          >Создать аккаунт</router-link
        >
      </div>
    </div>
  </div>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\views\CartView.vue

```
<script setup>
import AppCartItem from '@/components/AppCartItem.vue'
import { computed, onBeforeMount, ref, useTemplateRef } from 'vue'
import useCartItemStore from '@/stores/cartItems.js'
import useUserStore from '@/stores/user.js'
import { useRoute, useRouter } from 'vue-router'
import useProductStore from '@/stores/product.js'
import useOrderStore from '@/stores/order.js'
import { useToast } from 'vue-toastification'
import AppCartModal from "@/components/AppCartModal.vue";

const userStore = useUserStore()
const router = useRouter()
const route = useRoute()
const cartItemStore = useCartItemStore()
const productStore = useProductStore()
const orderStore = useOrderStore()
const toast = useToast()

const isModalOpen = ref(false)


onBeforeMount(async () => {
  if (userStore.user && userStore.user.id) {
    await cartItemStore.getCartItems(userStore.user.id)
  } else {
    console.error('Юзер не авторизован')
  }
})

const cartItemRef = useTemplateRef('cartItemRef')
const totalPrice = computed(() => Math.round(cartItemStore.cartTotalPrice * 100) / 100)
</script>

<template>
  <div class="container mx-auto px-6 py-10">
    <h1 class="text-3xl font-black uppercase mb-8 italic">Корзина</h1>

    <div class="flex flex-col lg:flex-row gap-10">
      <div class="grow lg:w-2/3">
        <div v-if="cartItemStore.currentCartItem.length > 0">
          <app-cart-item
            v-for="item in cartItemStore.currentCartItem"
```

```
            :key="item.id"
            :cart-item="item" />
<!--            ref="cartItemRef"-->

        </div>
        <div v-else class="text-center py-20 border-2 border-dashed border-gray-200 rounded-xl">
          <p class="text-gray-400 text-xl">Ваша корзина пока пуста</p>
        </div>
      </div>

      <div class="lg:w-1/3" v-if="cartItemStore.currentCartItem.length > 0">
        <div class="border border-black p-6 rounded-xl bg-white sticky top-10">
          <h2 class="text-2xl font-bold uppercase mb-6">Заказ</h2>

          <div class="flex flex-col gap-4 border-b border-gray-200 pb-6 mb-6 text-lg">
            <div class="flex justify-between">
              <span class="text-gray-600">Примерка: </span>
              <span class="font-bold text-green-600">бесплатно</span>
            </div>
            <div class="flex justify-between">
              <span class="text-gray-600">Доставка: </span>
              <span class="font-bold text-green-600">бесплатно</span>
            </div>
          </div>

          <div class="flex justify-between items-center mb-8">
            <span class="text-xl font-bold uppercase">Итог</span>
            <span class="text-2xl font-black italic">{{ totalPrice }} ₽</span>
          </div>

          <button
            class="w-full bg-black text-white py-4 rounded-lg font-bold uppercase hover:bg-gray-800 transition-all mb-4"
            @click="isModalOpen = true"
          >
            Оформить заказ
          </button>

          <router-link
            to="/catalog"
            class="block text-center text-sm font-bold uppercase underline hover:no-underline"
          >
            Продолжить покупки
          </router-link>
        </div>
      </div>
    </div>
    <Teleport to="body">
      <app-cart-modal :totalPrice="totalPrice" :is-modal-open="isModalOpen" @close-modal="isModalOpen = false"/>
    </Teleport>
  </div>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\views\CatalogView.vue

```
<script setup>
import useProductStore from '@/stores/product.js'
import { onBeforeMount, ref, computed } from 'vue'
import router from '@/router/index.js'
import AppProductCart from '@/components/AppProductCard.vue'

const productStore = useProductStore()

const selectedCategory = ref('Все')
const currentSort = ref('noSort')

onBeforeMount(async () => {
  await productStore.productFun()
})

const filteredAndSortedProducts = computed(() => {
  if (!productStore.products) return []
```

```
  let result = [...productStore.products]

  if (selectedCategory.value !== 'Все') {
    result = result.filter(product => product.category === selectedCategory.value)
  }

  if (currentSort.value === 'sortAsc') {
    result.sort((a, b) => a.basePrice - b.basePrice)
  } else if (currentSort.value === 'sortDesc') {
    result.sort((a, b) => b.basePrice - a.basePrice)
  }

  return result
})
</script>

<template>
  <div class="p-10 flex justify-start  min-h-screen gap-6">
    <div class="flex justify-start flex-col border border-black mt-8 h-fit bg-white w-56 p-5 shadow-sm">

      <div class="flex justify-start flex-col items-start text-xl">
        <h1 class="font-black text-2xl mb-3 text-[rgb(0,30,98)] uppercase tracking-tight">Категории</h1>

        <button
            @click="selectedCategory = 'Все'"
            class="mt-1 text-left w-full font-bold transition-colors cursor-pointer"
            :class="selectedCategory === 'Все' ? 'text-blue-600 underline' : 'text-gray-700 hover:text-blue-500'"
        >
          Все
        </button>
        <button
            @click="selectedCategory = 'Кроссовки'"
            class="mt-1 text-left w-full font-bold transition-colors cursor-pointer"
            :class="selectedCategory === 'Кроссовки' ? 'text-blue-600 underline' : 'text-gray-700 hover:text-blue-500'"
        >
          Кроссовки
        </button>
        <button
            @click="selectedCategory = 'Верх'"
            class="mt-1 text-left w-full font-bold transition-colors cursor-pointer"
            :class="selectedCategory === 'Верх' ? 'text-blue-600 underline' : 'text-gray-700 hover:text-blue-500'"
        >
          Верх
        </button>
        <button
            @click="selectedCategory = 'Носки'"
            class="mt-1 text-left w-full font-bold transition-colors cursor-pointer"
            :class="selectedCategory === 'Носки' ? 'text-blue-600 underline' : 'text-gray-700 hover:text-blue-500'"
        >
          Носки
        </button>
      </div>

      <div class="flex justify-start flex-col mt-8 border-t border-gray-300 pt-4">
        <h1 class="font-black text-2xl text-[rgb(0,30,98)] uppercase tracking-tight">Сортировка</h1>
        <select
            v-model="currentSort"
            name="selectSort"
            class="mt-3 w-full border border-black p-2 bg-gray-50 outline-none font-bold"
        >
          <option value="noSort">Без сортировки</option>
          <option value="sortAsc">По возрастанию</option>
          <option value="sortDesc">По убыванию</option>
        </select>
      </div>
    </div>

    <div class="flex items-start justify-start flex-wrap gap-4 mt-8 flex-1">
      <template v-if="filteredAndSortedProducts.length > 0">
        <app-product-cart
            v-for="product in filteredAndSortedProducts"
            :key="product.id"
            :product="product"
            @click="
            router.push({
```

```
              name: 'Product',
              params: {
                id: product.id,
              },
            })
          "
        ></app-product-cart>
      </template>

      <div v-else class="text-gray-500 italic text-xl mt-10 ml-6">
        Товары в категории "{{ selectedCategory }}" не найдены.
      </div>
    </div>
  </div>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\views\FavoriteView.vue

```
<script setup>
import {onMounted, watch} from 'vue'
import useAnalyticsStore from '@/stores/analytics.js'
import useUserStore from '@/stores/user.js'
import AppFavoritesProduct from '@/components/AppFavoritesProduct.vue'
import AppFavoriteVariant from '@/components/AppFavoritesVariant.vue'

const analyticsStore = useAnalyticsStore()
const userStore = useUserStore()

const loadData = async (userId) => {
  await analyticsStore.fetchUserFavorites(userId)
}

onMounted(async () => {
  if (userStore.user) {
    await loadData(userStore.user.id)
  }
watch(
    () => userStore.user,
    async (newUser) => {
      if (newUser) {
        await loadData(newUser.id)
      }
    }
)
})
</script>

<template>
  <div class="p-8 bg-gray-50 min-h-screen text-black">
    <div class="max-w-7xl mx-auto">
      <h1 class="text-5xl font-black text-[rgb(0,30,98)] uppercase mb-2 italic tracking-tighter">
        Мое Избранное
      </h1>
      <p class="font-bold text-gray-500 uppercase text-xs mb-8">
        Здесь хранятся товары и размеры, которые вы отложили
      </p>

      <div class="mb-12">
        <h2 class="text-2xl font-black uppercase mb-4 flex items-center gap-2">
          <span>Отложенные модели</span>
        </h2>

        <div
            v-if="analyticsStore.userFullProducts && analyticsStore.userFullProducts.length > 0"
            class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-6"
        >
          <AppFavoritesProduct
              v-for="item in analyticsStore.userFullProducts"
              :key="item.productId"
              :item="item"
          />
        </div>
                        <div   v-else   class="border-4   border-black   border-dashed   p-8   text-center   rounded-xl   bg-white
```

```
shadow-[4px_4px_0px_0px_rgba(0,0,0,1)]">
          <p class="text-gray-400 italic font-black uppercase text-sm">Вы пока не добавляли модели целиком</p>
        </div>
      </div>

      <div>
        <h2 class="text-2xl font-black uppercase mb-4 flex items-center gap-2">
          <span>Конкретные размеры и цвета</span>
        </h2>

        <div
            v-if="analyticsStore.userFullVariants && analyticsStore.userFullVariants.length > 0"
            class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4 gap-6"
        >
          <AppFavoriteVariant
              v-for="item in analyticsStore.userFullVariants"
              :key="item.productVariantId"
              :item="item"
          />
        </div>
                        <div   v-else   class="border-4   border-black   border-dashed   p-8   text-center   rounded-xl   bg-white
shadow-[4px_4px_0px_0px_rgba(0,0,0,1)]">
          <p class="text-gray-400 italic font-black uppercase text-sm">Вы пока не откладывали конкретные размеры</p>
        </div>
      </div>

    </div>
  </div>
</template>
```

[OTHER] Project_ASICS\src\views\HomeView.vue

```
<script setup>
import { useRouter } from 'vue-router'
const router = useRouter()
</script>

<template>
  <div
    class="main-banner h-screen bg-cover bg-center bg-no-repeat flex items-center justify-start px-10 pl-20"
  >
    <div class="flex space-x-6 flex-col">
      <div class="max-w-40 flex flex-wrap font-bold text-5xl">
        <h1 class="text-blue-950 flex items-center">Производительность и стиль</h1>
      </div>
      <div class="max-w-120 flex flex-wrap text-xl mt-5">
        <span class="text-blue-950"
          >Высококачественное спортивное снаряжение, разработанное для спортсменов, стремящихся к
          совершенству.</span
        >
      </div>
      <div class="flex justify-start h-12 w-50 mt-5">
        <button
          class="bg-[rgb(0,30,98)] rounded-2xl p-2 hover:cursor-pointer text-white"
          @click="router.push({ name: 'Catalog' })"
        >
          Купить сейчас
        </button>
      </div>
    </div>
  </div>
  <div class="flex items-center space-x-6 justify-center flex-col gap-5 mt-5 text-black">
    <h2 class="">Рекомендуемые категории</h2>
    <div class="flex items-center justify-between gap-5">
      <div class="flex items-center flex-col">
        <img
          src="../assets/volleyball.png"
          alt="volleyball"
          class="h-100 w-100 bg-cover bg-center bg-no-repeat flex items-center justify-center"
        />
        <router-link to="/catalog">
          <div class="flex items-center flex-col text-10 font-bold">
            <h2>Волейбольные</h2>
            <h2>Кроссовки</h2>
          </div>
```

```
        </router-link>
      </div>
      <div class="flex items-center flex-col">
        <img
          src="../assets/trousers.png"
          alt="volleyball"
          class="h-100 w-100 bg-cover bg-center bg-no-repeat flex items-center justify-center"
        />
        <router-link to="/catalog">
          <div class="flex items-center flex-col text-10 font-bold">
            <h2>Спортивный</h2>
            <h2>Стиль</h2>
          </div>
        </router-link>
      </div>
      <div class="flex items-center flex-col">
        <img
          src="../assets/tennis.png"
          alt="volleyball"
          class="h-100 w-100 bg-cover bg-center bg-no-repeat flex items-center justify-center"
        />
        <router-link to="/catalog">
          <div class="flex items-center flex-col text-10 font-bold">
            <h2>Теннисная</h2>
            <h2>Форма</h2>
          </div>
        </router-link>
      </div>
    </div>
  </div>
</template>

<style scoped>
.main-banner {
  background-image: url('../assets/ad_banner.png');
}
</style>
```

[OTHER] Project_ASICS\src\views\ProductView.vue

```
<script setup>
import { useRoute, useRouter } from 'vue-router'
import useProductStore from '@/stores/product.js'

import {computed, onBeforeMount, ref, watch} from 'vue'
import useCartItemStore from '@/stores/cartItems.js'
import { useToast } from 'vue-toastification'
import useUserStore from '@/stores/user.js'
import shoesAlt from "@/assets/shoes_alt.png"
import useAnalyticsStore from "@/stores/analytics.js";

const route = useRoute()
const router = useRouter()
const userStore = useUserStore()
const productStore = useProductStore()
const cartItemStore = useCartItemStore()
const analyticsStore = useAnalyticsStore()
const toast = useToast()

const selectedColorName = ref(null)
const selectedSize = ref(null)

const imageSrc = ref('')

const handleImageError = () => {
  imageSrc.value = shoesAlt
}

const addToCart = async () => {
  if (!userStore.user) {
    toast.info('Для добавление в корзину нужно войти в аккаунт')
    await router.push('/authorization')
    return
  }
```

```
  if (!selectedVariant.value) {
    toast.error('Пожалуйста, выберите цвет и размер')
    return
  }

  if (!selectedColorName.value) {
    toast.error('Пожалуйста выберите цвет и размер')
    return
  }
  const cart = {
    id: 0,
    userId: userStore.user.id,
    productVariantId: selectedVariant.value.id,
    quantity: 1,
    productName: '',
    color: '',
    size: 0,
  }
  const isSuccess = await cartItemStore.addToCart(cart)

  if (isSuccess) {
    toast.success('Товар успешно добавлен в корзину')
  }
}

onBeforeMount(async () => {
  await productStore.getProduct(Number(route.params.id))

  const variants = productStore.currentProduct?.variants || []
  if (variants.length > 0) {
    const queryColor = route.query.color
    const querySize = route.query.size

    if (queryColor && querySize) {
      selectedColorName.value = String(queryColor)
      selectedSize.value = Number(querySize)
    } else {
      selectedColorName.value = variants[0].color
      selectedSize.value = variants[0].size
    }

    await updateVariantImage()
  }
  if (userStore.user) {
    await analyticsStore.fetchUserFavorites(userStore.user.id)
  }
})

const availableColors = computed(() => {
  const variants = productStore.currentProduct?.variants || []
  const uniqueColorNames = [...new Set(variants.map((v) => v.color))]

  return uniqueColorNames.map((name) => ({
    id: name,
    name: name,
  }))
})

const availableSizes = computed(() => {
  const variants = productStore.currentProduct?.variants || []

  const filteredByColor = variants.filter((v) => v.color === selectedColorName.value)

  const uniqueSizes = [...new Set(filteredByColor.map((v) => v.size))]

  return uniqueSizes.sort((a, b) => a - b)
})

const selectedVariant = computed(() => {
  const variants = productStore.currentProduct?.variants || []
  return variants.find(
    (v) => v.color === selectedColorName.value && String(v.size) === String(selectedSize.value),
  )
})
```

```
const currentColorName = computed(() => selectedColorName.value || '')

const isVariantLiked = computed(() => {
  if (!selectedVariant.value) return false
  return analyticsStore.userVariantFavorites.includes(selectedVariant.value.id)
})

const handleVariantLikeClick = async () => {
  if (!userStore.user) {
    toast.info('Для добавления в избранное войдите в аккаунт')
    return
  }
  if (!selectedVariant.value) {
    toast.error('Сначала выберите цвет и размер товара')
    return
  }
  await analyticsStore.toggleVariantFavorite(userStore.user.id, selectedVariant.value.id)
}

watch(selectedColorName, (newColor) => {
  if (newColor) {
    const variants = productStore.currentProduct?.variants || []
    const filteredByColor = variants.filter((v) => v.color === newColor)

    if (filteredByColor.length > 0) {
      const sortedSizes = filteredByColor.map((v) => v.size).sort((a, b) => a - b)
      selectedSize.value = sortedSizes[0]
    }
  }
})

const updateVariantImage = async () => {
  if (selectedVariant.value) {
    const variantImage = await productStore.getVariantImage(selectedVariant.value.id)

    if (variantImage && variantImage.imagePath) {
      imageSrc.value = `http://localhost:5000/imagesVariant/${variantImage.imagePath}`
    } else {
      imageSrc.value = shoesAlt
    }
  } else {
    imageSrc.value = shoesAlt
  }
}
watch(selectedVariant, async () => {
  await updateVariantImage()
})
</script>

<template>
  <div
    v-if="productStore.currentProduct"
    class="flex flex-row min-h-screen p-10 md:p-20 w-full bg-white gap-16"
  >
    <div class="flex flex-col gap-6">
      <div class="border-4 border-black p-2 shadow-[12px_12px_0px_0px_rgba(0,0,0,1)] bg-white">
        <img
            :src="imageSrc"
            alt="Main Product"
            class="h-[500px] w-[500px] object-contain mix-blend-multiply"
            @error="handleImageError"
        />
      </div>
    </div>

    <div class="flex flex-col flex-1 max-w-2xl gap-8">
      <div>
        <h4 class="uppercase font-black text-blue-800 tracking-tighter text-lg">
          Asics / Training
        </h4>
        <div class="flex items-center justify-between gap-4 mt-2">
          <h1 class="font-black text-6xl uppercase leading-none mt-2">
            {{ productStore.currentProduct.name }}
          </h1>
          <button
```

```
            @click="handleVariantLikeClick"
                 class="p-3 border-4 border-black rounded-xl bg-white shadow-[4px_4px_0px_0px_rgba(0,0,0,1)] hover:translate-y-0.5
hover:shadow-[2px_2px_0px_0px_rgba(0,0,0,1)] transition-all flex items-center justify-center shrink-0"
            title="Добавить этот размер в избранное"
          >
            <svg
                xmlns="http://www.w3.org/2000/svg"
                :class="['h-8 w-8 transition-colors', isVariantLiked ? 'text-red-500 fill-current' : 'text-gray-400']"
                fill="none"
                viewBox="0 0 24 24"
                stroke="currentColor"
            >
              <path
                  stroke-linecap="round"
                  stroke-linejoin="round"
                  stroke-width="2.5"
                   d="M4.318 6.318a4.5 4.5 0 000 6.364L12 20.364l7.682-7.682a4.5 4.5 0 00-6.364-6.364L12 7.636l-1.318-1.318a4.5 4.5 0
00-6.364 0z"
              />
            </svg>
          </button>
        </div>
        <p class="text-4xl font-bold mt-4 text-[rgb(0,30,98)]">
          ₽{{ productStore.currentProduct.basePrice }}
        </p>
      </div>

      <hr class="border-t-4 border-black" />

      <div class="flex flex-col gap-2">
        <span class="font-black uppercase text-xl">Описание</span>
        <p class="text-lg font-medium leading-tight">
          {{
            productStore.currentProduct.description ||
            'Кроссовки обеспечивают отличную поддержку и амортизацию для интенсивных тренировок в зале и на улице.'
          }}
        </p>
      </div>

      <div class="flex flex-col gap-4">
        <span class="font-black uppercase text-sm italic"
          >Выбранный цвет: {{ currentColorName }}</span
        >
        <div class="flex gap-3">
          <button
            v-for="color in availableColors"
            :key="color.id"
            @click="selectedColorName = color.id"
            :class="[
              'px-6 py-2 border-2 border-black font-black uppercase transition-all',
              selectedColorName === color.id
                ? 'bg-black text-white translate-y-1 shadow-none'
                : 'bg-white shadow-[4px_4px_0px_0px_rgba(0,0,0,1)] hover:bg-gray-50',
            ]"
          >
            {{ color.name }}
          </button>
        </div>
      </div>

      <div class="flex flex-col gap-4">
        <span class="font-black uppercase text-sm italic">Размер: {{ selectedSize }}</span>
        <div class="flex gap-2">
          <button
            v-for="size in availableSizes"
            :key="size"
            @click="selectedSize = size"
            :class="[
              'w-14 h-14 border-2 border-black font-black transition-all flex items-center justify-center',
              String(selectedSize) === String(size)
                ? 'bg-[rgb(0,30,98)] text-white'
                : 'bg-white hover:border-blue-800',
            ]"
          >
            {{ size }}
```

```
          </button>
        </div>
      </div>

      <div class="mt-6">
        <button class="group relative w-full h-20 bg-black border-2 border-black">
          <div
             class="absolute inset-0 bg-[rgb(0,30,98)] translate-x-2 translate-y-2 group-hover:translate-x-0 group-hover:translate-y-0
transition-transform"
          ></div>
          <div
             class="absolute inset-0 bg-black flex items-center justify-center gap-4 text-white font-black text-2xl uppercase border-2
border-black"
            @click="addToCart"
          >
            <svg
              xmlns="http://www.w3.org/2000/svg"
              class="h-8 w-8"
              fill="none"
              viewBox="0 0 24 24"
              stroke="currentColor"
            >
              <path
                stroke-linecap="round"
                stroke-linejoin="round"
                stroke-width="3"
                d="M12 4v16m8-8H4"
              />
            </svg>
            Добавить в корзину
          </div>
        </button>
      </div>

      <div class="grid grid-cols-2 gap-4 mt-4 font-bold text-sm uppercase">
        <div class="flex items-center gap-2"> Оригинальный товар</div>
        <div class="flex items-center gap-2"> Быстрая доставка</div>
        <div class="flex items-center gap-2"> Гарантия качества</div>
        <div class="flex items-center gap-2"> Примерка перед покупкой</div>
      </div>
    </div>
  </div>

  <div v-else class="min-h-screen flex items-center justify-center bg-gray-100">
    <div class="text-center">
      <h2 class="text-4xl font-black uppercase animate-pulse">Загружаем данные...</h2>
      <p class="mt-2 font-bold text-gray-500 italic">Связываемся с сервером Asics</p>
    </div>
  </div>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\views\ProfileView.vue

```
<script setup>
import { useRouter } from 'vue-router'
import useUserStore from '@/stores/user.js'
import { ref, useTemplateRef, onMounted } from 'vue'
import useOrderStore from '@/stores/order.js'
import { useToast } from 'vue-toastification'
import AppProfileOrder from '@/components/AppProfileOrder.vue'

const router = useRouter()
const userStore = useUserStore()
const orderStore = useOrderStore()
const isModalOpen = ref(false)

const name = useTemplateRef('name')
const surname = useTemplateRef('surname')
const patronymic = useTemplateRef('patronymic')
const phone = useTemplateRef('phone')
const email = useTemplateRef('login')
```

```
const toast = useToast()

onMounted(async () => {
  if (userStore.user && userStore.user.id) {
    await orderStore.getUserOrderHistory(userStore.user.id)
  }
})

const regexValidation = (value, type) => {
  switch (type) {
    case 'password':
      return String(value).match(/(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}/)
    case 'email':
      return String(value).match(/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]/)
    case 'phone':
      return String(value).match(/^(?:\+7|8|7)?(\d{10})$/)
  }
}

const ed = async () => {
  const trimmedName = name.value.value.trim()
  const trimmedSurname = surname.value.value.trim()
  const trimmedPatronymic = patronymic.value.value ? patronymic.value.value.trim() : ''
  const trimmedEmail = email.value.value.trim()
  const trimmedPhone = phone.value.value.trim()

  if (!trimmedName || !trimmedSurname || !trimmedPhone || !trimmedEmail) {
    toast.error('Заполните все поля')
    return
  }
  if (!regexValidation(trimmedEmail, 'email')) {
    toast.error('Неверный формат почты')
    return
  }
  if (!regexValidation(trimmedPhone, 'phone')) {
    toast.error('Неверный формат номера телефона')
    return
  }

  const updatedData = {
    ...userStore.user,
    name: trimmedName,
    surname: trimmedSurname,
    patronymic: trimmedPatronymic,
    email: trimmedEmail,
    phone: trimmedPhone
  }
  const success = await userStore.editUser(updatedData)
  if (success) {
    isModalOpen.value = false
  }
}
const logout = () => {
  userStore.logout()
}
</script>

<template>
  <div class="p-10 flex items-start justify-start bg-gray-100 gap-10 min-h-screen">
    <div class="relative mt-10">
      <label class="absolute top-0 left-5 -translate-y-1/2 bg-gray-100 px-2">Мой профиль</label>
      <div
        class="flex items-start justify-center w-130 p-5 gap-1 border border-black flex-col"
        id="mainDiv"
      >
        <div class="flex items-center justify-between w-full mb-4">
          <h1 class="font-black text-3xl mt-3 text-[rgb(0,30,98)]">Персональные данные</h1>
          <button
            class="font-bold text-3xl hover:text-blue-500 cursor-pointer"
            @click="isModalOpen = true"
          >

          </button>
        </div>
        <h2 class="font-black mt-3 text-[rgb(0,30,98)]">Имя</h2>
```

```
        <span
          class="w-full h-10 flex items-center px-3 border border-black"
          v-if="userStore.user"
          >{{ userStore.user.name }}</span
        >
        <h2 class="font-black mt-3 text-[rgb(0,30,98)]">Фамилия</h2>
        <span
          class="w-full h-10 flex items-center px-3 border border-black"
          v-if="userStore.user"
          >{{ userStore.user.surname }}</span
        >
        <h2 class="font-black mt-3 text-[rgb(0,30,98)]">Отчество</h2>
        <span
          class="w-full h-10 flex items-center px-3 border border-black"
          v-if="userStore.user"
          >{{ userStore.user?.patronymic || 'Отсутствует' }}</span
        >
        <h2 class="font-black text-[rgb(0,30,98)]">Почта</h2>
        <span
          class="w-full h-10 flex items-center px-3 border border-black"
          v-if="userStore.user"
          >{{ userStore.user.email }}</span
        >
        <h2 class="font-black text-[rgb(0,30,98)]">Телефон</h2>
        <span
          class="w-full h-10 flex items-center px-3 border border-black"
          v-if="userStore.user"
          >{{ userStore.user.phone }}</span
        >

        <button
            @click="logout"
                    class="w-full  h-12  border  border-red-600  text-red-600  font-black  text-lg  uppercase  rounded-xl  hover:bg-red-50
cursor-pointer transition-colors mt-6"
        >
          Выйти из аккаунта
        </button>
      </div>
    </div>
    <div class="flex items-start justify-center w-150 p-5 border border-black flex-col mt-10">
      <h1 class="font-black text-3xl mt-3 text-[rgb(0,30,98)]">История заказов</h1>
      <div class="w-full flex flex-col">
        <template v-if="orderStore.orderHistory && orderStore.orderHistory.length > 0">
          <AppProfileOrder
              v-for="orderItem in orderStore.orderHistory"
              :key="orderItem.OrderId"
              :order="orderItem"
          />
        </template>

        <span v-else class="text-gray-500 italic">История пока пуста (нет доставленных заказов)</span>
      </div>
    </div>
    <Teleport to="body">
      <div
        v-if="isModalOpen"
        class="fixed inset-0 bg-black/50 flex items-center justify-center z-[999] p-4"
      >
        <div
          class="bg-white w-full max-w-lg border-2 border-black p-8 flex flex-col relative rounded-2xl"
        >
          <button
            @click="isModalOpen = false"
            class="absolute top-4 right-4 text-2xl font-bold cursor-pointer"
          >

          </button>

          <h1 class="font-black text-3xl mb-6 uppercase text-[rgb(0,30,98)]">Изменить данные</h1>

          <div class="flex flex-col gap-4">
            <div class="flex flex-col">
              <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Имя</label>
              <input
                :value="userStore.user.name"
```

```
              ref="name"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="text"
            />
          </div>

          <div class="flex flex-col">
            <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Фамилия</label>
            <input
              :value="userStore.user.surname"
              ref="surname"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="text"
            />
          </div>

          <div class="flex flex-col">
            <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Отчество</label>
            <input
              :value="userStore.user.patronymic"
              ref="patronymic"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="text"
            />
          </div>

          <div class="flex flex-col">
            <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Почта</label>
            <input
              :value="userStore.user.email"
              ref="login"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="text"
            />
          </div>

          <div class="flex flex-col">
            <label class="text-xs uppercase mb-1 text-[rgb(0,30,98)]">Телефон</label>
            <input
              :value="userStore.user.phone"
              ref="phone"
              class="w-full h-12 border border-black px-3 focus:bg-gray-50 outline-none"
              type="text"
            />
          </div>

          <div class="flex flex-row justify-between gap-3 mt-6">
            <button
              @click="isModalOpen = false"
                class="w-full h-14 rounded-xl border border-black bg-blue-50 text-[rgb(0,30,98)] text-xl uppercase hover:bg-gray-100
cursor-pointer"
            >
              Отмена
            </button>
            <button
              @click="ed"
                       class="w-full  h-14  rounded-xl  bg-[rgb(0,30,98)]  text-white  font-black  text-xl  uppercase  hover:bg-gray-800
cursor-pointer"
            >
              Сохранить
            </button>
          </div>
        </div>
      </div>
    </div>
  </Teleport>
 </div>
</template>

<style scoped></style>
```

[OTHER] Project_ASICS\src\views\RegistrationView.vue

```
<script setup>
```

```
import { useRouter } from 'vue-router'
import useUserStore from '@/stores/user.js'
import { useTemplateRef } from 'vue'
import { useToast } from 'vue-toastification'

const name = useTemplateRef('name')
const surname = useTemplateRef('surname')
const patronymic = useTemplateRef('patronymic')
const phone = useTemplateRef('phone')
const email = useTemplateRef('login')
const password = useTemplateRef('password')

const toast = useToast()
const router = useRouter()
const userStore = useUserStore()

const regexValidation = (value, type) => {
  switch (type) {
    case 'password':
      return String(value).match(/(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}/)
    case 'email':
      return String(value).match(/^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]/)
    case 'phone':
      return String(value).match(/^(?:\+7|8|7)?(\d{10})$/)
  }
}

const reg = async () => {
  if (
    !name.value.value ||
    !surname.value.value ||
    !phone.value.value ||
    !email.value.value ||
    !password.value.value
  ) {
    toast.error('Заполните все поля')
    return
  }
  if (!regexValidation(email.value.value, 'email')) {
    toast.error('Неверный формат почты')
    return
  }
  if (!regexValidation(password.value.value, 'password')) {
    toast.error(
      'Пароль должен быть не менее 8 символов, содержать нижний и верхний регистр и число',
    )
    return
  }
  if (!regexValidation(phone.value.value, 'phone')) {
    toast.error('Неверный формат номера телефона')
    return
  }
  const user = {
    id: 0,
    name: name.value.value,
    surname: surname.value.value,
    patronymic: patronymic.value.value,
    phone: phone.value.value,
    email: email.value.value,
    password: password.value.value,
    role: '',
  }
  await userStore.registerUser(user)

  toast.success('Добро пожаловать!')

  if (userStore.user) {
    await router.push('/')
  }
}
</script>

<template>
  <div class="min-h-screen w-full flex items-center justify-center bg-gray-100">
    <div class="flex items-start justify-center w-200 p-5 border border-black flex-col">
```

```
      <h1 class="font-black text-3xl mt-3">Регистрация</h1>
      <span class="font-black mt-3">Фамилия</span>
      <input class="w-full h-10 border border-black" type="text" ref="surname" />
      <span class="font-black mt-3">Имя</span>
      <input class="w-full h-10 border border-black" type="text" ref="name" />
      <span class="font-black mt-3">Отчество</span>
      <input class="w-full h-10 border border-black" type="text" ref="patronymic" />
      <span class="font-black mt-3">Почта</span>
      <input class="w-full h-10 border border-black" type="email" ref="login" />
      <span class="font-black mt-3">Номер телефона</span>
      <input class="w-full h-10 border border-black" type="tel" ref="phone" />
      <span class="font-black">Пароль</span>
      <input class="w-full h-10 border border-black" type="password" ref="password" />
      <button class="w-full h-15 bg-black text-white text-2xl mt-5 w" @click="reg">
        Зарегистрироваться
      </button>
      <div class="flex justify-center items-center self-center flex-row mt-2">
        <h3>Есть аккаунта?</h3>
        <router-link class="text-black border-b-2 hover:text-blue-500" to="/authorization"
          >Войти в аккаунт</router-link
        >
      </div>
    </div>
  </div>
</template>

<style scoped></style>
```